

PIEDPIN

Let's try CISSP!

なぜCISSPを学ぶのか

- ✓ CISSP(Certified Information Systems Security Professional)とは、世界標準の**セキュリティ・プロフェッショナル認定**です。
- ✓ (ISC)²（国際情報システムセキュリティ認証コンソーシアム、International Information Systems Security Certification Consortium）が認定を行っています。
- ✓ 情報セキュリティ専門家が共通言語として利用している『(ISC)² CISSP CBK』を理解している者に与えられる資格です。
- ✓ 2004年6月に、米国規格協会（ANSI）より**ISO/IEC17024**の認証を受け、信頼度がより高くなりました。（https://japan.isc2.org/cissp_about.html）
- ✓ 2022年7月1日現在、日本のCISSPは**3,699**人です。

(ISC)² が展開する資格ソリューション



国際的に認められた情報セキュリティ・プロフェッショナル認定資格です。米国規格協会（ANSI）よりISO/IEC17024の認証を受けた。



ベンダーフリー・カントリーフリーの情報セキュリティの資格です。情報セキュリティに従事しないが、組織を理解し、適切なコミュニケーションを行う。



クラウドサービスを安全に利用するために必要な知識を体系化した資格です。



ソフトウェアの脆弱性に端を発したセキュリティインシデントの増加や、政府や重要インフラにおけるセキュアなソフトウェアニーズに対応する資格です。

(ISC)² CISSP CBK ドメイン



ドメイン 1
セキュリティとリスク
マネジメント



ドメイン 2
資産のセキュリティ



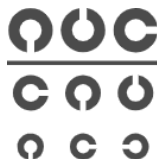
ドメイン 3
セキュリティアーキ
テクチャとエンジニ
アリング



ドメイン 4
通信とネットワー
クのセキュリティ



ドメイン 5
アイデンティティと
アクセスの管理



ドメイン 6
セキュリティの評
価とテスト



ドメイン 7
セキュリティの運用



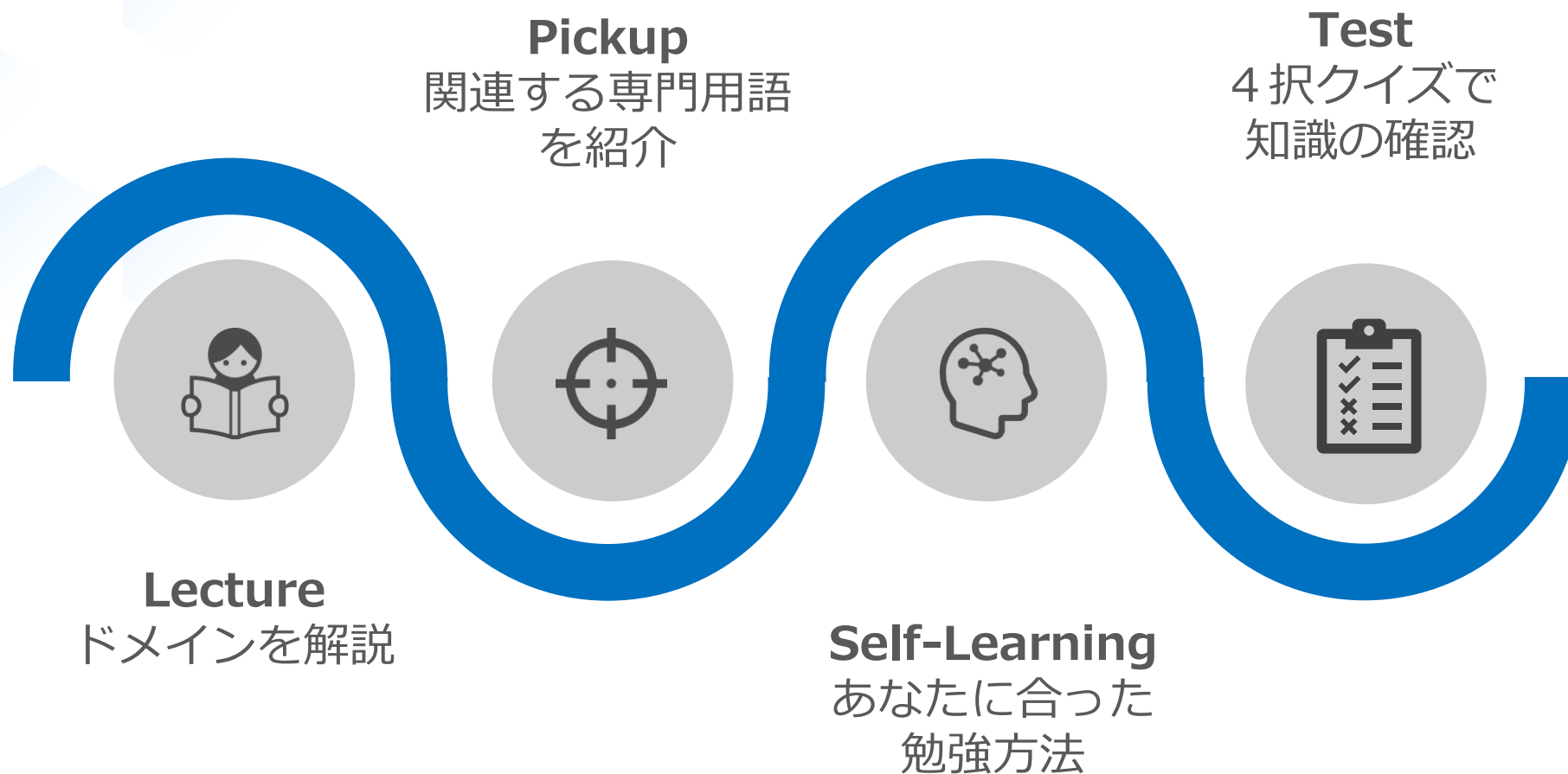
ドメイン 8
ソフトウェア開発
セキュリティ

あなたのゴールと本レッスンの内容

- ✓ 情報セキュリティ初心者のために設計されたCISSP講座
経験者からすると“当たり前”と感じる内容も含まれます。
- ✓ 本レッスンの対象は、CBKのドメイン 3 です。
他のドメインのレッスンは対象外となります。
- ✓ CISSP合格のためのアップグレードが必要です。
現代のセキュリティの関心、流行、考え方を織り込みます。



レッスンの流れ



ドメイン別試験比重



ドメイン 1
セキュリティと
リスクマネジメント

16%



ドメイン 2
資産のセキュリティ

10%



ドメイン 3
セキュリティアーキテク
チャとエンジニアリング

13%



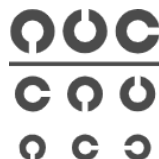
ドメイン 4
通信とネットワーク
のセキュリティ

13%



ドメイン 5
アイデンティティと
アクセスの管理

13%



ドメイン 6
セキュリティの評価
とテスト

12%



ドメイン 7
セキュリティの運用

13%



ドメイン 8
ソフトウェア開発
セキュリティ

10%



安全設計の原則

安全設計の原則

脅威モデリング（Threat Modeling）

● 脅威を認識すること

STRIDE

- マイクロソフトのPraerit GargとLoren Kohnfelderによって開発されたコンピューターセキュリティの脅威を識別するためのモデルです。

Spoofing	自分以外の誰かが成り済ますこと。
Tampering	意図しない形にデータを変更を行うこと。
Repudiation	何かをしていない、または責任を負っていないと主張すること。
Information disclosure	アクセス権限のない情報を入手すること。
Denial of service	サービス提供に必要なリソースが枯渇し、サービス停止すること。
Elevation of privilege	承認なく、上位の権限を取得すること。

DREAD

- マイクロソフトによって開発されたコンピューターセキュリティの脅威を識別するためのモデルです。
- Microsoftでは、2008年に利用が廃止されました。

Damage	攻撃のどの程度深刻さ
Reproducibility	攻撃の再現の容易さ
Exploitability	攻撃開始に必要な作業
Affected users	何人が影響を受けるか
Discoverability	脅威を発見する容易さ

安全設計の原則

最小特権 (Least Privilege)

● 権限は最小にすること

最小権限の原則 (Least Privilege)

- 権限は必要最低限であるべきというアクセス制御の原則です。
- 最小特権の原則では、すべてのユーザー、プログラム、サービスが対象になります。

知る必要性 (Need to Know)

- 知る必要性とは、機密性が高いと考えられる情報またはシステムへのアクセスに関して、アクセスを必要とする者に限られた期間だけ制限することです。

最小権限の原則

職権依存の アクセス権

職務や職位ベースでアクセス権が定義される。

あなたは、データベース管理者なので、ファイルアクセス権限はありません。



知る必要性

タスク依存の アクセス権

タスクベースでアクセス権が定義される。

本日の仕事では、ファイルアクセスは不要なのでアクセスできません。



安全設計の原則

最小特権 (Least Privilege)

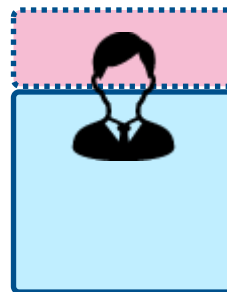
● 権限適正化のメリット

情報漏洩の抑制

- アクセス権を絞ることで、個人の権利が保護されます。
- 権限が適切に割り当てられていないと、同じ会社の他人の給与や住所を漏洩する事象につながります。

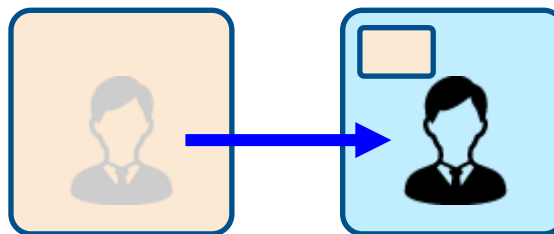
動作の安定

- 不要な権限がなくなれば、その分例外処理が減るでしょう。
- システム開発や動作テストでは、不要な権限がないほうが、想定外の挙動を減らすことができます。



権限超過

新しい職に就いたとき、管理者が間違えて通常よりも強いアクセス権限を付与していた。



権限クリープ

他のチームに配属になり、職権が変わったが、前の権限が残っていた。

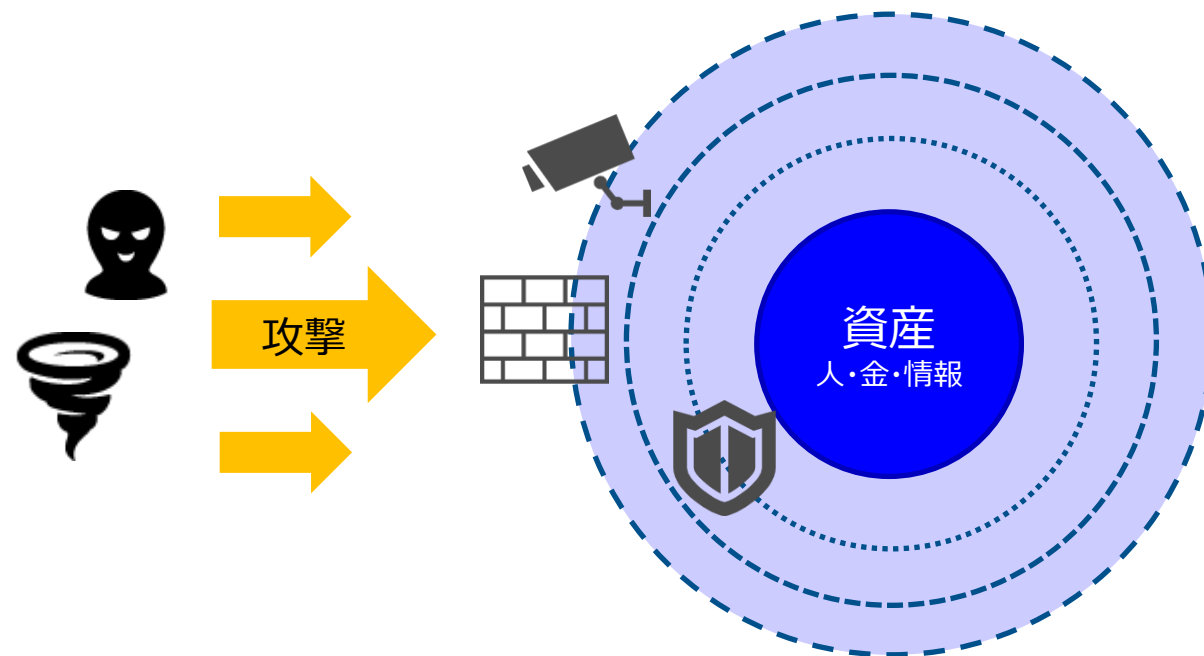
安全設計の原則

多層防御 (Defence in Depth)

● 防御策は多層的にすること

多層防御

- セーフガード（セキュリティ防御策）は、多層であるべきという考え方です。
- 特定の脆弱性のみに対応した高価な製品のように一面のみに特化したセキュリティを講じることは得策ではありません。



セーフガードの多層にすることで抜け目なく保護する

安全設計の原則

安全なデフォルト (Secure Default)

●デフォルトでも安全に利用できること

デフォルトの安全性

- ソフトウェアやハードウェアを最初にインストールしたときの設定（デフォルトの設定）でも、安全に動作できるようにしましょう。
- ただし、パスワードの再設定など利用者自身で設定するものを許さないわけではない。

未設定でも安全に



利用者は、セキュリティ意識や製品知識が高いわけではありません

安全設計の原則

安全な失敗 (Fail Securely)

● 障害時でも安全な状態をキープすること

障害時の安全確保

- フェールセーフ (Fail Safe) は、部品の故障が発生した際に、なるべく安全な状態に移行するような仕組みにすることです。
- フェールソフト (Fail Soft) とは、部品が故障した後もシステムの動作を継続させることです。

プログラミングとして安全な失敗

- Javaのコーディングにおいては、try catchステートメントにより、例外処理を取り扱えます。tryブランチにエラーが発生する可能性のあるコードを記載し、catchブランチにエラーが発生した場合に実行されるコードを記載します。

フェールセーフ



安全を優先

例：信号は赤信号になり、交通事故を起こさせなくする。

フェールソフト



継続を優先

例：飛行機の故障した部品は破棄し、稼働は継続する。

安全設計の原則

職務分離 (Segregation of Duties)

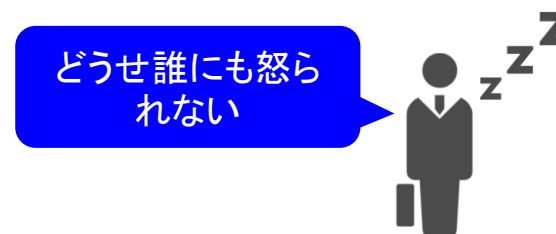
● 職務は分離すること

職務分離

- 職務権限を分けることでけん制関係を作ることです。
- 互に訴求できる関係性を維持することで、作業を分担しながら、信頼のある業務フローを作ります。



作業品質が劣ります



不誠実な作業を助長します

安全設計の原則

シンプルかつ小規模に保つ（Keep It Simple and Small）

● シンプルかつ小規模に保つこと

シンプルかつ小規模に保つ

- システムやプロセス、製品の設計や開発において、シンプルでコンパクトな設計を目指す原則です。
- 複雑なものよりもシンプルなもののほうが外形的に理解しやすく、保守しやすく、信頼性が高いという前提に基づいています。
- 反対に、複雑性は問題を増長します。

多機能で操作が難しい



柔軟な開発
開発速度
利便性



シンプルかつ小規模



柔軟な開発
開発速度
利便性



安全設計の原則

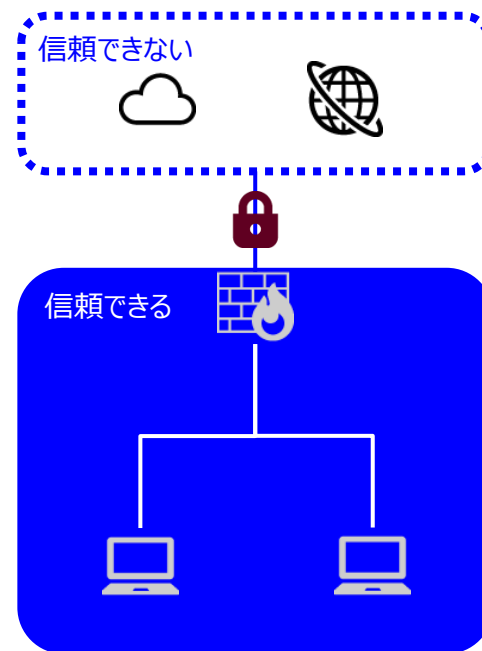
ゼロトラスト（Zero Trust）

●信頼されるネットワーク区分を前提にしない

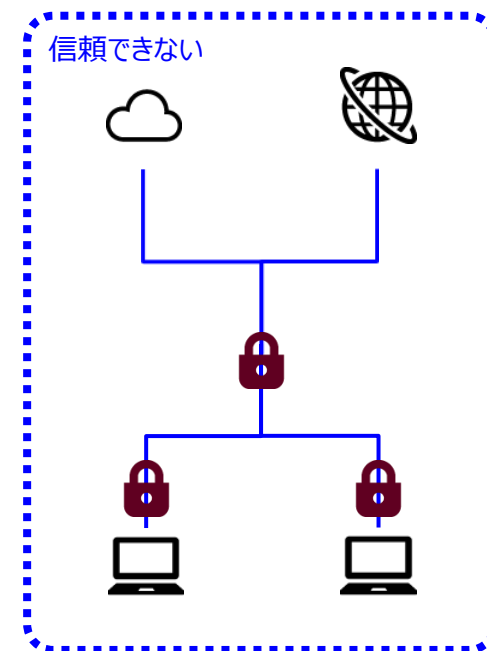
ゼロトラスト

- いかなるネットワーク環境も信頼されないというセキュリティの概念です。
- 「信頼と検証」のセキュリティ・モデルはもはや十分ではなく、「決して信用せず、常に検証する」ことが求められます。
- 従来の信頼境界に依存したアプリケーション設計からの脱却でもある。ファイヤーウォールやエアギャップなどのネットワークの分断手法は、有効であり利用されています。

従来の領域の
ネットワークセキュリティ



ゼロトラストの
ネットワークセキュリティ



安全設計の原則

プライバシー・バイ・デザイン (Privacy by Design)

● プライバシーを保護すること

プライバシー・バイ・デザイン

- プライバシーとデータ保護をシステムやプロセスの設計段階から考慮し、組み込むことを目的としたアプローチです。
- 1990年代にカナダの情報・プライバシー保護委員会のコミッショナーであるアン・カヴキアン (Ann Cavoukian) によって提唱されました。

PbDの要素	説明
事後的ではなく事前的	事前にプライバシー侵害を予防する先見的なアプローチです。
初期設定としてのプライバシー	システム内に組み込まれた自動的なプライバシー保護を提供します。
デザインに組み込まれるプライバシー	システム設計時からプライバシーを組み込む手法です。
ゼロサムではなくポジティブサム	プライバシーとセキュリティを両立するWin-Winのアプローチです。
すべてのライフサイクルを保護	データのライフサイクル全体を通じて安全性を確保する。
公開の維持	全ての関係者に可視的かつ透明で信頼できるプライバシー保護を目指す。
利用者中心主義を維持する	強力なプライバシー保護を設計し、利用者中心を維持する。

安全設計の原則

責任の共有 (Shared Responsibility)

● 責任範囲を明確にすること

- オンプレとは、自社内のサーバーや設備でシステムを運用する形態です。
- IaaS (Infrastructure as a Service) とは、クラウド上に仮想化されたサーバを提供します。AWSのEC2などが一例です。
- PaaS (Platform as a Service) とは、アプリケーションの実行環境を提供し、アプリケーションは個別に開発することが可能です。OpenShift、Cloud Foundryなどが一例です。
- SaaS (Software as a Service) とは、サービス提供者は開発不要で、機能の取捨選択でサービス展開可能です。Gmail、Zoom、Slackなどが一例になります。



管理範囲が責任範囲になります

安全設計の原則

セキュアアクセスサービスエッジ（Secure Access Service Edge）

● 維持可能なセキュリティにすること

- セキュアアクセスサービスエッジ（SASE）は、SD-WANとゼロトラストセキュリティを統合したセキュリティフレームワークです。
- クラウドデリバリー型のプラットフォームにより、安全にユーザーやシステムをアプリやリソースに接続します。
- SASEの主要な目的は、シームレスなユーザー体験、最適化された接続、包括的なセキュリティを提供し、いつでもどこでもデバイスやリモートシステムがアクセスできるようにすることです。

SASEの要素	説明
セキュア Web ゲートウェイ (SWG)	ウェブトラフィックを監視・制御し、マルウェアや不正なウェブサイトからの保護を提供する。
クラウドアクセスセキュリティブローカー (CASB)	クラウドサービス利用に対するセキュリティポリシーの適用と管理を行う。
ゼロトラストネットワークアクセス (ZTNA)	ゼロトラストの原則に基づき、ユーザーとデバイスに対する認証と認可を実施する。
ソフトウェアで定義されたワイド エリア ネットワーク (SD-WAN)	アプリケーションパフォーマンスを最適化し、インターネットやクラウドへの接続を安全に管理する。
サービスとしてのファイアウォール (FWaaS)	クラウドベースのファイアウォールを提供し、ネットワークトラフィックをフィルタリングする。



セキュリティモデルの 基本概念

セキュリティモデルの基本概念

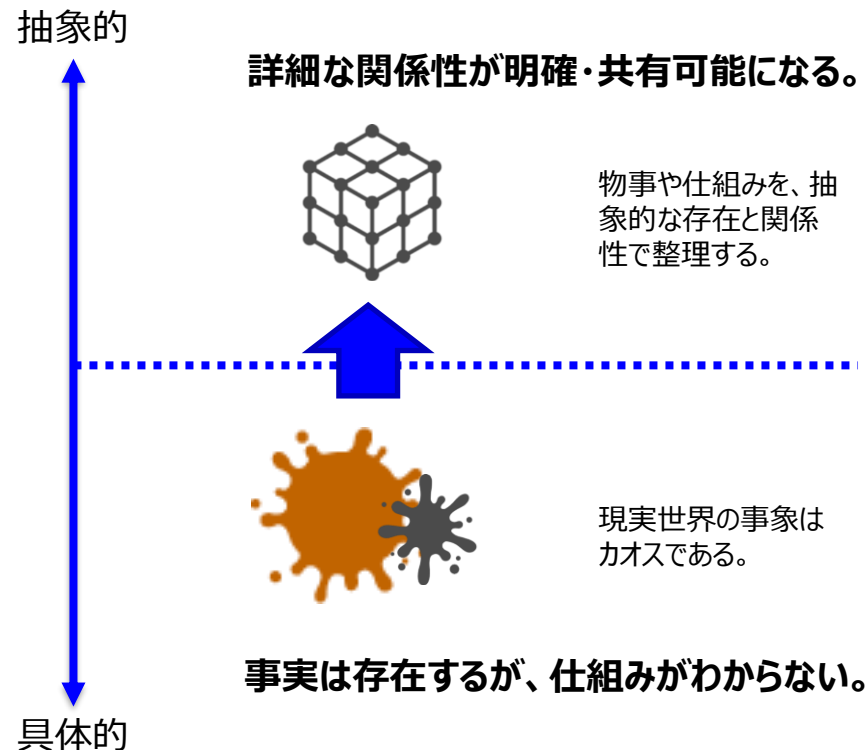
セキュリティモデル

● セキュリティモデル・アクセス制御モデル

- 組織情報の機密性と完全性を守るために、主体と対象と情報の流れによって構成されるモデルです。
- 上司・部下、閲覧・編集の権限の関係性において、どのようなルールを設ければ機密性・完全性が守られるのかを論理的に整理します。

モデル化

- 世の中で起こる事象はカオスです。それは1組織の中で行われるデータのやり取りでも同じです。
- モデル化とは、物事の構造や仕組み、抽象的な存在と関係性に焦点を絞り、表すことです。



セキュリティモデルの基本概念

セキュリティモデル

● Bell-LaPadula (BLP) モデル

データの漏洩が起きないようにするための機密性のセキュリティモデルです。

シンプルセキュリティ属性

- 上位のドキュメントを閲覧（Read Up）ないことです。

スターセキュリティ属性

- 下位のドキュメントを書き換えられ（Write Down）ないことです。

強スターセキュリティ属性

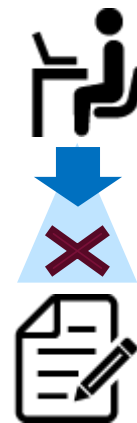
- 上位のドキュメントを閲覧（Read Up）なく、かつ下位のドキュメントを書き換えられ（Write Down）ないことです。

シンプルセキュリティ属性



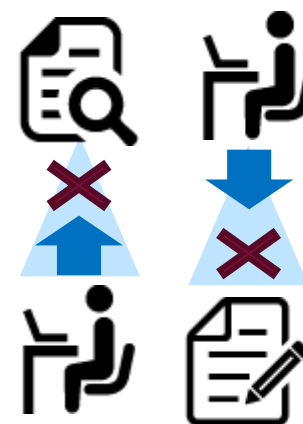
• No Read Up

スターセキュリティ属性



• No Write Down

強スターセキュリティ属性



• No Read Up
• No Write Down

セキュリティモデルの基本概念

セキュリティモデル

● Bibaモデル

データが勝手に変更されないことを示す完全性セキュリティモデルです。

シンプル完全性プロパティ

- 部下のドキュメントが見ること（Read Down）がないことです。

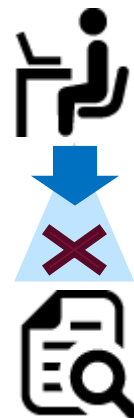
スター完全性プロパティ

- 上司のドキュメントを書き換えること（Write Up）がないことです。

Invocationプロパティ

- 上司のドキュメントが見ること（Read Up）、かつ上司のドキュメントを書き換えること（Write Up）がないことです。

シンプル完全性プロパティ



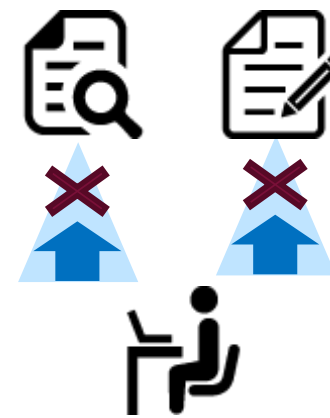
• No Read Down

スター完全性プロパティ



• No Write Up

Invocationプロパティ



• No Read Up
• No Write Up

セキュリティモデルの基本概念

セキュリティモデル

● 格子ベースアクセスコントロール

一つの主体が複数のアクセス権を持ち得ることを想定し、アクセスコントロールをある条件下で取りえるすべての関係性として考える中立型のセキュリティモデルです。

取りえる組み合わせ

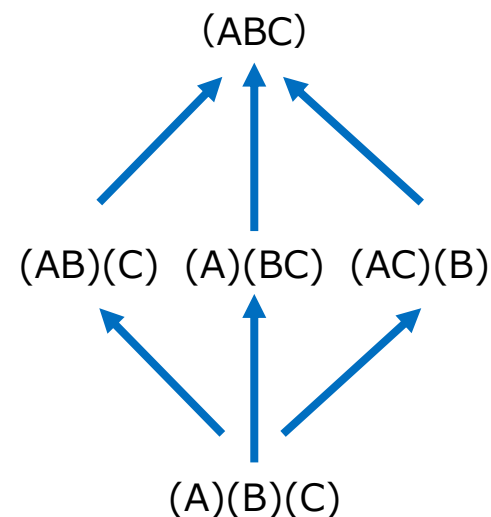
- A君・B君の場合：(A)(B)、(AB)、()
- A君・B君・C君の場合：(A)(B)(C)、(AB)(C)、(A)(BC)、(AC)(B)、(ABC)、()

構成する関係性

- A君・B君の場合：(A)(B) $\rightarrow$ (AB)
- A君・B君・C君の場合：(AB)(C)、(A)(BC)、(AC)(B) $\rightarrow$ (ABC)

いかなるグループを作るとしても以下の関係性が成り立っている。

$$(A)(B)(C) \rightarrow (AB)(C)、(A)(BC)、(AC)(B) \rightarrow (ABC)$$



格子ベースアクセスコントロールの状態遷移

セキュリティモデルの基本概念

セキュリティモデル

● Graham-Denning Model

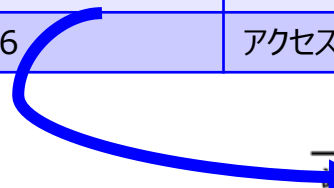
- サブジェクトとオブジェクトに対して、8つのルールをマッピングする中立型のモデルです。

R1	アクセス権を譲渡
R2	アクセス権を付与
R3	アクセス権の削除
R4	アクセス権の確認
R5	アクセス対象の削除
R6	アクセス対象の作成
R7	アクセス主体の削除
R8	アクセス主体の作成

● Harrison-Ruzzo-Ullman Model

- サブジェクトとオブジェクトに対して、6つのルールをマッピングする中立型のモデルです。
- アクセス制御行列モデルに置き換えた時、グラハムデニングモデルと同じ事象を説明できないことを示し、アクセス制御行列モデルのアクセス制御に対する決定不能性を証明しました。

R1	アクセス対象の作成
R2	アクセス対象の削除
R3	アクセス主体の作成
R4	アクセス主体の削除
R5	アクセス制御マトリクスに追記
R6	アクセス制御マトリクスの削除



	ユーザA	ユーザB	ユーザC	ユーザD
出し	○	○	○	○
実行	○	○		
削除	○		○	

セキュリティモデルの基本概念

セキュリティモデル

● Clark-Wilson Model

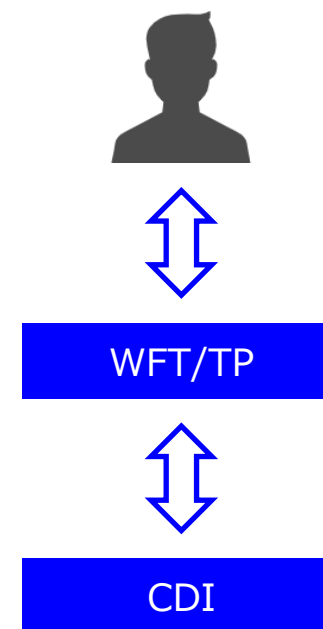
- ユーザーとデータの間にプログラムの存在を定義した完全性のセキュリティモデルです。
- 完全性を損なってはいけないデータをCDI (Constrained Data Item) 、完全性を損なう可能性があるデータをUDI (Unconstrained Data Item) として完全性の保証有無によってデータを分類します。

2つのプログラム処理

- IVP (Integrity Verification Procedure) とは、CDIが正しいことを確認するプログラムです。
- WFT (定型的トランザクション、Well-Formed Transactions) / TP (Transformation Procedure) とは、ある状態 (CDI or UDI) を正しい状態 (CDI) に変更するプログラムです。

セグリゲーション (Separation of Duties)

- トランザクションや処理に紐づく操作は独立しているべきであるという制約です。



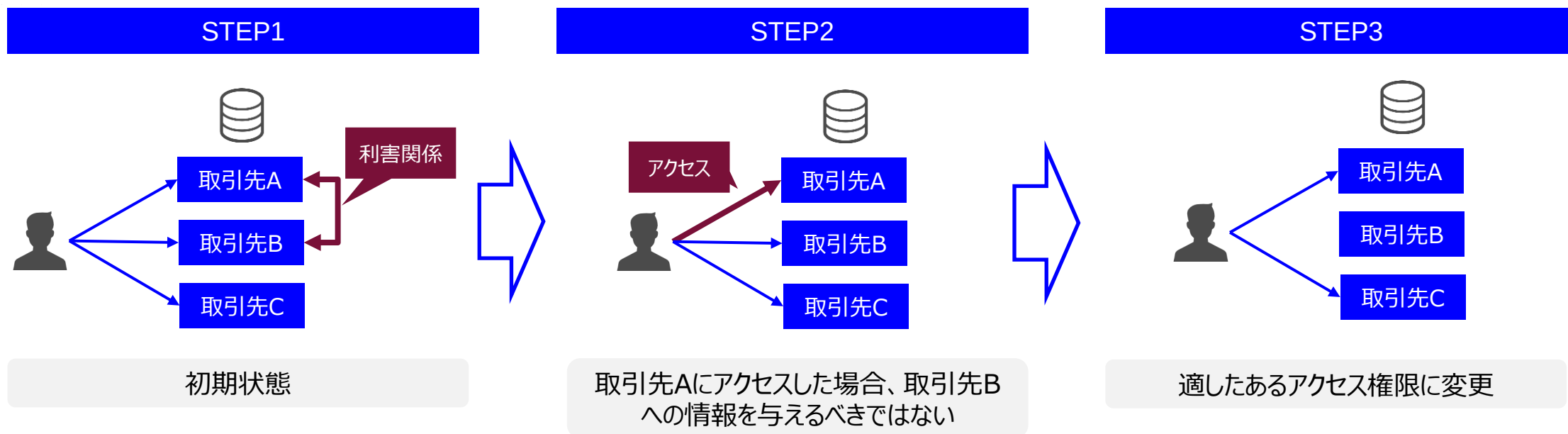
クラークウィルソンモデルの関係図

セキュリティモデルの基本概念

セキュリティモデル

● Brewer-Nash Model

- 将来的なアクセスを予想しアクセス制御を決めるのではなく、過去に遡ってアクセスできる範囲を狭めることを想定した機密性のセキュリティモデルです。
- このモデルでは、管理者とデータリソースが存在します。そして、管理者が一度アクセスしたデータセットに対して、依存関係があるデータセットには今後はアクセス不可となります。



セキュリティモデルの基本概念

エンタープライズアーキテクチャ

● TOGAF

- アーキテクチャ開発方法（ADM、Architecture Development Method）というアーキテクチャプロセス内の一連の実行可能なステップを提供します。
- エンタープライズアーキテクチャの開発におけるさまざまな組織のユースケースに適用できる一般的で適応性のある方法論ですTOGAF-ADMが反復サイクルを実行して、新しいエンタープライズアーキテクチャ要件を管理および開発することを考えると特に役立ちます。

項目	内容
アーキテクチャビジョン	プロジェクトの範囲、利害関係者を特定し、必要な承認を取得する初期フェーズ。ビジネスの目標と推進要因が明確化される。
ビジネスアーキテクチャ	アーキテクチャビジョンを満たすためにプロセスを定義するフェーズ。複数の利害関係者と協力して、詳細なビジネス分析とモデリングが実行する。
情報システムアーキテクチャ	アーキテクチャビジョンを満たすためにデータを定義するフェーズ。データのターゲット設計原則を決定する。
テクノロジーアーキテクチャ	アーキテクチャビジョンをサポートするために必要な基幹システムを定義するフェーズ。IT部門と経営幹部とともに決定する。
機会とソリューション	アーキテクチャビジョンが確定すると、さまざまな実装シナリオを決定する。技術的側面とビジネス的側面の両方が考慮すべきである考え。
移行計画	機能とソリューションにおける実装戦略（コスト、機会、リスク）に基づいて、移行計画を決定するフェーズ。
実装ガバナンス	変更要求、コンプライアンス評価、およびソリューションビルディングブロックを説明するフェーズ。
アーキテクチャの変更管理	新しい変更を組み込むための新しい変更管理プロセス。

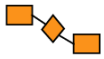
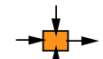
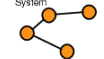
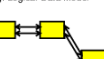
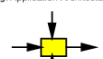
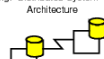
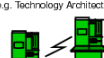
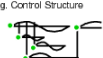
セキュリティモデルの基本概念

エンタープライズアーキテクチャ

● ザックマン (Zachman) フレームワーク

- 各職権に対して、何を、どのように、どこで、誰が、いつ、なぜを決めるエンタープライズアーキテクチャです。
- エンタープライズアーキテクチャとは、事業目標を達成するために経営体制を整えることです。

視点	内容
エグゼクティブの視点	ビジネスの目的と戦略を説明するスコープ
ビジネス管理の視点	組織で採用されているエンタープライズモデルおよびプロセス
アーキテクトの視点	ビジネス要件がどのように満たされるかを説明するシステムロジック
エンジニアの視点	システムの選択を実装するためにテクノロジーソリューションがどのように使用されるか
下請け業者の視点	特定のテクノロジーに関する個別要件
エンタープライズパースペクティブ	運用においてユーザーから見たときのシステムの機能

	DATA <i>What</i>	FUNCTION <i>How</i>	NETWORK <i>Where</i>	PEOPLE <i>Who</i>	TIME <i>When</i>	MOTIVATION <i>Why</i>
SCOPE (CONTEXTUAL)	List of Things Important to the Business 	List of Processes the Business Performs 	List of Locations in which the Business Operates 	List of Organizations Important to the Business 	List of Events Significant to the Business 	List of Business Goals/Strat 
Planner	ENTITY = Class of Business Thing	Function = Class of Business Process	Node = Major Business Location	People = Major Organizations	Time = Major Business Event	Ends/Mean = Major Bus. Goal/ Critical Success Factor
ENTERPRISE MODEL (CONCEPTUAL)	e.g. Semantic Model 	e.g. Business Process Model 	e.g. Business Logistics System 	e.g. Work Flow Model 	e.g. Master Schedule 	e.g. Business Plan 
Owner	Ent = Business Entity ReIn = Business Relationship	Proc = Business Process IO = Business Resources	Node = Business Location Link = Business Linkage	People = Organization Unit Work = Work Product	Time = Business Event Cycle = Business Cycle	End = Business Objective Means = Business Strategy
SYSTEM MODEL (LOGICAL)	e.g. Logical Data Model 	e.g. Application Architecture 	e.g. Distributed System Architecture 	e.g. Human Interface Architecture 	e.g. Processing Structure 	e.g. Business Rule Model 
Designer	Ent = Data Entity ReIn = Data Relationship	Proc = Application Function IO = User Views	Node = I/S Function (Processes, Storage, etc.) Link = Line Characteristics	People = Role Work = Deliverable	Time = System Event Cycle = Processing Cycle	End = Structural Assertion Means = Action Assertion
TECHNOLOGY MODEL (PHYSICAL)	e.g. Physical Data Model 	e.g. System Design 	e.g. Technology Architecture 	e.g. Presentation Architecture 	e.g. Control Structure 	e.g. Rule Design 
Builder	Ent = Segment/Table/etc. ReIn = Pointer/Key/etc.	Proc = Computer Function IO = Data Elements/Sets	Node = Hardware/System Software Link = Line Specifications	People = User Work = Screen Format	Time = Execute Cycle = Component Cycle	End = Condition Means = Action
DETAILED REPRESENTATIONS (OUT-OF-CONTEXT)	e.g. Data Definition 	e.g. Program 	e.g. Network Architecture 	e.g. Security Architecture 	e.g. Timing Definition 	e.g. Rule Specification 
Sub-Contractor	Ent = Field ReIn = Address	Proc = Language Stmt IO = Control Block	Node = Addresses Link = Protocols	People = Identity Work = Job	Time = Interrupt Cycle = Machine Cycle	End = Sub-condition Means = Step
FUNCTIONING ENTERPRISE	e.g. DATA	e.g. FUNCTION	e.g. NETWORK	e.g. ORGANIZATION	e.g. SCHEDULE	e.g. STRATEGY

SemanticScholar,
<https://www.semanticscholar.org/paper/The-Zachman-Framework-For-Enterprise-Architecture/458b055d4884797ec7b2dd758b2ed06c545fda85>



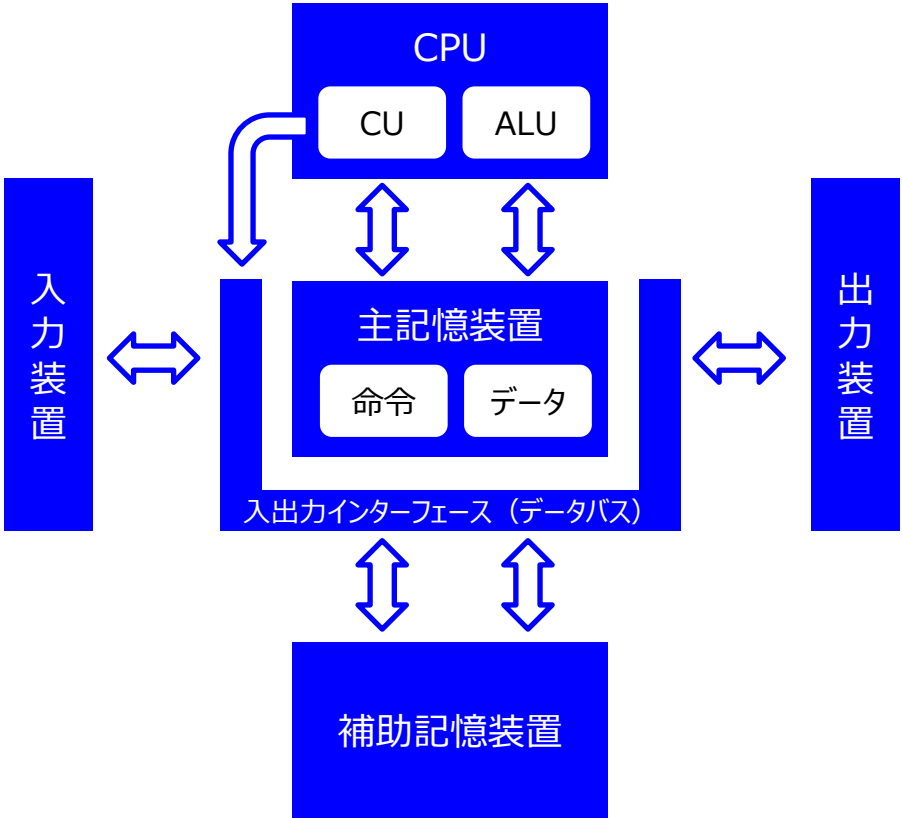
コンピューターの 構成要素

コンピューターの構成要素

ハードウェア

●コンピューターの構成要素

機能	装置	概要
入力	入力装置	データを入力する入口であり、マウスやキーボード、カードの読み取り機は入力装置です。
演算	中央処理装置 (CPU)	算術論理演算装置 (ALU) は、算術論理演算装置は主記憶装置内のデータを参照し、加減乗除算や比較を行います。
制御		制御装置 (Control Unit) は、主記憶装置内の命令を参照しデータバスを制御し、周辺機器の制御を行います。
記憶	記憶装置	主記憶装置 (内部記憶装置) には、入力されたデータと命令、処理された結果が記憶されます。主記憶装置内のデータは電源が切られると消えるため、電源が切れても必要なデータは記憶内容が失われない補助記憶装置 (外部記憶装置) に保存します。
出力	出力装置	データを入力する出口であり、ディスプレイとプリンターなど、処理されたデータは最終的にはコンピュータの利用者に返されます。



コンピューターの構成要素

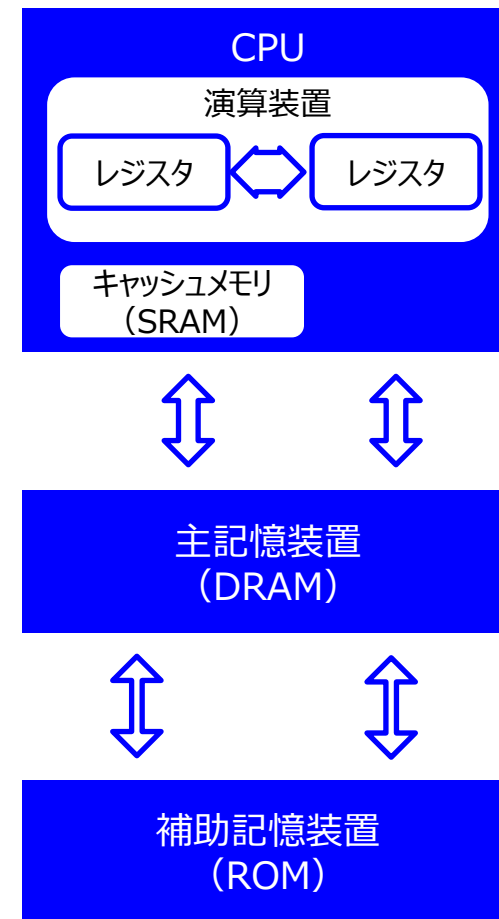
ハードウェア

● 記憶装置

- 記憶装置の使い分けは、記憶装置自体の容量（大きさ）とサイクルタイムの期間（スピード）で整理すると良いでしょう。
- サイクルタイムは、制御装置が読み書きの指令を出してから次の読み書きが可能になるまでの準備が完了するまでの時間を指します。
- サイクルタイムは、利用頻度に合わせてCPUに近いほど良くなっていきます。CPUの中にはレジスタと呼ばれる小容量の記憶装置が設けられています。このレジスタが最も性能が良く、補助記憶装置に近づくほど遅くなるように設計されています。レジスタと主記憶装置の性能差を埋めるためのキャッシュメモリが利用されます。

サイクルタイムは実質読み書き時間

ICメモリにおいては準備自体の時間がほとんどないため、実質的に制御装置が読み書きの指令を出してから読込動作を完了するまでの時間がサイクルタイムになります。



コンピューターの構成要素

ハードウェア

●ROM (Read Only Memory)

- 書き込み不可・読み出しのみ可能なメモリを指します。ゲームソフトや音楽CDなどが一例です。
- 不揮発性メモリであるROMは、電源がなくても整合性を維持します。

種類	概要
PROM (Programmable Read Only Memory)	読み出し時とは異なる装置を用いて内容の書き込みができる。
EPROM (Erasable Programmable Read Only Memory)	繰り返し内容の書き換えができ、紫外線の照射によりデータの一括消去が可能です。
EEPROM (Electrically Erasable Programmable Read-Only Memory)	繰り返し内容の書き換えができ、電氣的に内容の書き換えができます。大量のデータを格納する用途では、EEPROMの一種であるフラッシュメモリの方が経済的です。

コンピュータの構成要素

ハードウェア

● RAM (Random Access Memory)

- CPUが何らかの処理を行ったり、画面上にデータを表示したり、処理するデータに対して割り当てられる作業用のメインメモリです。
- RAMのデータは頻繁に書き換えられ、電源が切れるとデータも消えます。CPUから直接アクセスでき、現在実行中のプロセスの命令とデータを保持するために使用されます。

項目	用途	揮発性	概要
SRAM (Static Random Access Memory)	レジスタ キャッシュ	揮発性	フリップフロップ回路で電流の流れ方でデータを保持しているため、リフレッシュの必要ありません。
DRAM (Dynamic Random Access Memory)	メインメモリ	揮発性	情報を保持するために定期的に情報を読み出して書き込む動作（リフレッシュ）が必要です。
EEPROM フラッシュメモリ	ディスク	不揮発性	電氣的に消去および再プログラムが可能な不揮発性メモリの一種です。

なにがRandom ?

「ランダム」とは、CPUがメモリ内の任意の場所にランダムにアクセス（ジャンプ）できることを意味します。テープなどのシーケンシャルメモリは、オフセットゼロから開始して、メモリの目的の部分までメモリを順番に読み取ります。

コンピューターの構成要素

ハードウェア

● 中央処理装置 (CPU、Central Processing Unit)

算術論理演算装置 (Arithmetic Logic Unit)

- 主記憶装置内のデータを参照し、加減乗除算や比較を行います。

制御装置 (Control Unit)

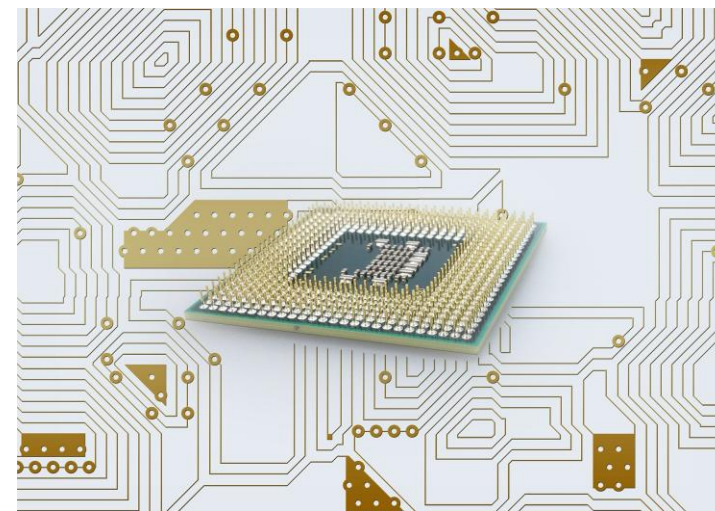
- 主記憶装置内の命令を参照しデータバスを制御し、周辺機器の制御を行います。

クロック

- コンピューター内の動作タイミングを合わせる機構です。一定の規則でクロック信号を発生させる回路です。

キャッシュメモリ

- CPUが命令をフェッチして実行するときにCPUに追いつくために必要です。CPUが最も頻繁に使用するデータは、キャッシュメモリに保存されます。

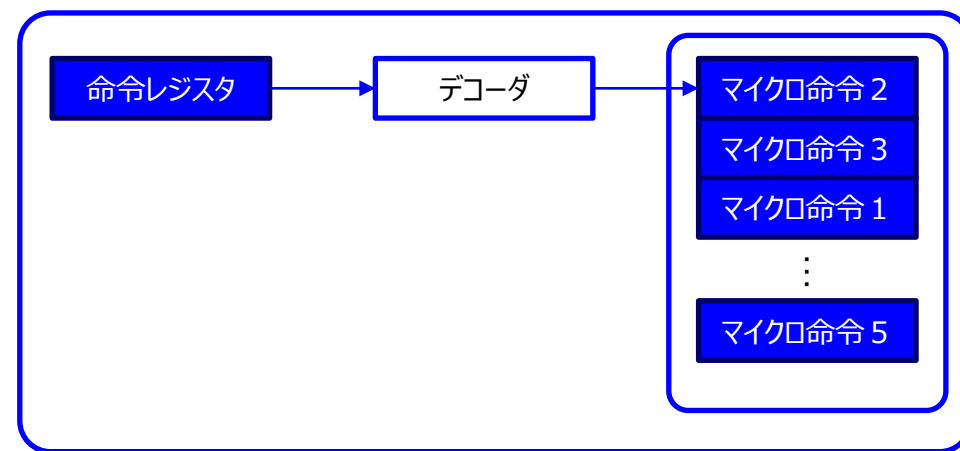


コンピュータの構成要素

ハードウェア

● マイクロプログラム

- マイクロプログラムとは、機械語命令の実行に必要なCPUの動作を、レジスタ間のデータ転送、シフト操作、演算回路、バスを直接制御を組み合わせた命令（マイクロコード）に変換したものです。
- ある命令コードをマイクロコードに変換する方式をマイクロプログラム制御方式と言います。論理回路全体の設計が単純化されます。
- CPUは機械語の命令コードを一つずつ実行していきますが、その命令コードを処理するための論理回路が必要です。しかし、命令コードに合わせていくつもの論理回路を持つておくことは難しいため、マイクロコードと呼ばれるさらに分解した形で表現する必要があるのです。



現在はマイクロプログラムは使われない

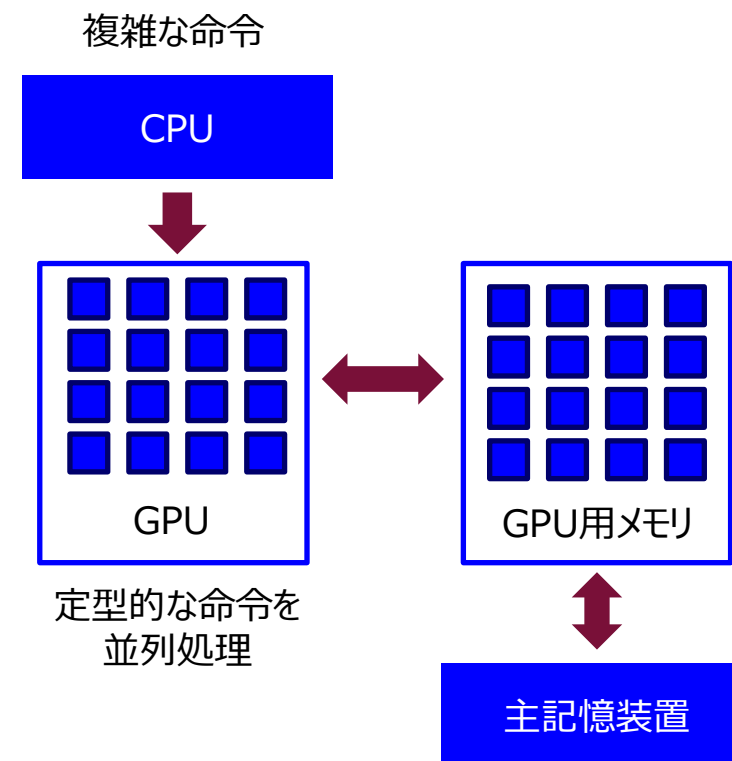
RISC方式のプロセッサでは、単純な動作をする命令セットを扱っています。パフォーマンスの向上や消費電力の削減を目指し、シンプルな命令セットを直接ハードウェアで実行するRISCアーキテクチャにシフトしており、マイクロプログラム制御方式はあまり利用しません。

コンピューターの構成要素

ハードウェア

●GPU (Graphics Processing Unit)

- 画像処理などは大量のデータに座標や画素の演算を繰り返します。高画質の3D対戦ゲームであれば、求められる性能は膨大です。
- データを分割し、並行処理を行う専用の演算装置をGPUと言います。内部に何千もの演算装置があります。
- 画像処理だけではなく、AIのディープラーニングなどにも利用されています。

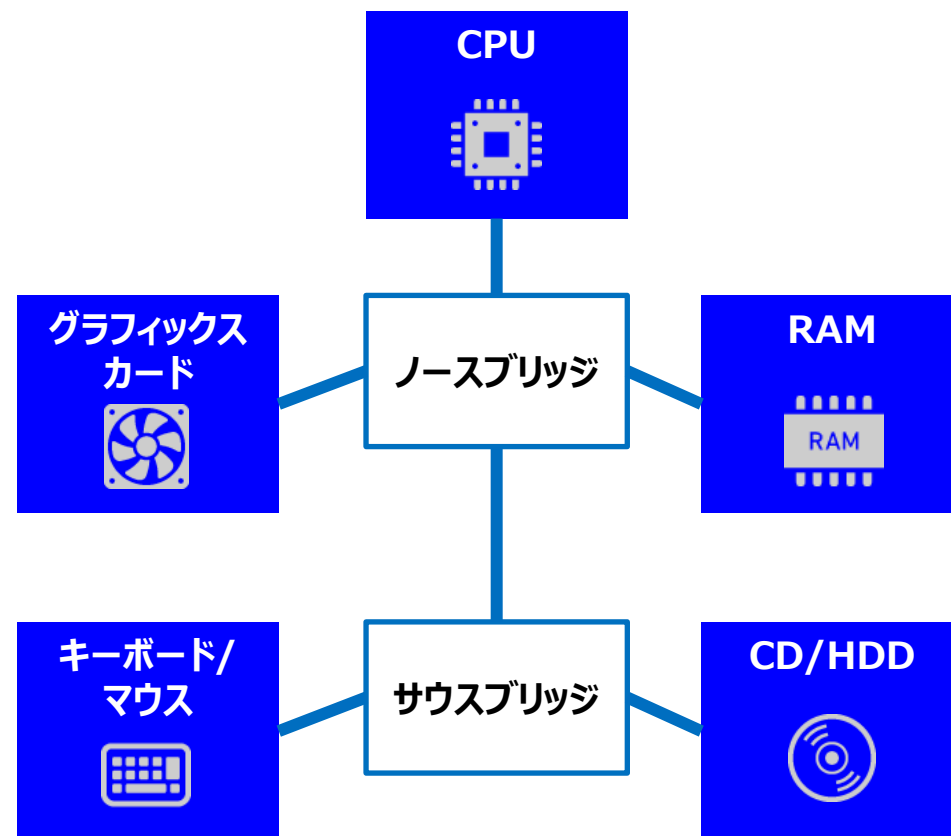


コンピュータの構成要素

ハードウェア

● システムユニット

- マザーボード (Motherboard) は、CPU、メモリスロット、ファームウェア、PCIスロットからなります。
- 周辺機器 (Peripherals) とは、マウス、キーボード、モニター、システムユニットに接続されているものすべてを指します。
- コンピュータバス (Regular Computer Bus) とは、コンピュータの主要な通信チャネルです。内部ハードウェアと入出力デバイス、キーボード、マウス、モニター、Webカメラなどの間で通信します。
- ノースブリッジとは、メモリーの読み書きや動画像の表示のチップなどのCPUに近くを担当します。サウスブリッジとは、ハードディスクの読み書き、オーディオ、LANなどのチップなどの、CPUから見て遠くを担当します。
- ノースブリッジをCPU自体 (Intel Sandy Bridge / AMD Fusion) に統合する動きもあります。



システムユニットとその配置

コンピューターの構成要素

ファームウェア

●フラッシング攻撃（Phlashing） / 永続的なサービス拒否攻撃（PDOS、Permanent Denial of Service）

- **永続的なサービス拒否攻撃**とは、ハードウェア破壊のための攻撃になります。ハードウェアを交換または再インストールする以外に方法がなく、データが永久に失われる可能性もあります。
- **フラッシング攻撃**は、この永続的なサービス拒否攻撃をリモートで行います。

フラッシング攻撃の一例

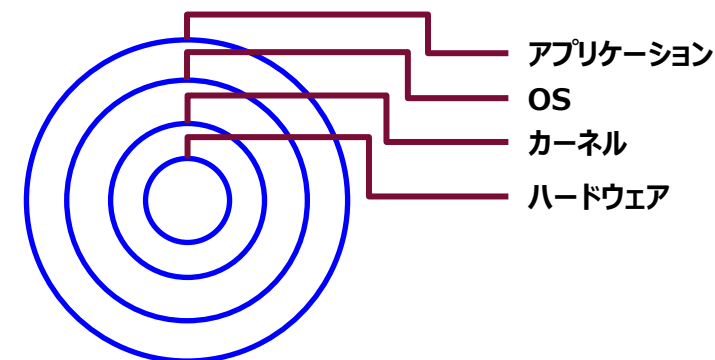
- ファームウェアのアップデート口に対して、直接大量に通信を発生させハードウェアを停止させます。
- BIOSダウンロード時に悪意のあるBIOSをダウンロードさせ、脆弱なコンピューター起動をさせます。

コンピューターの構成要素

ソフトウェア

●階層化（Layering）

- 上層は一つ下層を経由し、その例外を許さないモデルです。
- 例えば、ハードウェアと直接やり取りするアプリケーションは存在してならず、必ずOSとカーネルを経由する。ハードウェアを変更したとしてもアプリケーションには無関係であることも示します。



階層化モデル

●リングモデル（Ring Model）

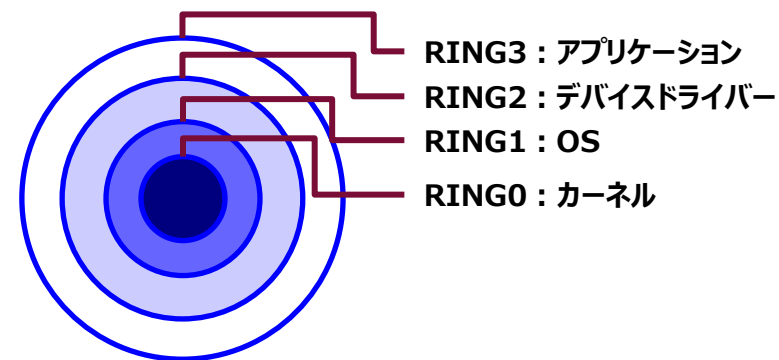
- リングモデルは、権限が階層化されたシステム設計を促進し、セキュリティや安定性を向上させます。
- システム全体の信頼性を確保するために、中心部（リング0）の保護が最優先されます。
- CPUはプログラムの実行モードを切り替えます。

弱い権限



- ◁アプリケーション操作権限
- ◁デバイスドライバー操作権限
- ◁ OS操作権限
- ◁カーネル操作権限

強い権限



リングモデル



コンピューターの構成要素

ソフトウェア

●オペレーティングシステム（OS、Operating System）

- コンピューターは、プログラム（ソフトウェア）の命令に従って、主記憶装置（ハードウェア）からプログラムを読み込み、CPU（ハードウェア）に命令を実行させたりと、ソフトウェアの内容をハードウェアに対する指示に変換しなければなりません。
- 更にOSの機能を細分化すると、ソースコードを機械語に解釈するための言語プロセッサ、CPUの処理管理を行う制御プログラム、ファイル生成や整列などのプログラム実行としての共有で必要となるサービスプログラムに区分できます。

コンピュータの構成要素

ソフトウェア

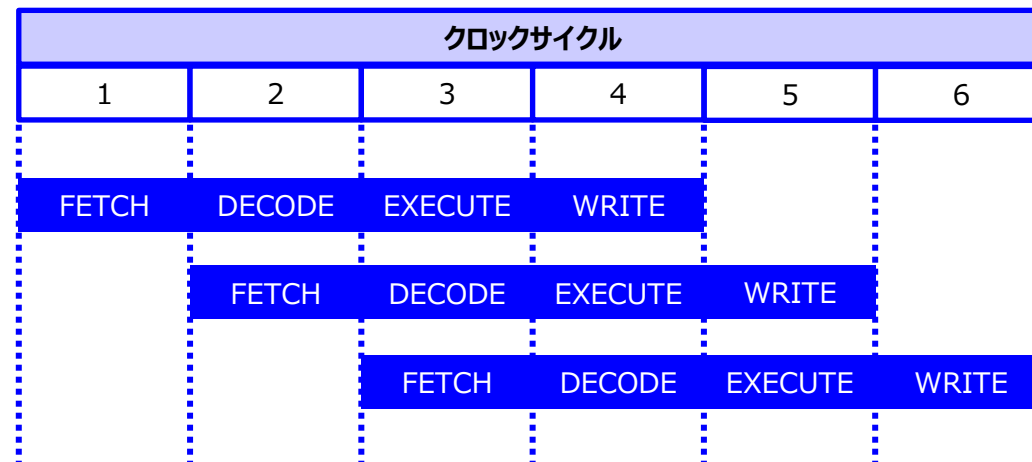
●命令パイプライン

ジョブ・タスク

- ジョブとは、利用者から見てコンピュータにさせる仕事の単位です。
- タスクとは、プログラムを実行するCPU側から見た処理単位です。ユーザーからジョブを受け付け、タスクとして処理しています。

命令パイプライン

- CPUを中心とした処理機構をプロセッサと言います。GPUなど補助的な機構などもあるため、それらをまとめて処理する機構としてプロセッサを呼んでいます。
- 右の図では、4クロック必要な1命令コードが4段階並行されるため理論上最大で4倍の性能を出すことができます。



FETCH

命令コードを主記憶装置からレジスタに転送。

DECODE

読み出した命令コードを解釈。

EXECUTE

命令コードに従って処理を実行。

WRITE

処理した結果をレジスタに反映。

コンピューターの構成要素

ソフトウェア

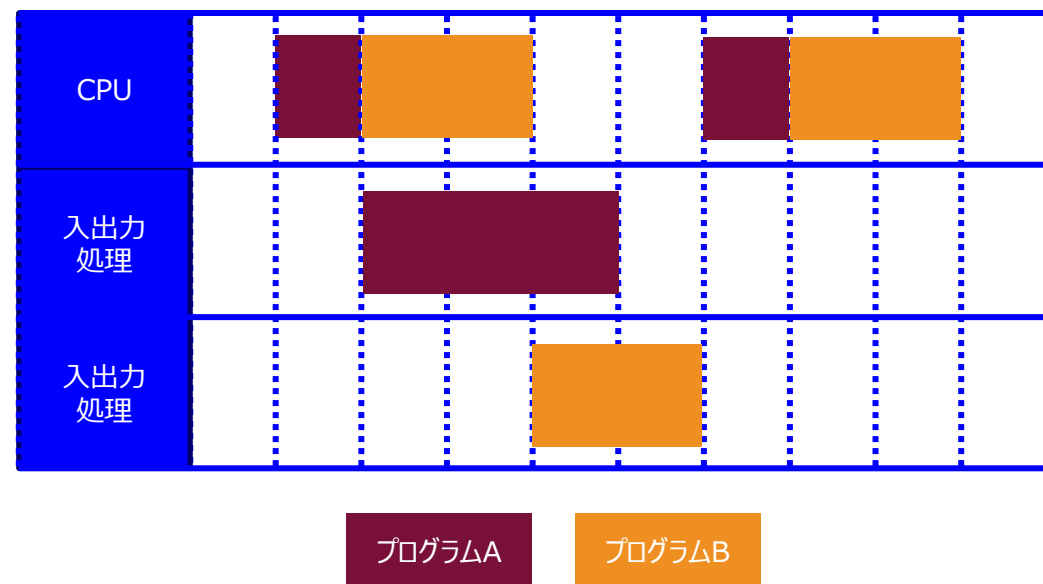
● 処理をより早く

マルチプログラミング

- CPUは1つコアで同時に2つの処理を実行するということとはできません。
- マルチプログラミングとは、CPUの非稼働時間を減らし見かけ上は同時に実行することです。
- CPUの待機時間を利用して、他の処理を実行しリードタイムを短くします。

割り込み

- 割り込みが発生した場合はカーネルが起動し、実行中のプログラムから解放され、割り込みしたプログラムが実行されます。
- 多くの場合、周辺機器からの割り込みを意味しますが、実際にはプログラムからの例外処理も含まれます。



CPUはプログラムの処理空き時間を利用し、他のプログラムの実行する

コンピューターの構成要素

ソフトウェア

●カーネル

カーネル

- アプリケーションの実行を許可しプロセス管理やハードウェアとソフトウェアを中継するプログラムであり、オペレーティングシステムの中核をなします。

モリシックカーネル (Monolithic Kernel)

- OS機能も含めてほとんどの機能をカーネルに盛り込んだ設計です。

マイクロカーネル (Micro Kernel)

- OSの中核として必要最小限の機能だけをカーネルに盛り込んだ設計です。

機能	内容
プロセス管理	プロセスの作成、実行、終了といった一連の流れを制御します。 プロセス間のリソースの取り合いを防ぐ役割もあり、システムの安定性を維持します。
メモリ管理	プロセスごとに適切なメモリ領域を割り当て、また必要なくなったメモリ領域を解放します。
デバイスドライバの提供	ハードウェアデバイスとのやり取りを担当するソフトウェアです。デバイスごとのドライバを入れないとハードウェアデバイスを動かすことはできません。カーネルを通じてハードウェア制御コマンドの発行などを行います。
ファイルシステムの管理	コンピューター上のソフトウェアとしてのファイルと、ハードウェア上の記憶としてのファイルを管理します。

コンピューターの構成要素

ソフトウェア

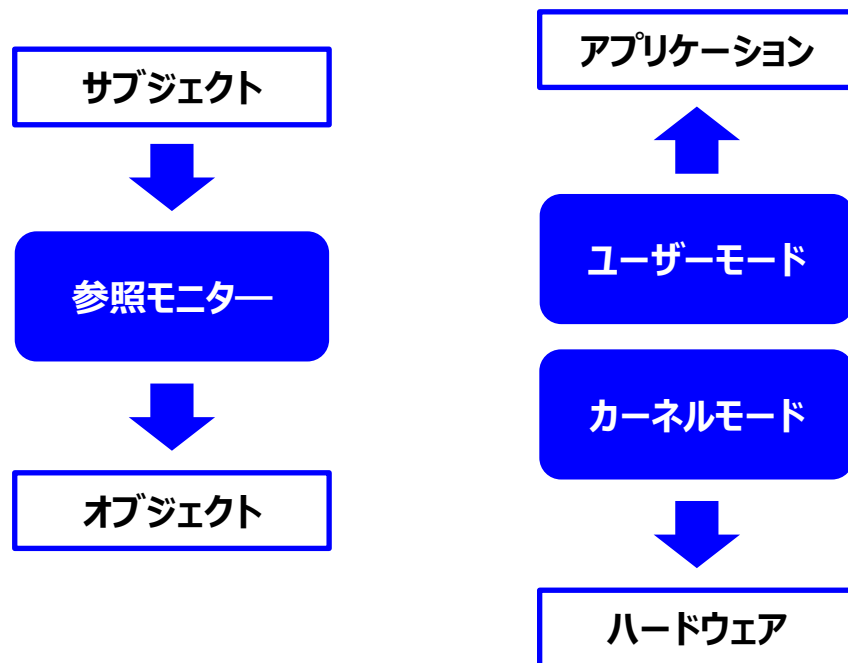
●セキュリティカーネル

参照モニター（Reference Monitor）

- カーネルのアクセス制御のコンセプトです。参照モニターは、ハードウェア、ソフトウェア、およびファームウェアを利用し実現されます。
- 「迂回不可」「改ざん防止」「小規模」という3つの条件を満たしておくことが望まれる。

CPUの動作モード

- カーネルモードとは、メモリ、CPU、ディスクなどへの無制限のアクセスを許可します。ハードウェアを操作するために相当量ゆるく設計されたアクセス権です。この自由度の高さは低レベルでのシステム制御を可能にしますが、不適切な操作やバグによりシステム全体がクラッシュする可能性もあります。
- ユーザーモードとは、ハードウェアに直接アクセスできず、APIを介して送信されます。アプリケーションを実行するために厳密に設計されたアクセス権です。





セキュリティ アーキテクチャー

セキュリティアーキテクチャー

クライアントベースシステム

● クライアントベースシステム

- ・ クライアント側に主処理を任せます。
- ・ クライアント端末の性能が、サービスの質が大きく依存します。
- ・ 家庭用ゲーム機などがクライアントベースシステムです。

運用



サービスを提供するアプリケーションがクライアント端末をアップデートする必要があり、統一管理がやや難しくなります。

稼働率



基本的に、クライアント端末がある限りはサービスの利用ができます。



サーバー



クライアント

クライアント側がベースで処理を行います

セキュリティアーキテクチャー

サーバーベースシステム

●サーバーベースシステム

- サーバー側に主処理を任せます。
- サーバーの性能が、サービスの質が大きく依存します。
- 一般のWebサイトなどがサーバーベースシステムです。

運用



サーバーのみを管理し、クライアント依存の仕様を減らすことができます。

稼働率



サーバーがダウンしてしまうとサービス停止につながります。



サーバー



クライアント

サーバー側がベースで処理を行うシステムです

セキュリティアーキテクチャー

データベース

●ACID特性

原子性 (Atomicity)

- トランザクションは完全に実行されるか、まったく実行されないかのどちらかでなければならないという性質です。

一貫性 (Consistency)

- トランザクションの終了状態にかかわらず、データベースの整合性が保たなければならないという性質です。

独立性 (Isolation)

- トランザクションを複数同時に実行しても、単独実行の場合と同じ処理結果にならないという性質です。

耐久性 (Durability)

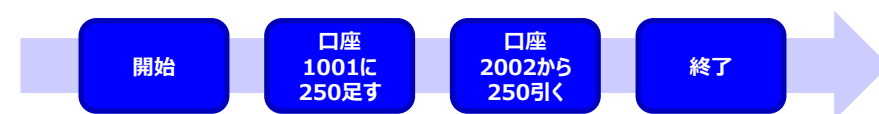
- トランザクションの結果は、障害が発生しても失われてはいけないという性質です。

```
BEGIN TRANSACTION
```

```
UPDATE bankaccounts SET balance = balance + 250  
WHERE account_number=1001; ※1
```

```
UPDATE bankaccounts SET balance = balance - 250  
WHERE account_number=2002; ※2
```

```
END TRANSACTION
```



※1 : account_number=1001のレコードbalanceを250足す

※2 : account_number=2002のレコードbalanceを250引く

(つまり、口座番号2002から口座番号1001に250の送金)

両方のUPDATEが実行できて初めて送金が成り立つ。

セキュリティアーキテクチャー

データベース

●データベース管理システム

データベース管理システム（DBMS、DataBase Management System）

- データベースを管理し、外部のソフトウェアからの要求に応じてデータベースの操作を行う専門のソフトウェアです。
- 汎用的なDBMSは、データベースの定義、作成、クエリ、更新、および管理できるように設計されています。
- MySQL、PostgreSQL、MongoDB、MariaDB、Microsoft SQL Server、Oracle、Sybase、SAP HANA、SQLite、IBMDB2などがあります。

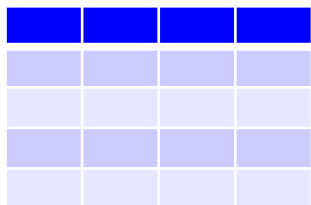
セキュリティアーキテクチャー

データベース

●データベースの体系

リレーショナルデータベース (Relational database)

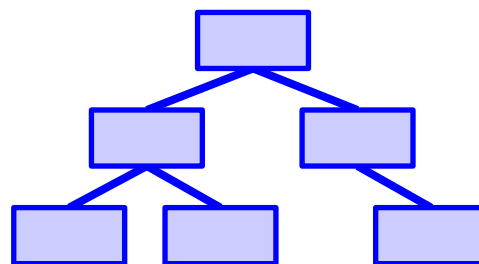
- 一件のデータ（レコード）を複数の属性の値（フィールド）の組として表現し、レコードをデータを格納していくデータベース構造です。



リレーショナルデータベース

階層型データベース (Hierarchical database)

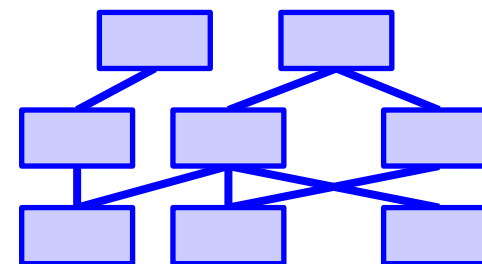
- データの編成方法には、ツリーのような構造を使用します。子ノードが一つの親ノードを持ち、ツリー状に構成されます。



階層型データベース

ネットワーク型データベース (Network database)

- 1つの子ノードが複数の親ノードを持つことができます。



ネットワーク型データベース

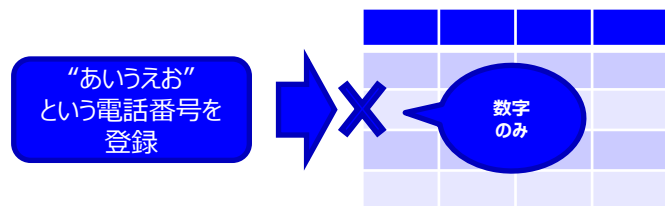
セキュリティアーキテクチャー

データベース

● データベースの制約

セマンティック整合性 (Semantic consistency)

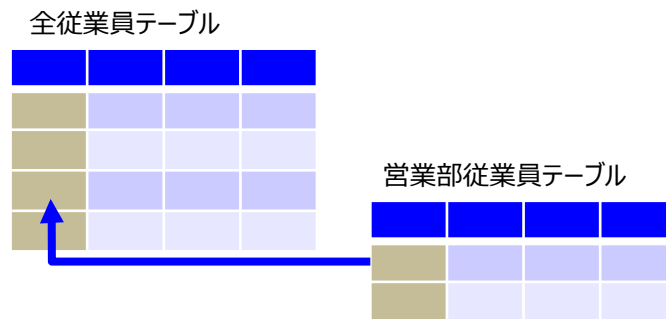
- 各属性値を属性データ型と一致させる制約です。



セマンティック整合性

参照整合性 (Referential integrity)

- 外部キーによって、異なるテーブル間でのデータの一貫性を保証するための制約です。



参照整合性

エンティティ整合性 (Entity integrity)

- テーブル内の行（レコード）が一意に識別されることを保証するためのルールや制約です。
- 主キーの制約がこの整合性を担保します。



エンティティ整合性

セキュリティアーキテクチャー

データベース

●リレーショナルデータベース

候補キー (Candidate key)

- あるテーブルにおいて、そのレコードが一意であることを担保できる属性を指します。
- 右の全従業員テーブルでは、従業員IDや氏名やメールアドレスの属性を持っています。氏名は他の人と被っているため、候補キーにはなりません。

主キー (Primary key)

- レコードを一意に識別することができるように指定される属性への制約です。この制約がかけられた属性は、値を持たないこと (NULL) ができず、他のレコードの同じ値も持つことはできなくなります。
- 一般的なアカウント管理テーブルにおいては、名前やメールアドレスとは別にIDが割り当てられています。

全従業員テーブル

従業員ID (PK)	氏名	メールアドレス
00001	A男	sato1@email.com
00002	A子	sato2@email.com
00003	B男	suzuki1@email.com
00003	A子	suzuki2@email.com

主キーとなっている属性には空や同じ値を入れない

セキュリティアーキテクチャー

データベース

●リレーショナルデータベース

外部キー（Foreign key）

- テーブルのある属性に、別のテーブルの特定の属性に含まれる値しか格納できないようにする制約です。
- 営業部をしている従業員のテーブルの中には、全従業員テーブルの中に存在しない従業員は存在してはいけません。したがって、営業部テーブルの従業員ID属性は全従業員テーブルのID属性を参照していることが望めます。

全従業員テーブル

従業員ID(PK)	氏名	メールアドレス
00001	A男	sato1@email.com
00002	A子	sato2@email.com
00003	B男	suzuki1@email.com
00004	A子	suzuki2@email.com

営業部従業員テーブル

従業員ID(FK)	契約件数
00001	11
00003	20

外部キーとなっている属性には指定されている属性以外の属性値を入れられない

セキュリティアーキテクチャー

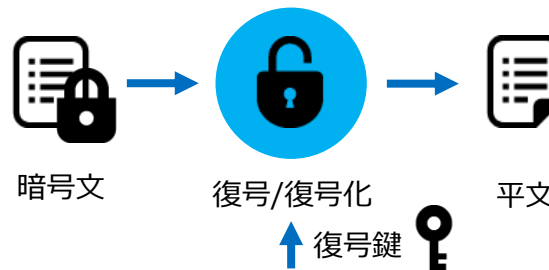
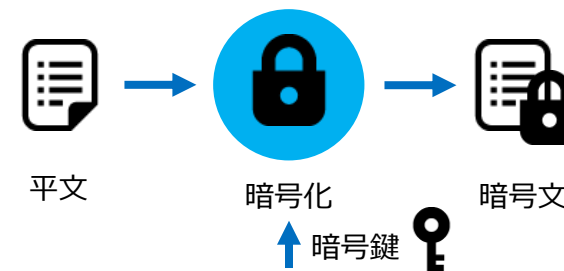
暗号システム

●暗号化

- 平文とは、相手とやり取りする解釈可能な内容です。
- 暗号文とは、平文を変換した解釈不可能な内容です。
- 暗号化とは、平文から暗号文に変換する行為です。
- 復号化（復号）とは、暗号文から平文に変換する行為です。
- 暗号鍵とは、平文から暗号化に変換する鍵です。
- 復号鍵とは、暗号化から平文に変換する鍵です。

“復号化”と“復号”は、どちらが正しい？

暗号文にすることは暗号化。では平文にすることは、復号化というのでしょうか？
“暗号”文に変“化”する意味を“暗号化”という言葉になりますが、復号文という状態はなく、“復号”状態に変“化”という意味は日本語としては成り立ちません。よって、「復号化する」よりも「復号する」というほうが適切です。ただ、「暗号化」と「復号化」は文面で記載した際にきれいに対をなすため、体裁上利用されることがあります。なので、復号化という文字を見つけても、鬼の首を取ったように指摘することではありません。



セキュリティアーキテクチャー

暗号システム

● スパルタンスキュタレー暗号 (Spartan scytale cipher)

- 古代ギリシャのスパルタで使用されたとされる暗号技術の一種です。この暗号は、最も古い転置暗号の一つと考えられています。
- 受け取り側が書き手と同じサイズの棒に巻き付けることで、また読めるようになります。

● ヴィジュネル暗号

- A-Zのストライプに並んだ表を使う暗号方法です。アルファベットを26回繰り返して、VigenèreSquareという表を形成する。
- 右の図では、横の平文に対して、縦の鍵が決まる。平文がCISSPで、キーがABCDEの場合、暗号文はCJUVTになる。プレーンテキストがキーよりも長い場合、キーは折り返される。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

ヴィジュネル暗号

セキュリティアーキテクチャー

暗号システム

● 暗号円盤

- A-Zが書かれた円盤を使い、外側の文字と内側の文字を置換する暗号方式です。
- 外側に書いてあるアルファベットを、内側に書いてあるアルファベットに書き換えます。戻すときには、内側のアルファベットを外側のアルファベットに置換します。円盤は回るようになっており、使いまわしが可能です。



● ジェファーソンディスク (Jefferson disk)

- 暗号文を作り出す南京錠のような見た目の装置です。
- まず、それぞれの輪っかを回転させ、一行に平文を作ります。ここで他の列を見ると、当然文章になっていない文字列、暗号文になっています。受け取った相手は、同じジェファーソンディスクを持っていれば、一行に暗号文になるように回転させると、ある一行が平文になっている。

セキュリティアーキテクチャー

暗号システム

● 暗号機

- 暗号文を作り出し、解読する装置です。
- 手計算による暗号化よりも高速かつ複雑な暗号化を行うことができたため、第一次世界大戦から第二次世界大戦にて各国で広く使用されました。電子計算機や集積回路が発明されると、計算速度と汎用性に劣り、機械式暗号は役割を終えました。

世界大戦時の暗号機の例

- エニグマとは、第二次世界大戦のときにナチス・ドイツが用いていた暗号機です。
- SIGABAとは、第二次世界大戦から1950年代までに米国が用いていた暗号機です。エニグマよりも解読困難になった強化版です。

エニグマはなぜ解読された？

当時エニグマは解読不可能と言われていました。しかし、いくつか脆弱性があります。まず、通信鍵を2回送付するというものです。当時は通信技術に難があったため、検証のために鍵を2回送付していたのですが、これは推測に利用されてしまいます。また、通信鍵を選んだのが人間であるために覚えやすい鍵が選定されていたこともあります。潜在的には、国防軍鍵表を送信者に事前に共有していなければならないため、鍵配送問題を持っています。

「イミテーション・ゲーム エニグマと天才数学者の秘密」は、第2次世界大戦時、ドイツ軍が世界に誇った暗号機エニグマによる暗号の解読に成功し、連合国軍に勝機をもたらしたイギリスの数学者アラン・チューリングの人生を描いた映画です。

セキュリティアーキテクチャー

暗号システム

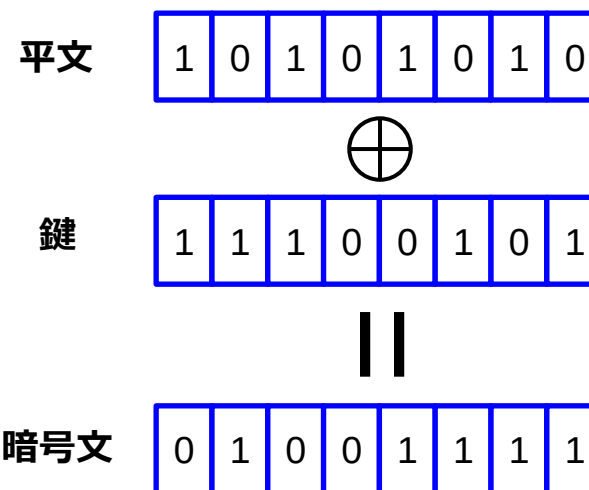
●バーナム暗号

- 平文とランダムに発生させた暗号鍵に排他的計算（XOR）を適用する事で暗号文を作り出す方法です。
- 排他的とは、片方が真（計算上は1と表現される）であるとき、結果も真である論理演算である。
- 例えば、平文（10101010）、暗号鍵（11100101）であった場合、暗号文は（01001111）となります。

バーナム暗号の脆弱性

- 暗号鍵と復号鍵が一緒であるため、対話相手にも共通の鍵を事前に共有しておく必要があります。事前に共有する段階で鍵がばれてしまえば無意味になります。（**鍵配送問題**）
- バーナム暗号での暗号鍵は乱数を使います。コンピューター計算によって算出した乱数（疑似乱数）に強度が求められます。

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0



排他的論理和

セキュリティアーキテクチャー

クラウドベースシステム

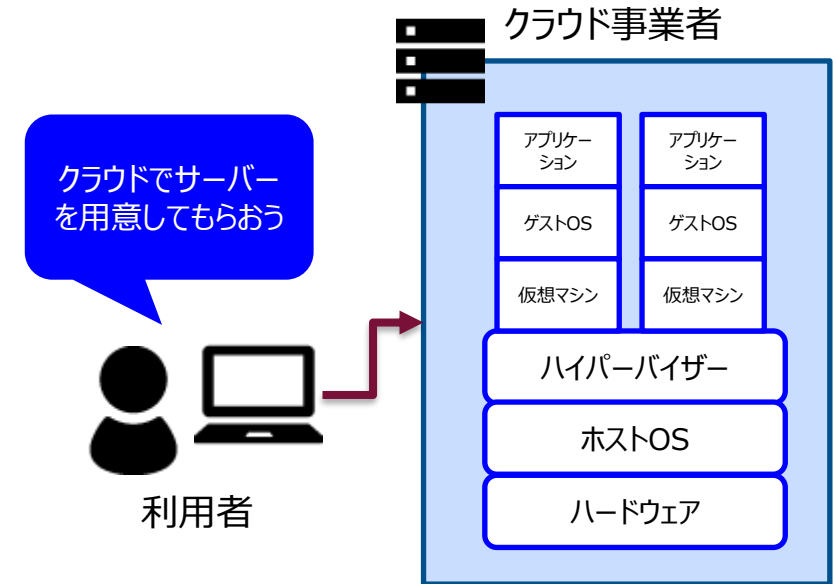
●クラウド

クラウドコンピューティング (Cloud computing)

- インターネットを通じてリモートのサーバーやデータセンターでコンピュータリソースやサービスを提供・利用する形態を指します。
- ユーザーは物理的なハードウェアやソフトウェアを自分で管理することなく、必要なときに必要なリソースを利用できるようになります。

弾力性 (Elasticity)

- クラウドコンピューティングにおいて、リソースの供給能力を動的に増減させることができる特性を指します。
- アプリケーションやサービスの需要に応じて、計算能力、ストレージ、ネットワークリソースを迅速にスケールアップ（増加）またはスケールダウン（減少）できます。



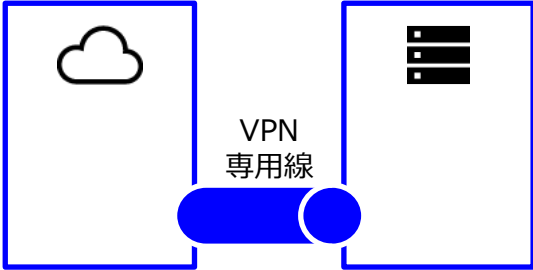
- ✓ 物理的な管理不要
- ✓ 短時間で立ち上げ
- ✓ 現地に出向く必要なし

セキュリティアーキテクチャー

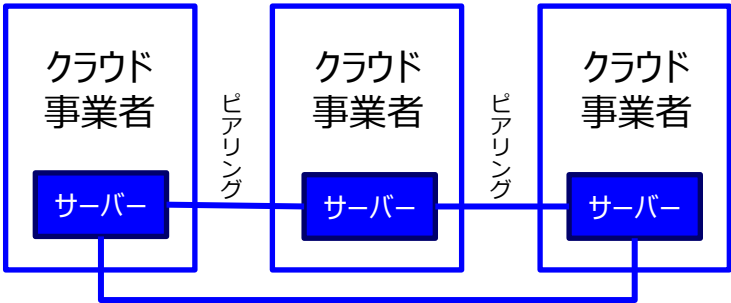
クラウドベースシステム

●クラウド

クラウドの利用形態	内容
パブリッククラウド	利用者を問わずに利用できるクラウド環境です。
プライベートクラウド	自社専用のクラウド環境です。自社サーバーでクラウド環境を構築・運用する形態をオンプレミスプライベートクラウドと呼び、クラウド事業者が利用者のためにクラウド環境を構築する形態をホステッドプライベートクラウドと呼びます。
マルチクラウド	複数のクラウドベンダーを利用する使い方です。クラウドベンダーの極度な依存を減らしたり、クラウドサービス自体の障害に備えます
シェアクラウド、商用クラウド	業界の特色を加味したクラウド環境です。
コミュニティクラウド	同種の企業が共同で構築して運用するクラウド環境です。
ハイブリットクラウド	クラウド環境とオンプレミス環境、コミュニティクラウドなどの各クラウドなどを合わせた使い方です。



ハイブリットクラウド



マルチクラウド

セキュリティアーキテクチャー

仮想化システム

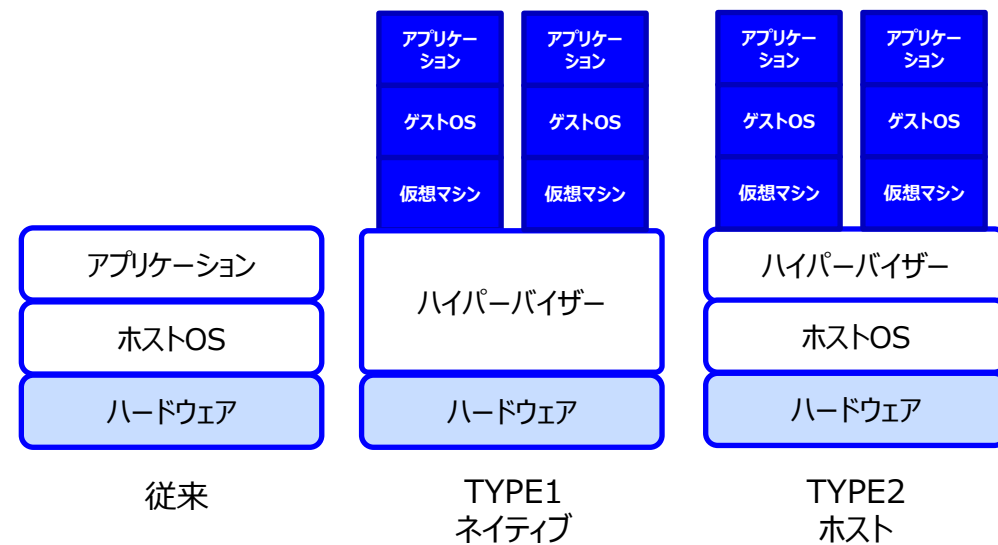
●仮想化

ハイパーバイザー

- 一つのマシンに複数のOSが動作しているように見せる技術です。
- ハードウェアを制御して処理能力や記憶能力を個々に分けます。個々のハードウェアリソースにOSを乗せることで、複数あるように動作します。これをハイパーバイザーといいます。
- VMware vSphere、Microsoft Hyper-V、Citrix XenServer、などがハイパーバイザーの一例です。

ハイパーバイザーの種類

- Type 1（ネイティブ、ベアメタル）ハイパーバイザとは、ハイパーバイザーがハードウェア上で直接動作し、全てのOSはそのハイパーバイザー上で動作する方式を指します。
- Type 2（ホスト）ハイパーバイザーとは、ハードウェア上でまず別のOSが稼働し、その上でハイパーバイザーが稼働し、ハイパーバイザーの上で更に別のOSを稼働します。



ハイパーバイザーの種類と構成

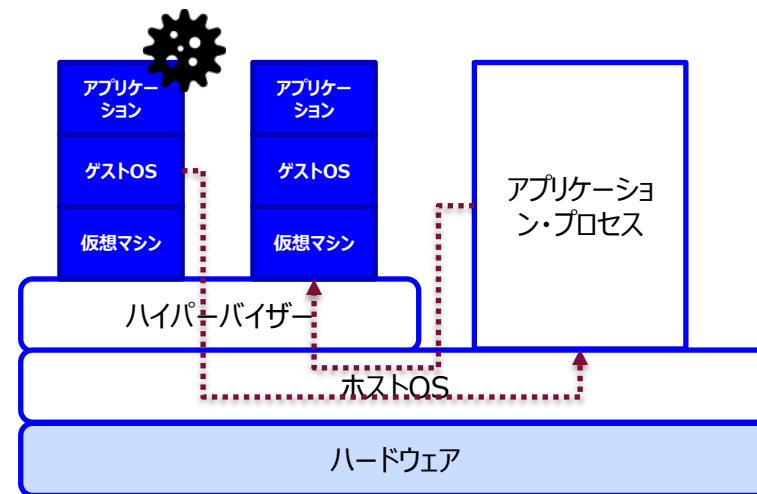
セキュリティアーキテクチャー

仮想化システム

● 仮想化

ハイパーバイザーの脆弱性

- **VMエスケープ**とは、仮想化されたコンピュータにおいて、仮想ホストから本体のコンピュータを操作することです。
- 仮想化をしている基盤のコンピュータに侵入された場合、他の仮想ホストにも侵入されます。つまり、セキュリティレベルの異なるホストに入られる可能性があります。



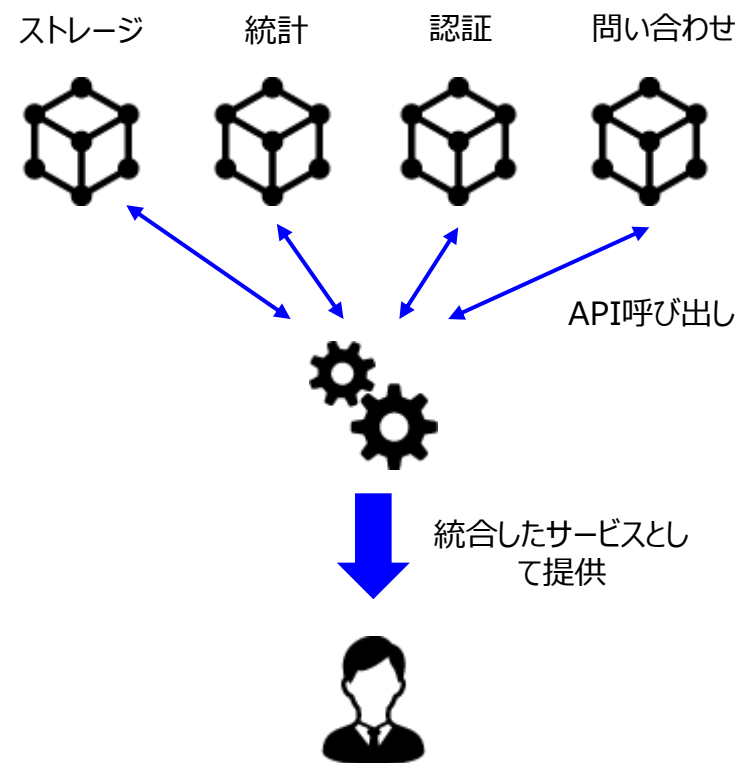
VMエスケープし、ホストそして他の仮想マシンに遷移

セキュリティアーキテクチャー

マイクロサービス

● マイクロサービスアーキテクチャ (Microservices Architecture)

- 大規模なアプリケーションを独立した小さなサービスの集合体として構築するソフトウェア設計の手法です。
- 個々のアプリケーションが提供するAPI (Application Programming Interface) を利用し、サービスが必要な機能のみを呼び出します。
- 連結部分のAPIが標準に則ることで、コンポーネントごと実行環境や開発体制が疎になり、迅速な開発を手助けします。



機能ごとに開発し、連携させてサービスを作る。

セキュリティアーキテクチャー

コンテナ化

●コンテナ コンテナ

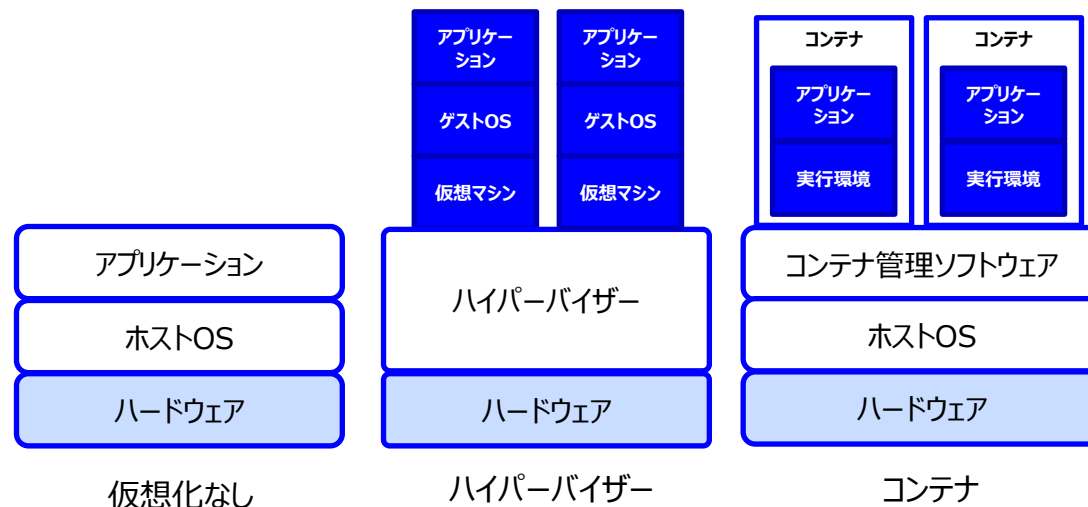
- コンテナ技術とは、アプリケーション実行環境を仮想化するものです。アプリケーションごとに実行環境となるOSと同包されたコンテナという形で管理できます。
- コンテナもコンテナ管理ソフトウェアでコンピュータリソースを管理し、コンテナを動作します。

コンテナとハイパーバイザーとの違い

- コンテナの場合には、アプリケーションが実行される環境として指定されているOSが、純粋なOSと同じではありません。カーネル部分はホストOSに頼る関係性になっています。



イメージという“型”から実際に動作するコンテナを生成する



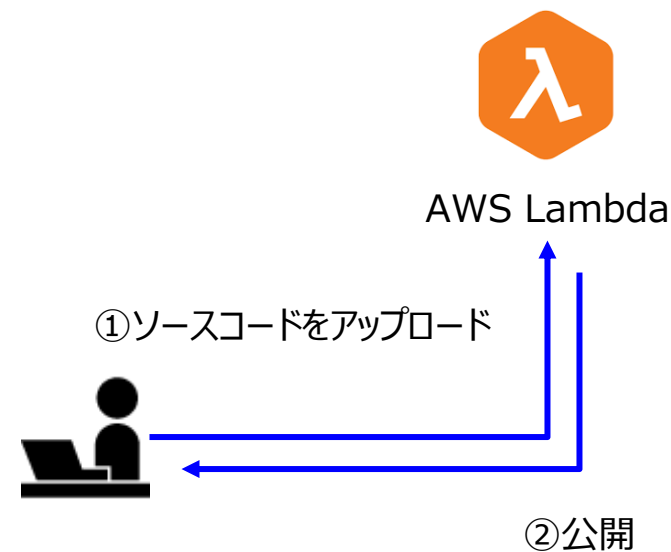
コンテナはコンテナ管理ソフトウェア上で動作する

セキュリティアーキテクチャー

サーバーレス

●サーバーレス (Serverless)

- 開発者がサーバーのインフラストラクチャ管理を意識することなく、アプリケーションのコードを実行できる環境を提供するクラウドコンピューティングのアーキテクチャの一種です。
- サーバーレスでは、物理的なサーバーや仮想サーバーの管理がクラウドプロバイダーに完全に委ねられ、開発者はコードの作成とデプロイに集中できます。



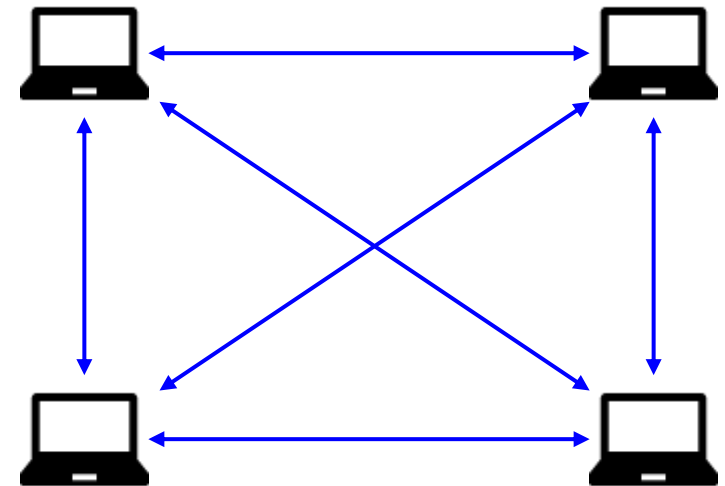
サーバーの知識は必要なく、ソースコードを公開できる

セキュリティアーキテクチャー

分散システム

●ピアツーピア

- クライアントとクライアント同士が通信し合い、処理をおこない、サービスを成立することです。
- 従来は、ピアツーピアと言いながらも集中型のサーバーを経由しているサービスもありましたが、昨今ではユーザー端末同士が接続する形態も可能となっています。
- ビットコインなどの分散システムで利用されています。



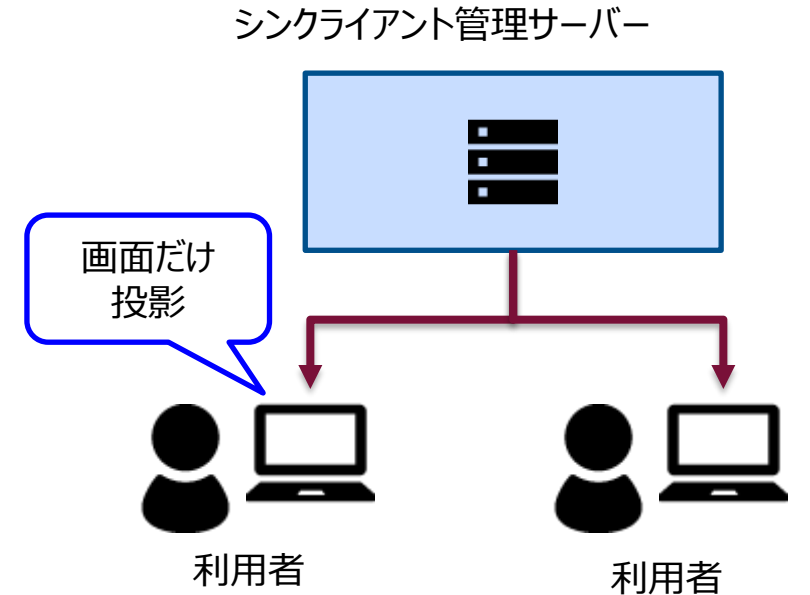
クライアント同士の関係で成り立つサービス形態

セキュリティアーキテクチャー

分散システム

●シンクライアント

- 統合管理を目的に最低限の機能だけを持つコンピューターです。
- 各コンピュータは最低限の機能（少なくとも、画像投影）を持ち、個々のコンピューターを管理サーバーで集中監視・制御します。大抵の場合、再起動時にコンピューターはクリアされセキュリティが担保された状態を保持します。



- ✓利用者側の端末では、実際の処理は行っていない
- ✓利用者側の端末には、機密情報が保管されない

セキュリティアーキテクチャー

高性能コンピューティングシステム

● グリッドコンピューティング

- コンピューターをネットワークで連結し、大量のデータ処理をさせる方式です。
- ネットワークはインターネットであることもあり、地理的に離れているコンピューターの力を借りることもあります。処理するタスクのほとんどをネットワーク経由で送付するため、十分なネットワークを確保している必要があります。
- 相互運用性、移植性、透過性、拡張性、セキュリティに優れています。

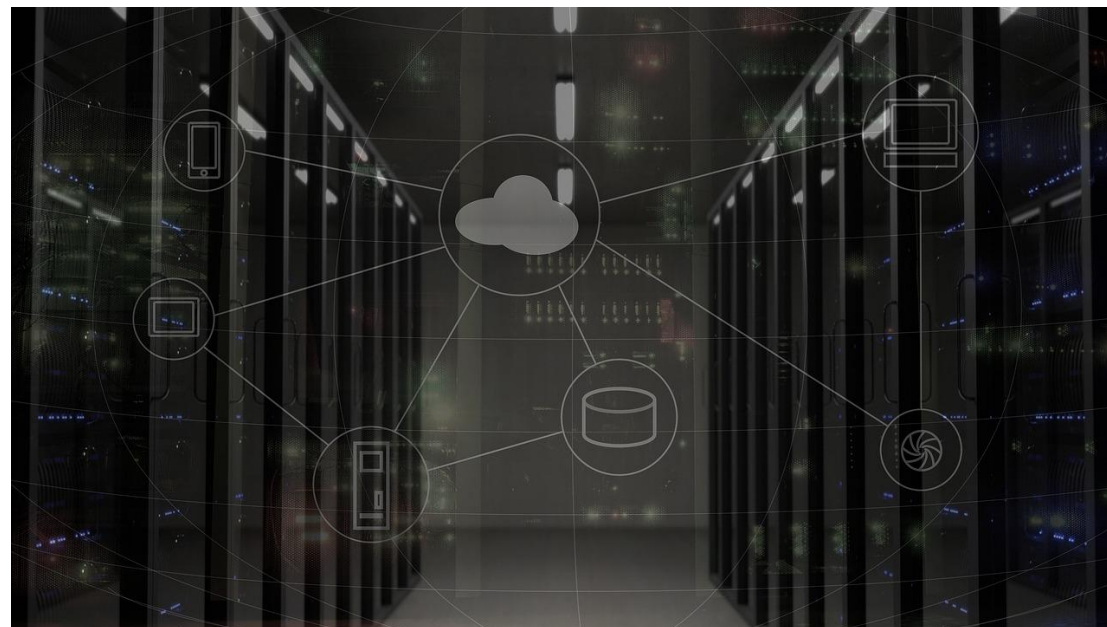
	クラウド	グリッド
内容	ユーザーからはシステム構成を意識せず、いつでも、様々なサービスでも提供する。	複数のコンピュータを並行処理し、一度に大量の処理を行う。
仮想化技術との関係	仮想化が必須	仮想化は任意

セキュリティアーキテクチャー

高性能コンピューティングシステム

● High-Performance Computingシステム

- 一般的なコンピューター性能をはるかにしのぐ速度でデータを処理し、計算を実行できるソリューションを指します。
- ハイパフォーマンスコンピューティングの一つであるスーパーコンピューティングは、並列して動作する複数のコンピューターシステムのコンピューティングリソースを集中的に使用して、大量の処理を実行します。



セキュリティアーキテクチャー

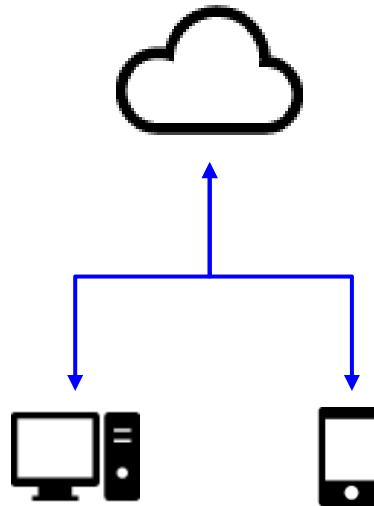
エッジコンピューティングシステム

● エッジコンピューティング

- 利用者側の端末自体やその近くに設置するコンピュータでデータを処理するコンピューティング技術です。
- データが生成された場所に近いエッジデバイスやローカルサーバーで処理されるため、データをクラウドに送信して処理するよりも遅延が大幅に減少します。これが特に重要なのは、リアルタイムでの処理が必要な自動運転車などのアプリケーションです。

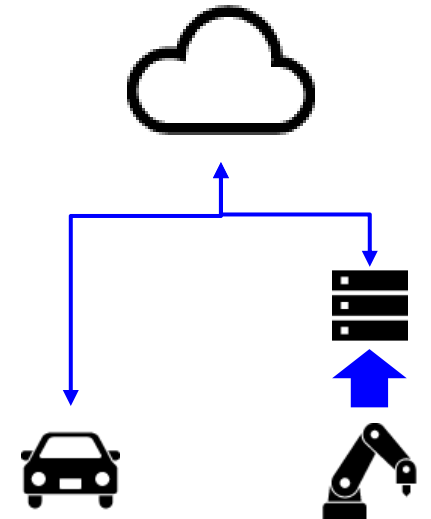
クラウドコンピューティング

集中処理



エッジコンピューティング

特定データのみ処理



セキュリティアーキテクチャー

モノのインターネット（IoT）

●IoT（Internet of Things）

- IoT（Internet of Things）とは、日常品をインターネットに接続してサービス拡充する動きです。
- IoTが普及すると、遠隔でも状況を把握し、機器の制御が可能になります。パソコン、スマホだけではなく、テレビや電子レンジやカーテンなども全部インターネットに接続し、制御をタイムリーに伝達することができるようになります。

IoTデバイスを壊すBrickerBot

IoTデバイスの普及によって、さまざまな家電が永続的なサービス拒否攻撃（PDOS）を受ける可能性があります。BrickerBotは、Webサイトへのハッキングプロセスを自動化するためのツールであり、本来は倫理的なハッカーがWebサイトの仕組みを理解できるようにするものでした。しかし、多くのIoTデバイスがWeb通信、つまりWebサイトに似た構造を持つことで、Web系の攻撃がIoTデバイスにも有効になってきています。BrickerBotを利用した攻撃では、変更されていないログインパスワードを持つIoTデバイスを狙い攻撃することが観測されています。

セキュリティアーキテクチャー

モノのインターネット（IoT）

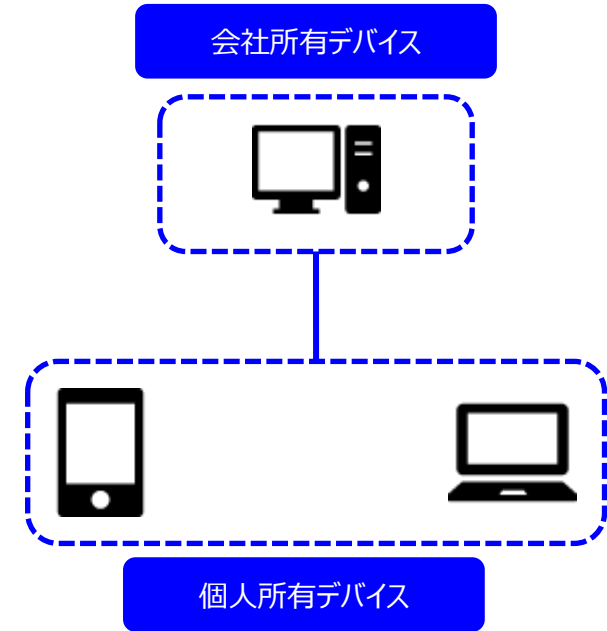
● 私物端末の業務利用

BYOD（Bring Your Own Device）

- 自身が持っているスマートフォンやパソコンなどの機器を会社の業務に利用する動きです。
- 対義語はBYAD（Bring Your Assigned Device）であり、管理する組織側が指定する端末を購入し、利用させることです。
- 企業としては、端末のコストの削減、棚卸管理の削減ができ、個人としては、個人の使用性に合った機器の選定ができるメリットがあります。
- BYOD端末には、モバイル端末管理アプリの導入します。

COPE（Corporate Owned Personally Enabled）

- BYODとは逆に、会社の資産を個人利用も可能とすることです。



モバイル端末管理（Mobile Device Management）

- ✓ 端末にパスコードをかけ、他人に利用させないこと
- ✓ フルディスク暗号化すること
- ✓ バックアップ管理すること
- ✓ リモートワイプ（遠隔削除）

セキュリティアーキテクチャー

組み込みシステム

● 組み込みシステム (Embedded System)

- 特定の機能やタスクを実行するために、ハードウェアとソフトウェアが一体となって設計されたコンピュータシステムのことです。
- 一般的なコンピュータのように汎用性を持つのではなく、特定の目的に特化して設計されています。
- 例えば、家電製品、医療機器、自動車の制御システムなどに組み込まれています。

Raspberry Pi

Raspberry Piとは、ARMプロセッサを搭載したシングルボードコンピュータです。マウスやキーボードだけではなく、HDMI出力でディスプレイを接続し、GPU搭載のデスクトップパソコンとして安価に利用することができます。

Raspberry Piの一番の使い方は、自作のIoTデバイスを開発することです。ドラえものの道具のように自分の想像力が詰め込まれた道具を作りたいと思ったことないでしょうか？ Raspberry Piでは、ソフトウェアコーディングとセンサーなどの付属デバイス制御が非常に簡易につながります。そのため、ビギナーからでもコーディングを学びながら、デバイスの動きを見て、オリジナルなIoTデバイスが作れます。



セキュリティアーキテクチャー

オペレーショナル・テクノロジー／産業制御システム（ICS）

●産業用制御システム（ICS, Industrial Control System）

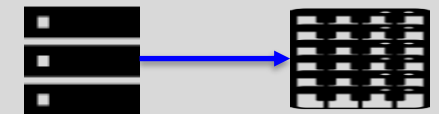
- 産業用制御システムには、過酷な環境下でも確実に実行できる必要があります。

SCADAシステムの構成要素

- 監視制御システム（Supervisory Control And Data Acquisition）とは、リアルタイムで分散する産業機器を監視するシステムです。監視制御システムは、監視コントロール機器と各生産機器の間で命令を出すのに、DNP3（Distributed Network Protocol）という通信プロトコルを使います。
- 分散制御システム（Distributed Control System）とは、生産ラインの前後など関連産業機器が互いに監視し合います。
- プログラム可能論理制御装置（Programmable Logic Control）とは、誤動作しない専用の制御装置です。



マスターステーション



リモートターミナル
ユニット（RTU） 高性能電子装置
（IED）

アウトーステーション



暗号ソリューション

暗号ソリューション

暗号ライフサイクル

● 暗号ライフサイクル (Cryptographic Lifecycle)

- 暗号化の危殆化とは、暗号化システムや暗号アルゴリズムがもはや安全ではなくなり、攻撃者によって解読される可能性が高くなった状態を指します。

Generation	暗号化に使用する秘密鍵や公開鍵を安全に生成します。
Distribution	生成された暗号鍵を安全に配布し、適切な当事者のみが鍵にアクセスできるようにします。
Operational Use	暗号鍵を用いてデータの暗号化や復号、署名や検証といった実際の暗号プロセスを実行します。
Storage	運用中および運用終了後も、鍵は安全に保管される必要があります。
Rotation/Renewal	セキュリティを維持するために、一定期間ごとに鍵を更新（ローテーション）します。
Destruction/Revocation	使用期限が切れた鍵や必要なくなった鍵は、安全に廃棄されます。
Audit and Compliance	暗号キーのライフサイクルが適切に管理されているかを定期的に監査し、コンプライアンス要件を満たしていることを確認します。

暗号ソリューション

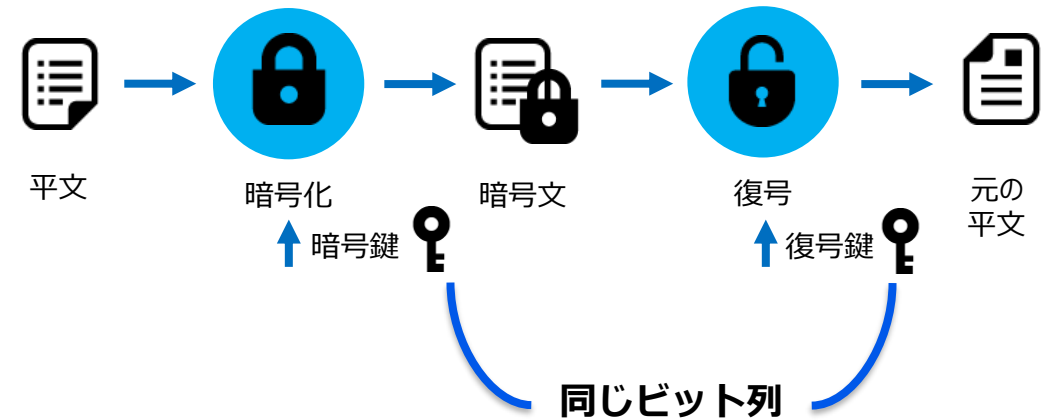
共通鍵暗号化方式

● 共通鍵暗号化方式

- 暗号鍵と復号鍵が同じである暗号化方式です。暗号の時に使う鍵と復号の時に使う鍵は同じビット列になります。
- 暗号鍵でも復号できるため、暗号化するモジュールも復号するモジュールも鍵を漏洩することは許されません。

鍵配送問題

- 対話相手にも共通の鍵を事前に共有しておく必要がある。鍵を配送するための安全な経路が必要であることから、安全な経路を達成するという暗号化の目的が保証しているわけではありません。



安全な通信方法を提供するための鍵をどのように安全に連携するのか？

暗号ソリューション

共通鍵暗号化方式

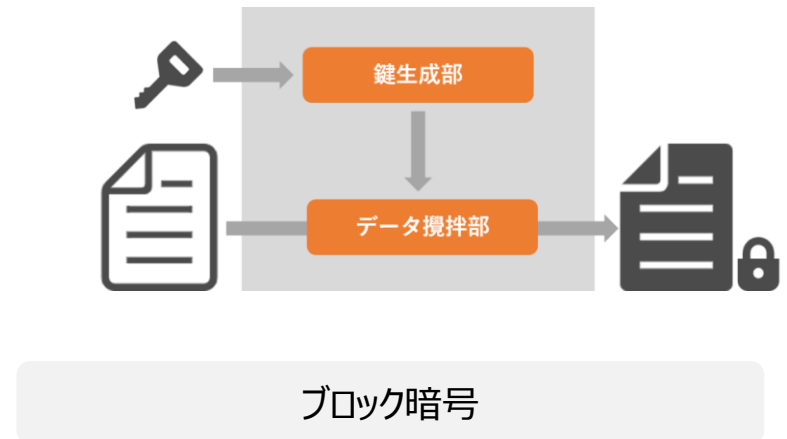
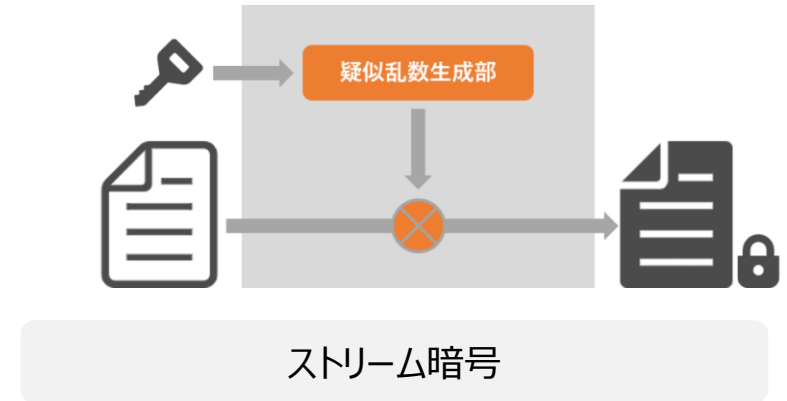
●ブロック暗号／ストリーム暗号

ストリーム暗号

- 疑似乱数生成器を利用し、平文と乱数のXORを取る暗号方法です。
- ストリーム暗号では、1ビット、8ビット、32ビットなどの単位で暗号化や復号が行われます。
- バーナム暗号や使い捨てパッドなどは、ストリーム暗号になります。

ブロック暗号

- データ攪拌部と鍵生成部を利用し、平文をブロックに分ける暗号方法です。
- ある特定のデータのまとまり（ブロック）を一度に暗号化をします。ブロックの長さは、暗号化アルゴリズムによって異なります。
- DESやAESなどは、ブロック暗号に当たります。



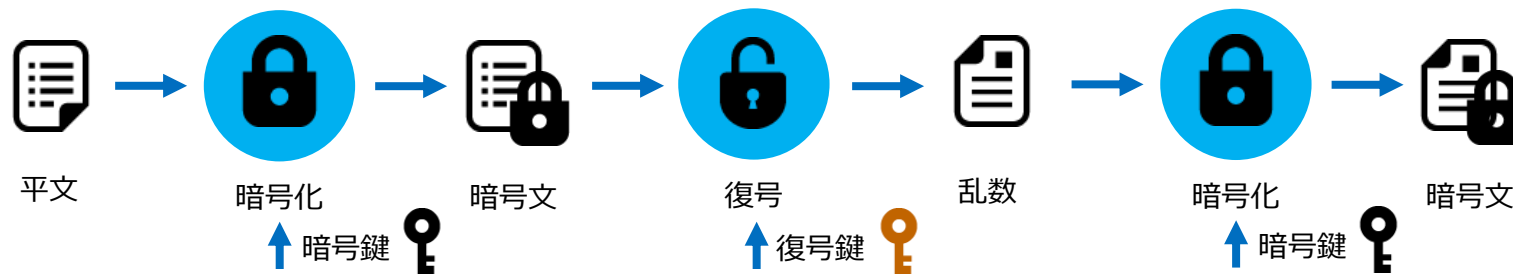
暗号ソリューション

共通鍵暗号化方式

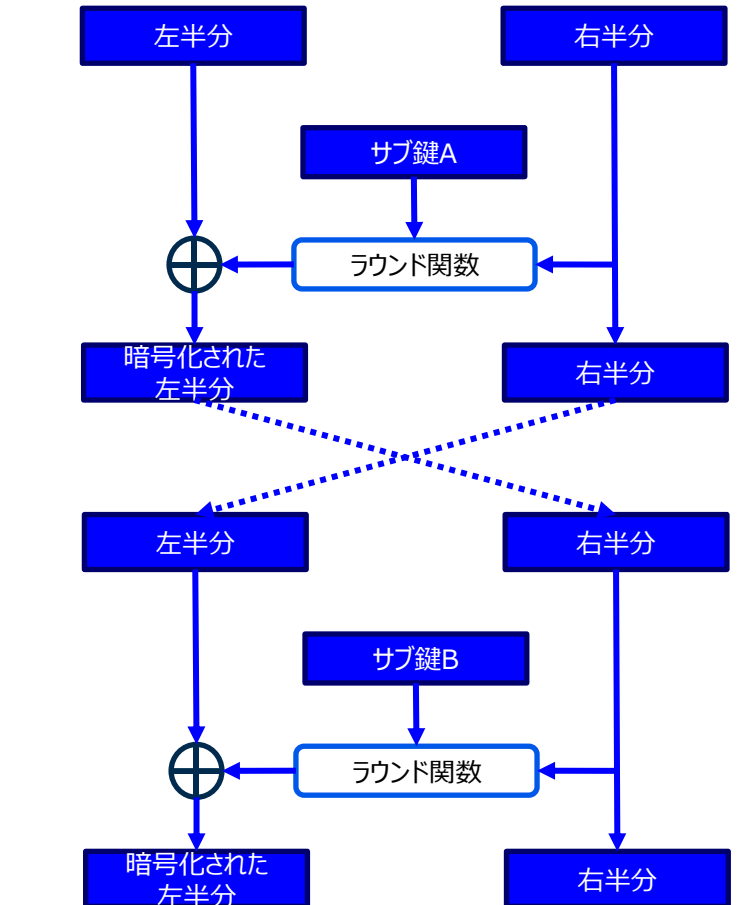
● 共通鍵暗号化方式

DES (Data Encryption Standard)

- 国際的に広く使われたブロック暗号化方式標準です。
- DESでは、暗号化の構造としてフィステルネットワーク (Feistel Network) が利用されます。入力を2分割し、右半分とサブ鍵をラウンド関数で算出された値と左半分の排他を取ります。この一回をラウンドと言います。次のラウンドで、右左を入れ替え実行します。ラウンドを複数回実行し、暗号化が成り立ちます。
- 1回の暗号強度が弱いため、暗号化⇒復号⇒暗号化とDESを3段階で利用する3DESが登場しました。



2Key-3DESは、1回目と2回目の暗号鍵は同じ方が管理の手間が少ない



Feistel Network

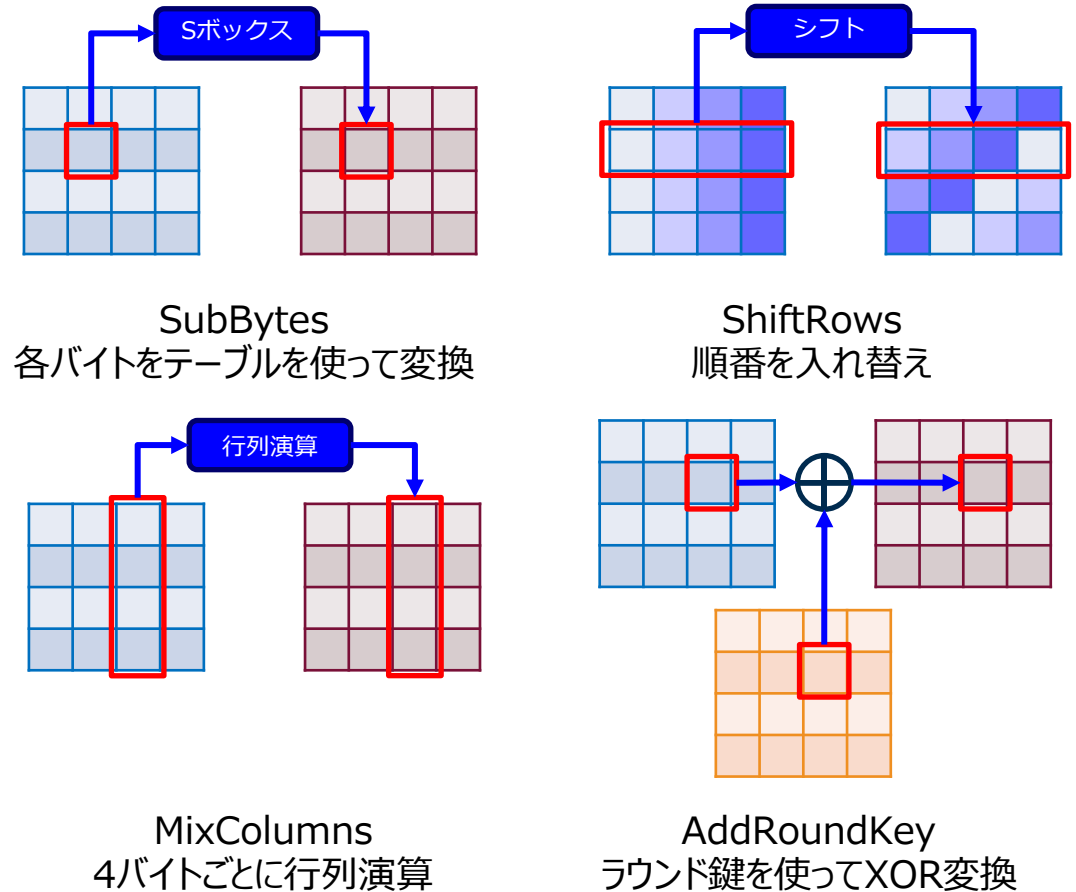
暗号ソリューション

共通鍵暗号化方式

● 共通鍵暗号化方式

AES (Advanced Encryption Standard)

- DESの後続として登場したブロック暗号化方式標準です。128/192/256ビットの鍵を利用し、128ビットのブロック暗号化です。
- 世界標準の暗号化方式を選考の結果、評価対象となる15個の暗号化アルゴリズム（CAST-256、Cryption、DEAL、DFC、E2、Frog、HPC、LOK197、Magenta、MARS、RC6、Rijindael、SAFER+、Serpent、Twofish）から最終候補の5個（MARS、RC6、Rijindael、Serpent、Twofish）が残り、最終的にRijindaelという暗号化方式が選出されました。
- AES（Rijindael）では、暗号化の構造としてSPN構造（Substitution Permutation Network Structure）が利用されます。



SubBytes、ShiftRows、MixColumns、AddRoundKeyを繰り返すことで暗号化します。

暗号ソリューション

公開鍵暗号化方式

●ゼロ知識証明

- 自身の保持する情報が命題が真であることを伝えるために、真であること以外の知識も伝えることなく証明できるようなやりとりの手法です。
- ゼロ知識には、証明者、検証者があり、証明者は理由を言わず、検証者に事実を証明することを目的にします。

鍵のついた箱



検証者にある手紙を書いてもらい、その手紙の内容を証明者が言い当てました。証明者は、“鍵を見せずに”鍵を持っていることを証明しました。

暗号化通信



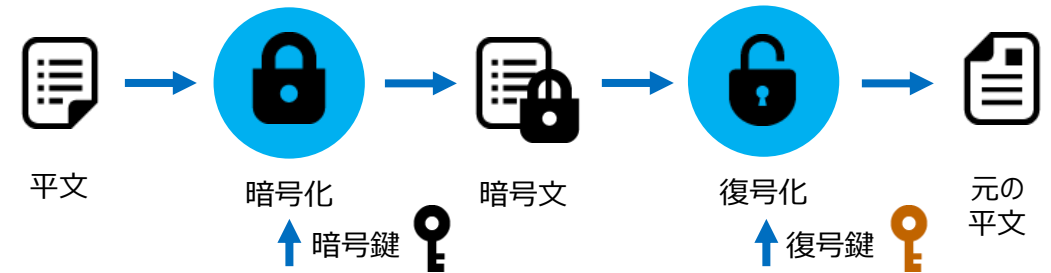
ある送信者がある受信者に対して、暗号化したメッセージを送ります。メッセージに対して真っ当な応答が返ってきました。受信者は暗号化したメッセージを復号する鍵を持っていることも証明になります。

暗号ソリューション

公開鍵暗号化方式

● 公開鍵暗号化方式

- 共通鍵暗号化方式は根本的に鍵配送問題を抱えていますが、暗号鍵と復号鍵を別の値とし、数学的暗号鍵から復号鍵が予想困難とした公開鍵暗号化方式があります。
- 暗号鍵は**公開鍵**と呼び、復号鍵は**プライベート鍵（秘密鍵）**と呼びます。



ディフィー・ヘルマン (DH, Diffie-Hellman) 鍵共有法

- 公開鍵暗号化方式を説明する前に、共通鍵暗号化方式の鍵配送問題に対応する技術もある。
- 公開してもよい値を共有し、通信主体内での計算によって共通鍵を生成します。よって、公開してもよい情報のみで共通鍵を生成することが可能です。
- 通信の主体がそれぞれA、Bという値を持つとき、素数XとYにおいて以下のようになります。

$$(X^A \bmod Y)^B \bmod Y = (X^B \bmod Y)^A \bmod Y = X^{A \times B} \bmod Y$$

暗号ソリューション

公開鍵暗号化方式

● 公開鍵暗号化方式

RSA (Rivest-Shamir-Adleman)

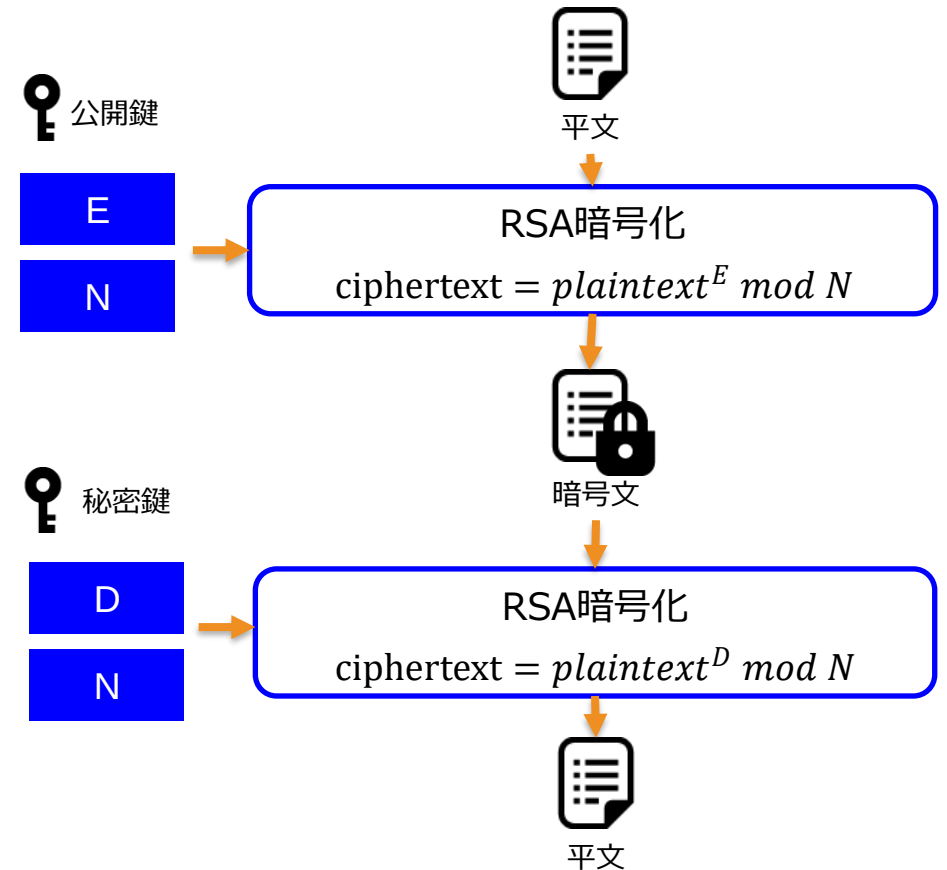
- デファクトスタンダードとなっている公開鍵アルゴリズムです。
- RSAの暗号文は、平文をE回掛けた結果をNで割った余りです。EとNの組み合わせが公開鍵（暗号鍵）になります。

$$\text{ciphertext} = \text{plaintext}^E \bmod N$$

- RSAの平文は、暗号文をD回掛けた結果をNで割った余りです。DとNの組み合わせが秘密鍵（復号鍵）になります。

$$\text{plaintext} = \text{ciphertext}^D \bmod N$$

- EとDの最小公倍数を決めることは簡単ですが、EからDを求めるのは困難になります。



暗号ソリューション

公開鍵暗号化方式

● 公開鍵暗号化方式

ElGamal暗号

- Taher ElGamalによる公開鍵暗号アルゴリズムです。
- RSAは素因数分解が困難なことを利用しますが、ElGamalは離散対数を求めることが困難であることを利用します。
- 無料の暗号化規格GnuPGに実装されました。

楕円曲線暗号（ECC、Elliptic Curve Cryptography）

- 楕円曲線を利用した公開鍵暗号アルゴリズムです。
- 楕円曲線という曲線を決め、その曲線上に乗っているある点に対して、特別な乗算を定義します。ひとたび乗算をすると、ある点や乗算を発見することが難しく暗号鍵になりえる。
- RSAよりも鍵長を節約できるため、重宝されています。

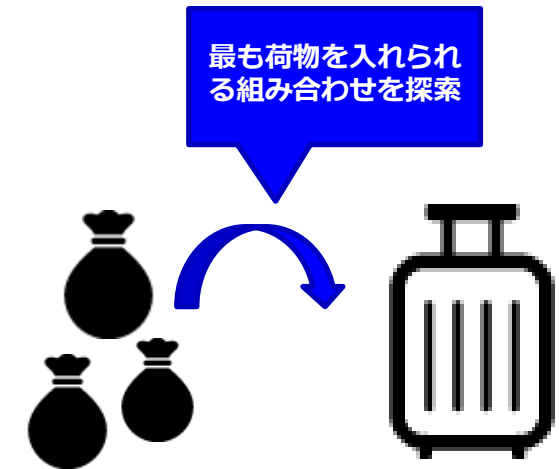
暗号ソリューション

公開鍵暗号化方式

● 公開鍵暗号化方式

Merkle-Hellmanナップサック暗号

- Ralph C. MerkleとMartin Edward Hellmanが発表したナップサック問題（部分和問題）を利用した公開鍵暗号です。
- 秘匿目的であり、デジタル署名などの認証を目的としたものではありません。
- 部分和問題は、与えられた n 個の整数から任意の部分集合を選び、その集合内の数の和が与えられた数 N に等しくなるようにできるかを判定する問題です。NP完全であることが知られています。



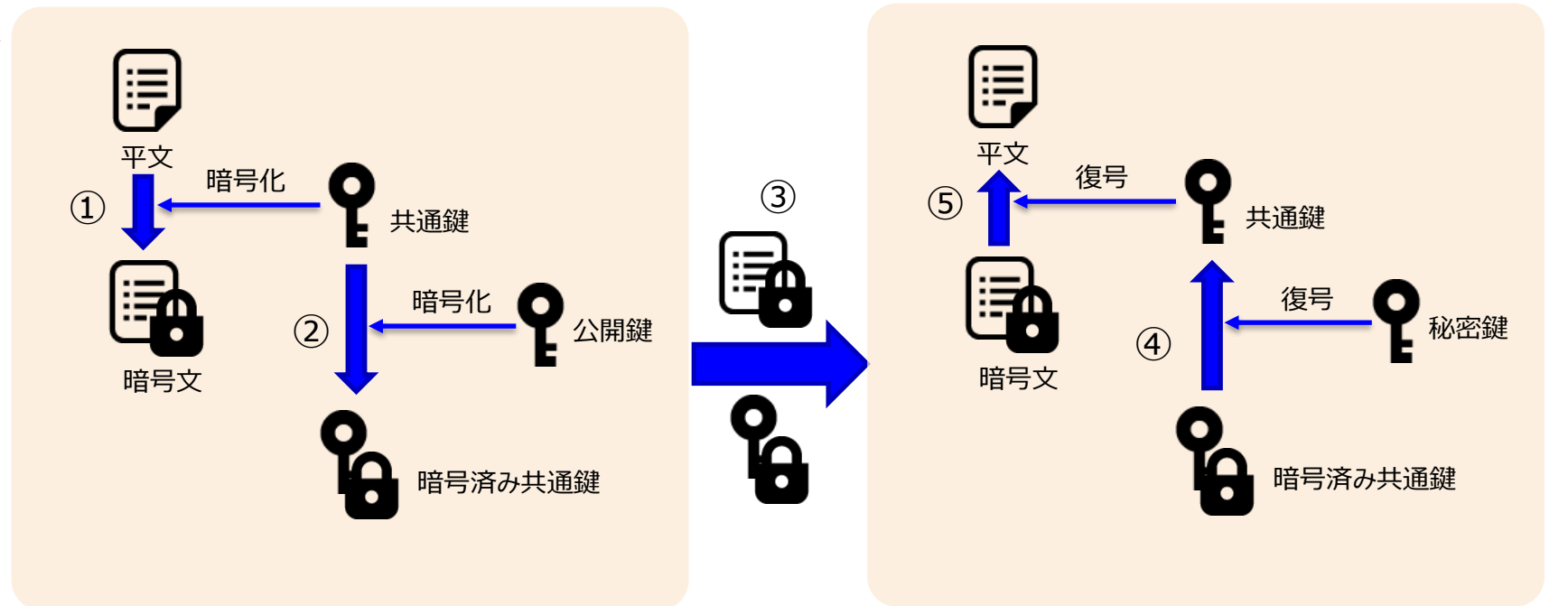
ナップサック問題

暗号ソリューション

公開鍵暗号化方式

●ハイブリッド暗号化システム

- 共通鍵暗号化方式は鍵配送問題を持ちます。その一方で、公開鍵暗号化方式は暗号・復号処理速度に時間がかかります。
- 共通鍵暗号化方式を公開鍵暗号化方式で安全に渡し、実際の通信内容のやり取りの暗号化には共通鍵暗号化方式を利用するハイブリッド暗号化システムが利用されています。



① 共通鍵で平文を暗号化
② 共通鍵を公開鍵で暗号化

③ 暗号文と暗号済み共通鍵を送付

④ 暗号済み共通鍵を秘密鍵で復号
⑤ 暗号文を共通鍵で復号

ハイブリッド暗号化システムでは、共通鍵暗号化方式と公開鍵暗号化方式の互いの欠点を補う

暗号ソリューション

公開鍵暗号化方式

●その他の暗号化方式

RIPEMD-160

- 160ビットのハッシュ値を持つハッシュ関数です。
- 前身のRIPEMDのアップグレードになるこのバージョンは、互換性維持という目的でのみ使われている。
- 広く使われていませんが、ビットコインを運用するためのブロックチェーンで使われています。

量子暗号

- 光子の量子的な性質により、盗聴が不可能な通信です。
- そもそも盗聴とは受信者にばれることなく、内容を読み取ることです。しかし、量子暗号では盗み見ようとする物理的な刺激により、光子の状態が変化してしまい、受信者が盗聴されている（データが崩れている）ことを検知できます。

暗号ソリューション

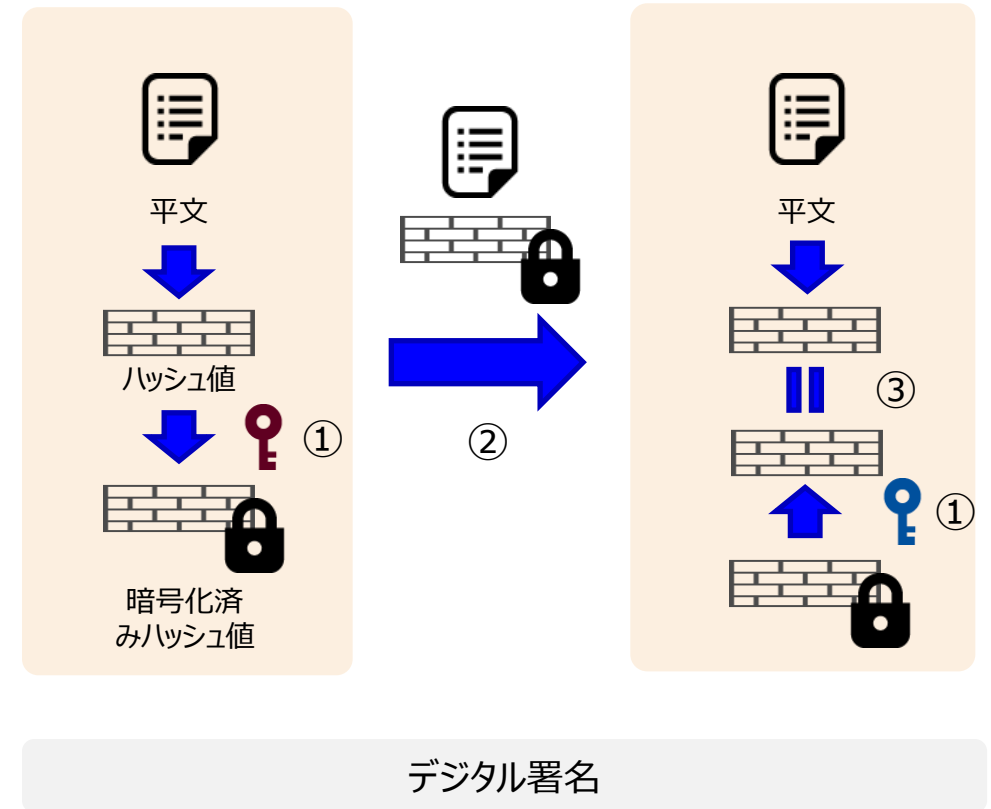
電子署名および電子証明書

●デジタル署名

送信されてきたデータが本人から送信されたことを証明するための技術です。完全性と本人性を得ることができます。

デジタル署名の手順

- ① 署名するには、公開鍵暗号化方式のプライベート鍵と公開鍵を用意します。
- ② 署名するデータのハッシュ値をプライベート鍵で暗号化し、署名するデータと暗号化されたハッシュ値を送ります。
- ③ 受け取り側は、暗号化を公開鍵で復号したハッシュ値と、署名データのハッシュ値を比較します。もしもハッシュ値が同じであれば、同じ内容のデータだと言えます。



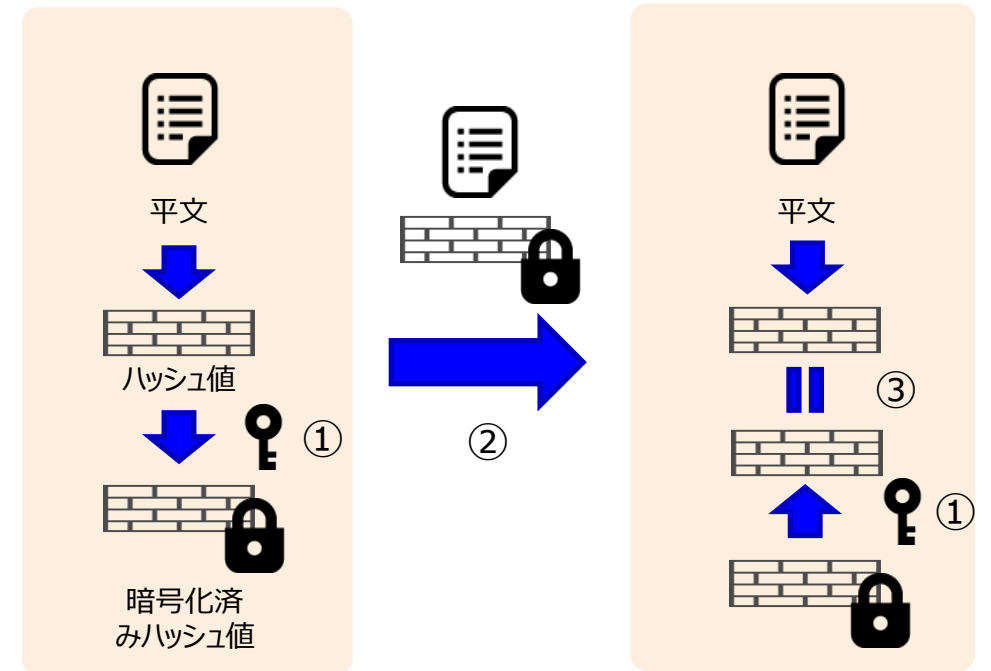
暗号ソリューション

電子署名および電子証明書

●メッセージ認証コード（MAC、Message Authentication Code）

メッセージ認証コードの手順

- ① 署名するには、共通鍵暗号化方式の共通鍵を用意します。
- ② メッセージ本体に送信者と受信者で共有している鍵を基にハッシュ値を算出します。この固定値のことをMAC値と呼びます。それを共通鍵で暗号化し、署名するデータと暗号化されたハッシュ値を送ります。
- ③ 受け取り側は、暗号化を共通鍵で復号したハッシュ値と、署名データのハッシュ値を比較します。もしもハッシュ値が同じであれば、同じ内容のデータだと言えます。



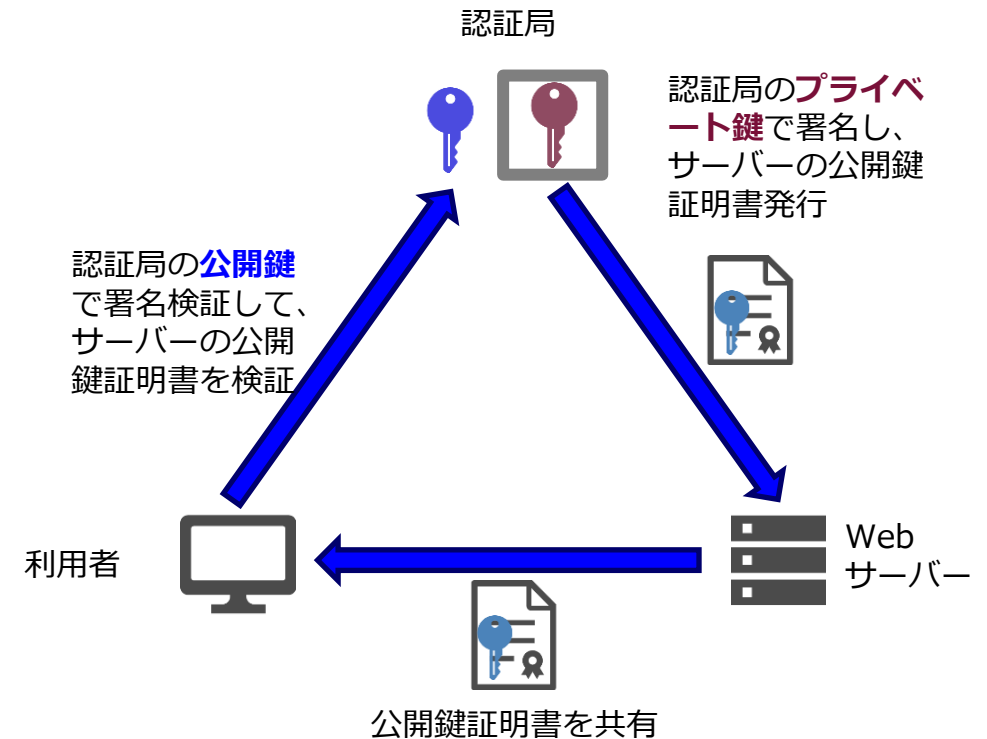
メッセージ認証コード

暗号ソリューション

公開鍵基盤 (PKI)

● 公開鍵基盤 (PKI、Public Key Infrastructure)

- 公開鍵基盤により、インターネットのようなオープンな通信経路でも真正性を保ち公開鍵を連携させることができます。
- 公開鍵を利用して暗号する側は、真正性がない公開鍵で機密データを暗号化することはしてはいけません。



公開鍵基盤

暗号ソリューション

公開鍵基盤 (PKI)

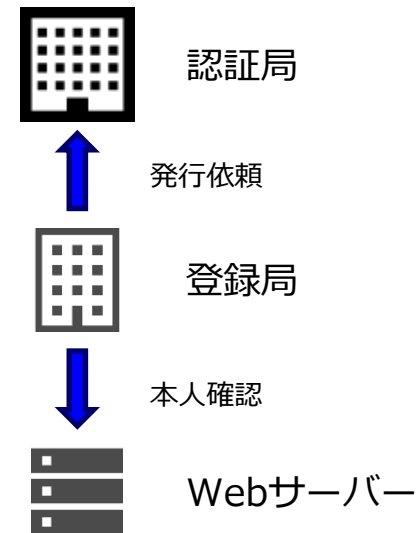
● ライフサイクル

発行手続き

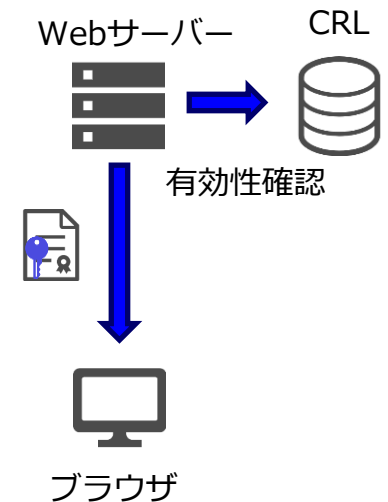
- 公開鍵証明書は発行したいサーバーの管理者が行います。
- 登録局 (RA) とは、証明書発行における身元確認を行います。発行自体はしません。
- 公開鍵証明書は、認証局から発行されます。

発行と失効管理

- 証明書発行のための要求を証明書署名要求 (CSR) といいます。認証局は受け取ったCSRから公開鍵証明書を生成します。
- リポジトリには、証明書と証明書失効リスト (CRL) を格納し、公開されます。
- OCSPステープリングとは、サーバーが公開鍵証明書を渡す時に、CRLを確認してから連携することです。



公的な公開鍵証明書を発行する際には登録局を経由する



OCSPステープリング

暗号ソリューション

公開鍵基盤 (PKI)

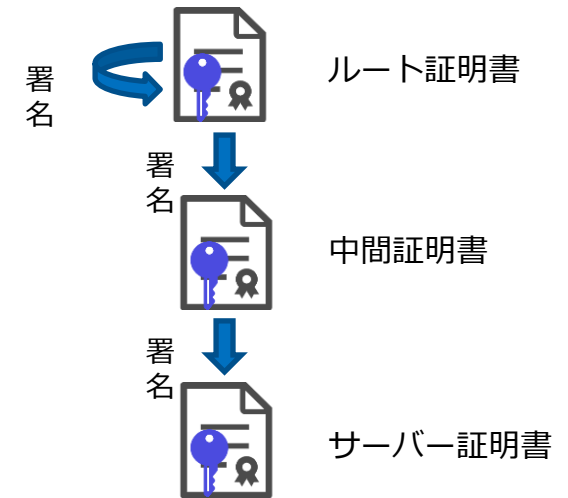
● ライフサイクル

多段的な証明書構成

- 利用者（暗号化する側）が認証局を信頼していることで、認証局が発行した公開鍵を利用しているサーバー（復号する側）の公開鍵を信頼できます。
- これは、上位の公開鍵証明書を信頼されることで、下位の公開鍵証明書が正当なものであると検証できること構成で実現されます。
- 最も信頼され信頼の幹に存在する証明書のことをルート証明書、利用者が暗号化に使う公開鍵に付与されている直接の証明書をサーバー証明書、証明書を信頼するためだけに存在する中間証明書からなります。

自己署名証明書

- 自分自身で署名した証明書を自己署名証明書と言います。ルート証明書は自己署名証明書です。
- ルート認証局は自分自身で証明書を署名・発行（自己署名証明書）します。



暗号ソリューション

公開鍵基盤 (PKI)

● 公開鍵証明書

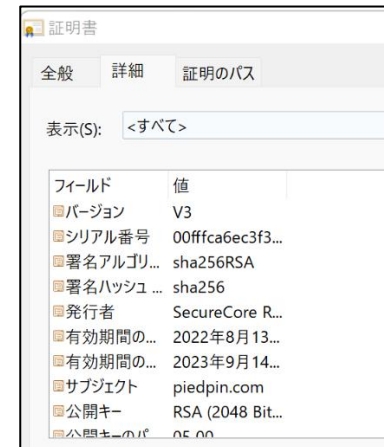
- 公開鍵証明書とは、認証局のプライベート鍵でWebサーバー側に使ってもらふ公開鍵に証明書を付けたものです。
- 公開鍵証明書のフォーマットは国際標準の X.509 によって規定されています。署名前証明書、署名アルゴリズム、署名値の3つのセクションに分かれています。
- 署名前証明書は、証明書の所有者や公開鍵、有効期限が記載されています。
- 署名アルゴリズムは、署名前証明書に対する署名をする際の署名方法です。
- 署名値は、署名アルゴリズムを使って算出される想定値です。
- EV証明書とは、証明書の所有者が合法的な企業であることを確認するワンランク上の証明書です。

公開鍵証明書

署名前証明書

署名アルゴリズム

署名値



フィールド	値
バージョン	V3
シリアル番号	00ffca6ec3f3...
署名アルゴリズム	sha256RSA
署名ハッシュ	sha256
発行者	SecureCore R...
有効期間の...	2022年8月13...
有効期間の...	2023年9月14...
サブジェクト	piedpin.com
公開キー	RSA (2048 Bit...
公開キーのバ...	05 00

署名前証明書のフィールド項目

- バージョン
- シリアル番号
- アルゴリズム識別子
- 発行者
- 有効期限
- 主体者
- 主体者公開鍵情報
- 発行者ユニーク識別子
- 主体者ユニーク識別子
- 拡張領域

暗号ソリューション

鍵管理

●ケルクホフスの原理

- 暗号は秘密鍵（つまり復号鍵）以外の全てが知られても安全であるべきという考え方です。
- 19世紀にAuguste Kerckhoffsによって提案された原理である。
- 暗号アルゴリズム、暗号文が漏洩したとしても平文に解読されてはいけません。

暗号ソリューション

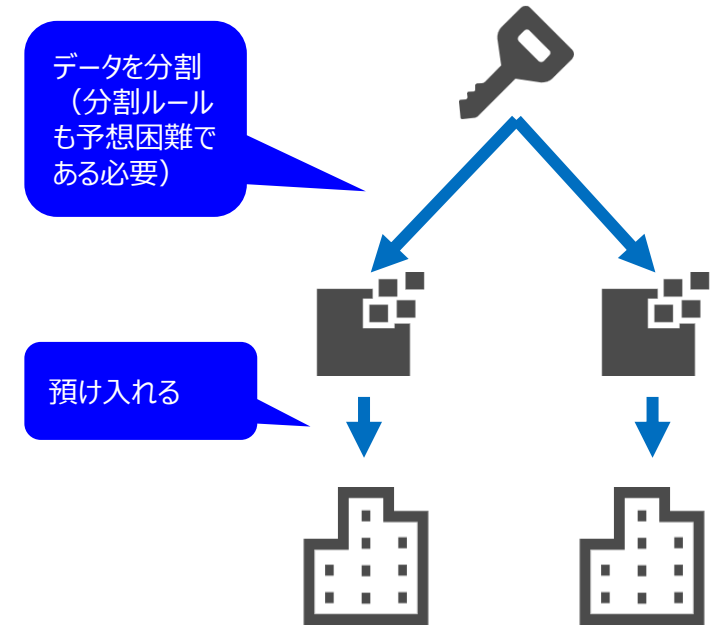
鍵管理

● 鍵供託方式（キーエスクローシステム）

- 第三者機関が公開鍵と秘密鍵のペアのコピーを保持します。
- 一つの秘密鍵を信頼されている第三者機関に保管する事は十分とは言えない。第三者機関であっても漏洩する可能性があるためです。そのため、秘密鍵を2つ以上に分割し、それぞれ異なる第三者機関に預けることで完全な鍵を作成させるまでの時間稼ぎを行う場合もあります。

● キー暗号化キー（KEK、Key-Encrypting Key）

- 鍵を暗号化する鍵です。
- たとえ暗号鍵が漏れても復号するのに、また別の暗号鍵が必要であるため安全であるとは言えるが、鍵は基本安全な場所に保管されている前提であることは変わりません。
- もしくは、ハイブリッド暗号化のような一般通信暗号化のフローにも利用されます。



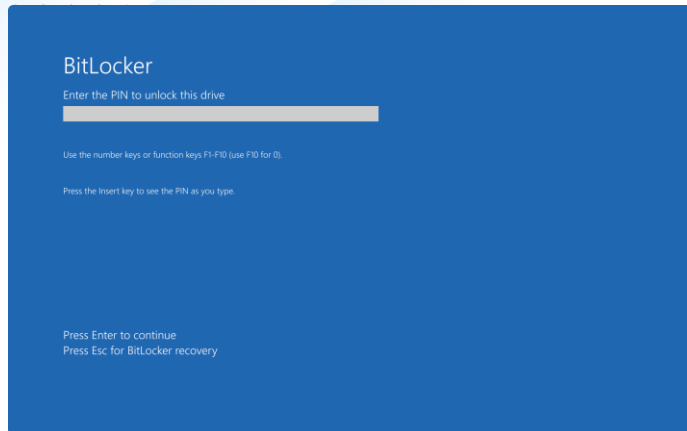
鍵供託方式（キーエスクローシステム）

暗号ソリューション

Trusted Platform Module (TPM)

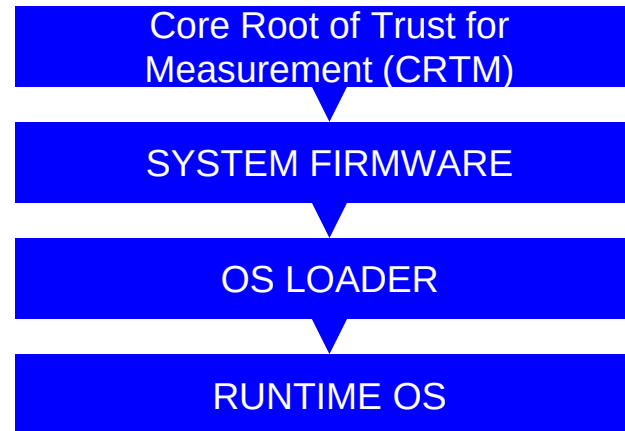
● Trusted Platform Module (TPM)

- コンピュータのセキュリティを強化するために設計されたハードウェアベースのセキュリティチップです。



BitLocker

Microsoftのディスク暗号化ツールで、TPMを使用して暗号化キーを安全に管理し、システムの起動プロセスを保護します。



セキュアブート

システムが改ざんされていないかを確認し、安全に起動するためにTPMを利用します。



デジタル証明書の発行と管理

TPMを使って安全に証明書を発行し、デジタル署名や認証プロセスを強化します。



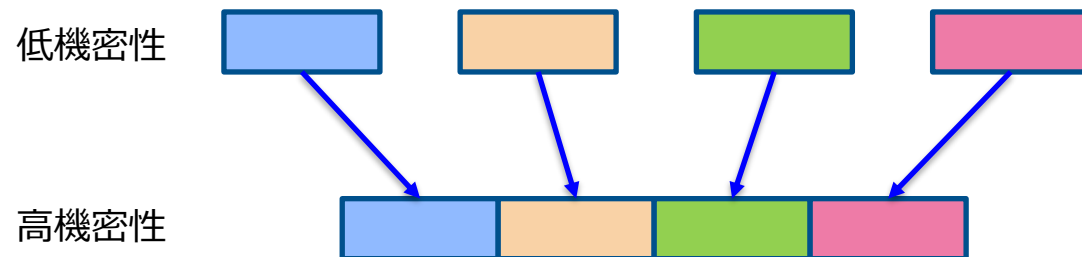
暗号解読

暗号解読

ブルートフォース

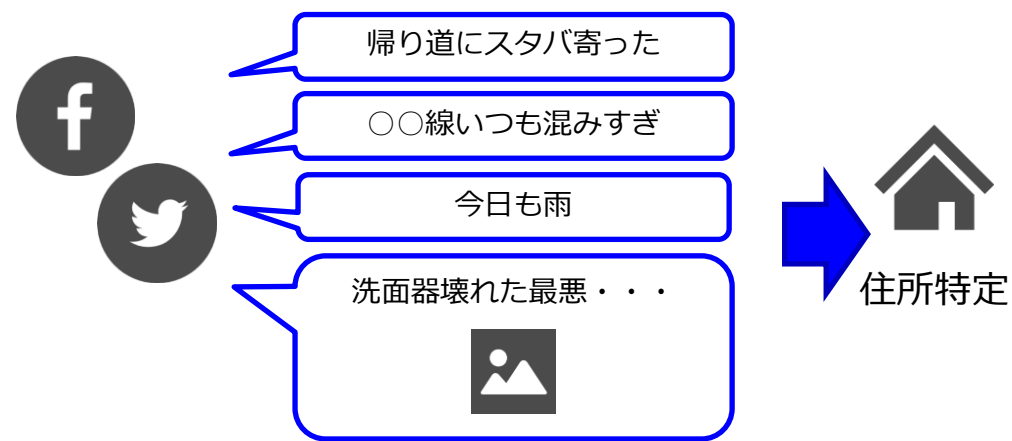
● 集約攻撃 (aggregation attack)

- 別々の情報を収集し、より強い機密情報を得ることです。
- 投票前にそれぞれの意見を事前に調査することで、投票の結果を予想することがこれに当たります。



● 推測攻撃 (inference attack)

- 公表されている情報、もしくはアクセス可能な情報からより高い機密性のある情報を得ることです。
- SNSの投稿から住所を特定する行為がこれに当たります。



暗号解読

ブルートフォース

●ブルートフォース (brute force)

- パスワードや暗号鍵などを強引に解読する手法を指します。
- この手法では、あらゆる可能な組み合わせを一つずつ試すことで、正しい解答を見つけようとします。
- ブルートフォース攻撃には時間がかかりますが、試行回数制限や多要素認証などの対策を講じないと、コンピューターの性能が高い場合には成功する可能性があります。

0000
0001
0002
...
9998
9999



LOGIN
4桁の数字パスワード
を入れてください。

● ● ● ●

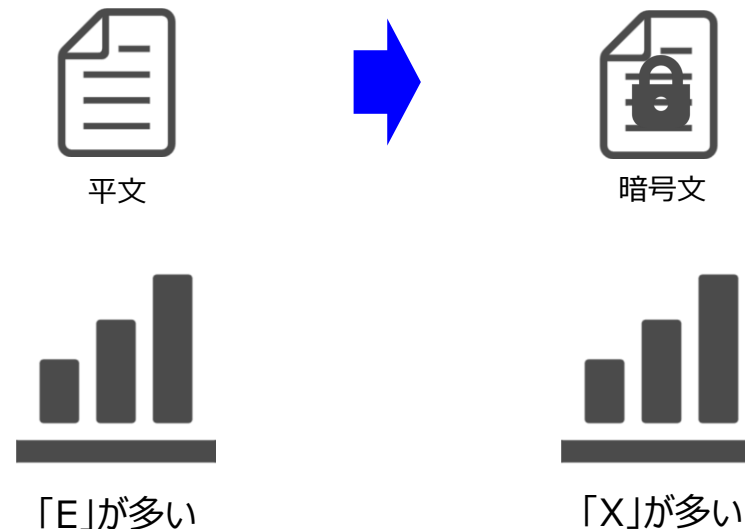
4桁の数字パスワード（0000から9999まで）をブルートフォースで解読する場合、最大で10,000回の試行で正しいパスワードにたどり着くことができます。

暗号解読

頻度分析

● 頻度分析 (frequency analysis)

- 暗号解読の手法の一つで、暗号文の中で各文字や文字列が出現する頻度を分析し、対応する平文の文字を推測する方法です。
- この手法は、一段階のみ置き換える暗号化方式であるシーザー暗号や単一換字式暗号に対して有効です。



もし暗号文中に「X」が最も頻繁に出現する場合、それは平文における「E」に対応する可能性が高いと考えられます。

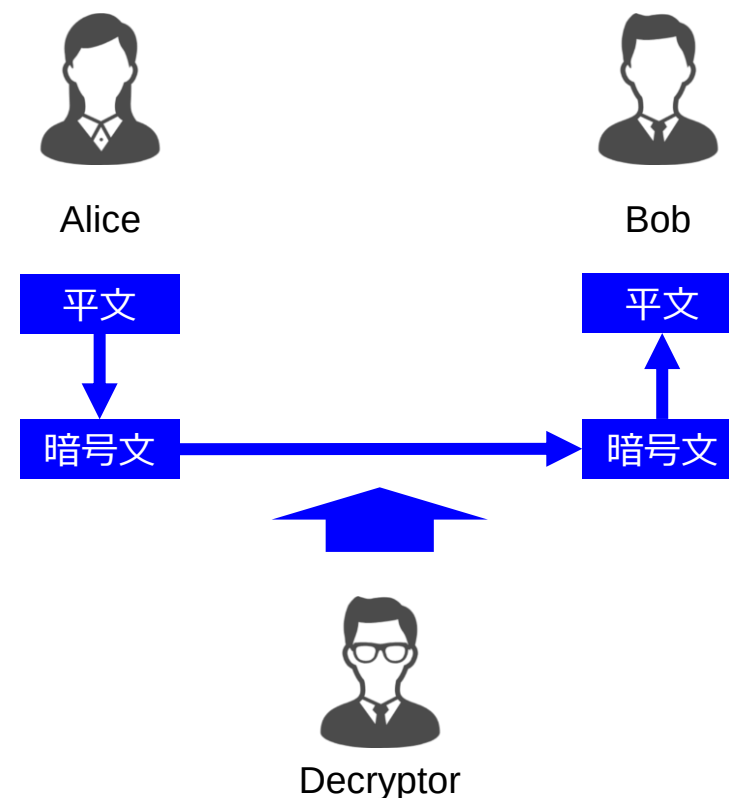
暗号解読

暗号文単独

● 暗号文単独攻撃 (cipher only attack)

- 解読者が暗号文（暗号化されたメッセージ）のみを手に入れた状態で、その暗号を解読しようとする攻撃です。
- この攻撃手法では、解読者は平文（元のメッセージ）や暗号化に使用された鍵に関する情報を持っておらず、暗号文だけを頼りに解読を試みます。

名称	選択	繰り返し	取得対象	解読難易度
暗号文単独攻撃	NO	NO	暗号文	大
既知平文攻撃	NO	NO	平文	大
選択暗号文攻撃	YES	NO	暗号文	中
選択平文攻撃	YES	NO	平文	中
適応的選択暗号文攻撃	YES	YES	暗号文	小
適応的選択平文攻撃	YES	YES	平文	小



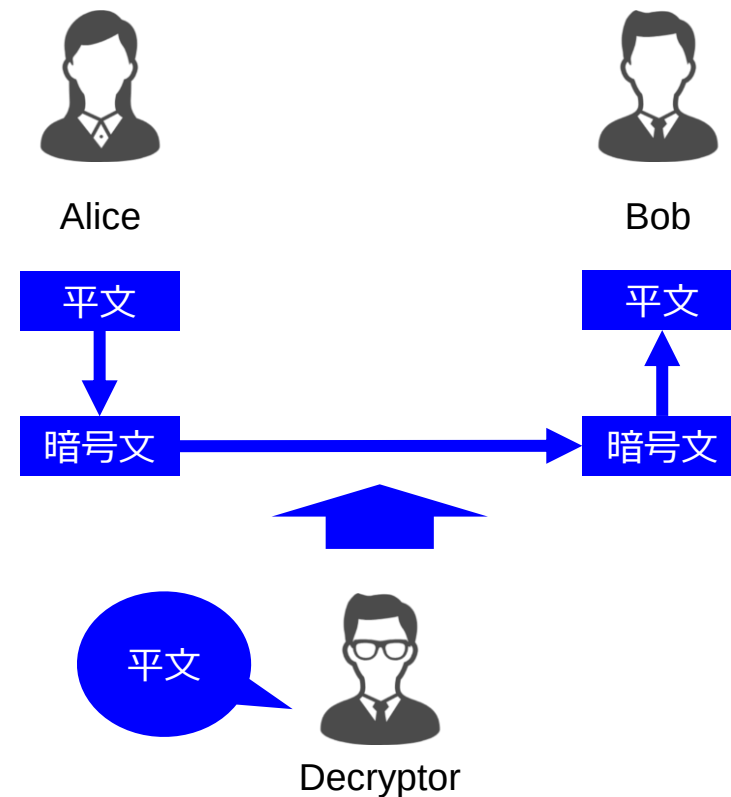
暗号解読

既知平文

● 既知平文攻撃 (known plaintext attack)

- 攻撃者が暗号文だけでなく、対応する平文（元のメッセージ）の一部もしくは全部を既に知っている状態で行う攻撃です。

名称	選択	繰り返し	取得対象	解読難易度
暗号文単独攻撃	NO	NO	暗号文	大
既知平文攻撃	NO	NO	平文	大
選択暗号文攻撃	YES	NO	暗号文	中
選択平文攻撃	YES	NO	平文	中
適応的選択暗号文攻撃	YES	YES	暗号文	小
適応的選択平文攻撃	YES	YES	平文	小



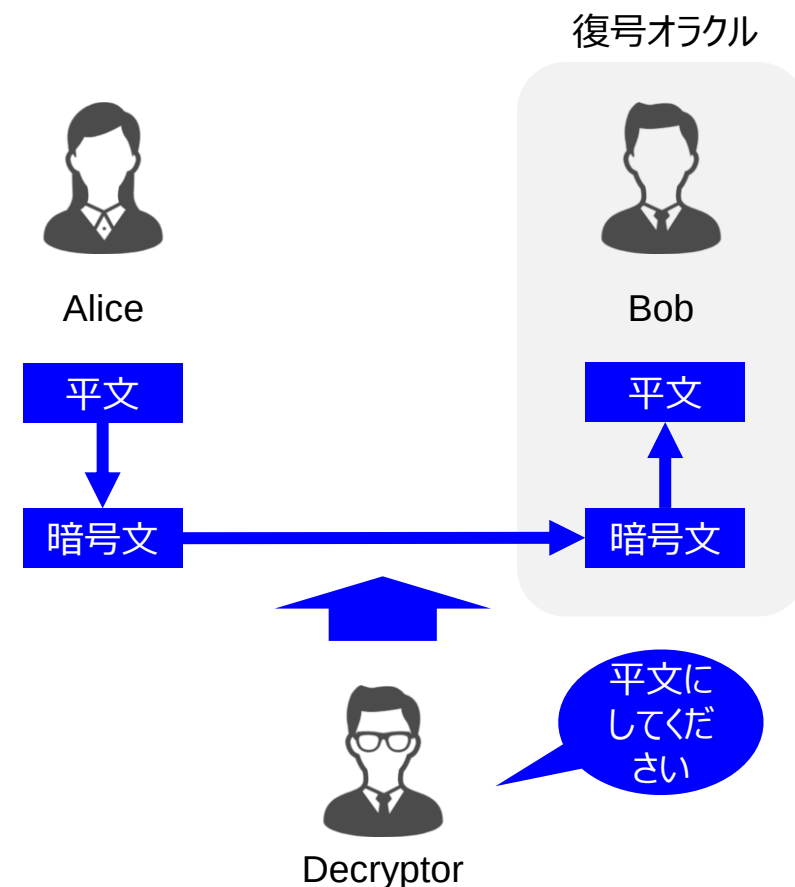
暗号解読

選択暗号文

● 選択暗号文攻撃 (chosen ciphertext attack)

- 攻撃者が特定の暗号文を選択し、その暗号文に対応する平文を取得できる権限や手段を持っている場合に行う攻撃です。
- 適応的選択暗号文攻撃 (adaptive chosen ciphertext attack) とは、解読者は取得する暗号文は自由に選択し、平文を取得できる状態です。その結果を見てから、再度取得する暗号文は自由に選択することを繰り返すことができます。

名称	選択	繰り返し	取得対象	解読難易度
暗号文単独攻撃	NO	NO	暗号文	大
既知平文攻撃	NO	NO	平文	大
選択暗号文攻撃	YES	NO	暗号文	中
選択平文攻撃	YES	NO	平文	中
適応的選択暗号文攻撃	YES	YES	暗号文	小
適応的選択平文攻撃	YES	YES	平文	小



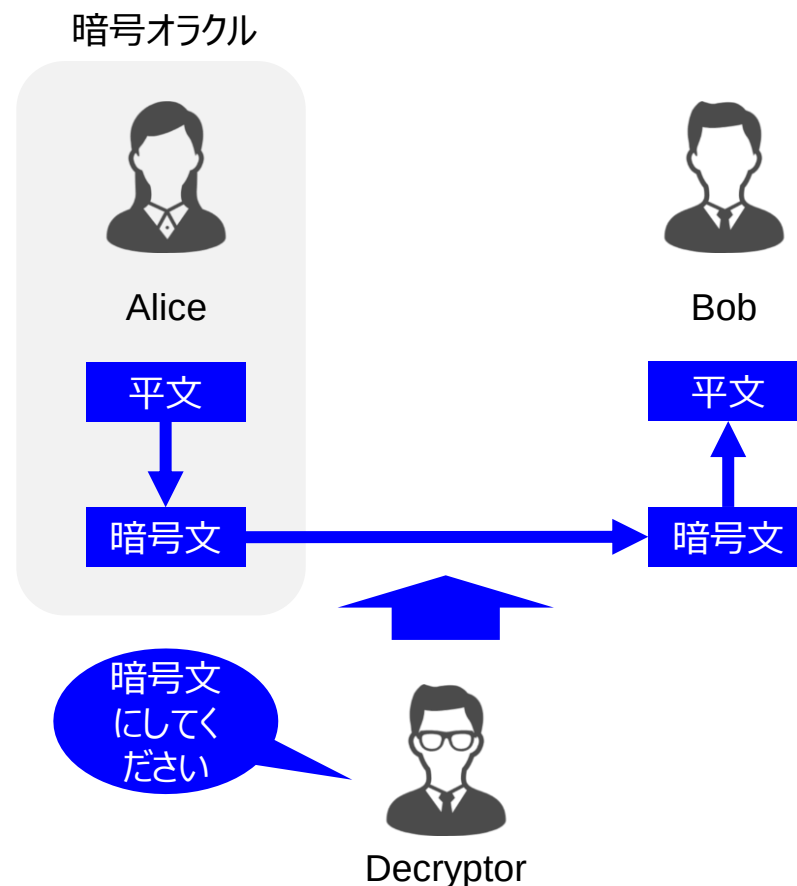
暗号解読

選択平文

● 選択平文攻撃（chosen plaintext attack）

- 攻撃者が特定の平文（元のメッセージ）を選び、それを暗号化させる権限や手段を持っている場合に行う攻撃です。
- 適応的選択平文攻撃（adaptive chosen plaintext attack）とは、解読者は取得する平文は自由に選択し、暗号文を取得できる状態です。その結果を見てから、再度取得する平文は自由に選択することを繰り返すことができる。

名称	選択	繰り返し	取得対象	解読難易度
暗号文単独攻撃	NO	NO	暗号文	大
既知平文攻撃	NO	NO	平文	大
選択暗号文攻撃	YES	NO	暗号文	中
選択平文攻撃	YES	NO	平文	中
適応的選択暗号文攻撃	YES	YES	暗号文	小
適応的選択平文攻撃	YES	YES	平文	小



暗号解読

選択平文

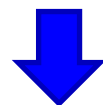
● BEAST攻撃 (Browser Exploit Against SSL/TLS Attack)

- 暗号ブロック連鎖モード (CBC) を使ったWeb通信の脆弱性を狙った攻撃です。
- 暗号ブロック連鎖モードは部分的にはれてしまうと他の部分に何が書いてあるのかも推測可能です。ブロック中1バイト以外わかっている状態だと全体の暗号文の中身を推測できるといわれています。

リクエスト電文

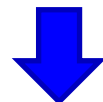
```
POST /AAAAAA HTTP/1.1
Host: www.example.com
Cookie: secret=abcde
```

※黒は既知、赤は未知 ? で表現します。



暗号ブロック (16バイト) に変換

```
POST /AAAAAA HTTP/1.1¥r¥n?????? ??????????????????
```



パスに1234を挿入し、再リクエスト。
挿入されたパス分ブロック内の文字列がずれる。

```
POST /AAAAAA1234 5678 HTTP/1.1¥r¥n? ??????????????????
```

最後の1バイトのみが不明となる。

- ? に適当な文字を入れ、元の同じ暗号文が得られたとき、元の平文だと確認できる。
- 1バイトはビット演算で256個しかないため、最大で256回繰り返し操作で、何の文字かがわかってしまう。

暗号解読

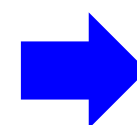
選択平文

●CRIME攻撃 (Compression Ratio Info-Leak Mass Exploitation Attack)

- SSLやSPDY (あるいはHTTPボディ部のgzip圧縮) で使われる圧縮アルゴリズムに注目した攻撃手法です。
- SSLには圧縮機能が存在しており、元データの中に似たようなデータが繰り返されるほど圧縮されるため、もし攻撃者が任意の文字列を元の平文に埋め込んだ場合には推測可能となります。

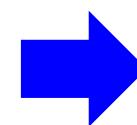
```
POST /index HTTP/1.1  
Host: www.example.com  
Cookie: secret=abcde
```

圧縮



```
POST /index HTTP/1.1  
Host: www.example.com  
Cookie: secret=abcde  
Cookie: secret=12345
```

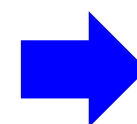
圧縮



圧縮率が下がる

```
POST /index HTTP/1.1  
Host: www.example.com  
Cookie: secret=abcde  
Cookie: secret=abcde
```

圧縮



圧縮率が上がる

暗号解読

実装攻撃

●実装攻撃 (implementation attack)

- エラーや欠陥だけでなく、暗号化システムをプログラムするために採用された方法論など、ハードウェアやソフトウェアコードを悪用することに重点を置く攻撃です。
- 基盤に対する攻撃であり、リバースエンジニアリング攻撃、サイドチャネル攻撃、フォールト攻撃、タイミング攻撃、ロジカル攻撃に分かれます。

●リバースエンジニアリング (reverse engineering)

- すでに出荷済みの他社の製品に対して、分解や解析などを行い、その内部処理構造を明らかにすることです。



設計



アプリやサービス



設計



※逆コンパイルなど



アプリやサービス

元の設計と完全に一致させるのではなく、解析によって元の設計を推測します。

暗号解読

実装攻撃

● サイドチャネル攻撃

- 物理的な情報からシステムデータを盗聴する攻撃です。
- 通信やコンピュータ処理するときには、電波などがデータを送る機器から発してしまう。電波などはシステムデータの処理に応じて変化することがあり、その挙動を読み取ります。

電力解析攻撃

- 暗号を処理する機器に対して、様々な入力情報を与えて、消費電力の違いから暗号方法を推測する攻撃です。
- 消費電力が激しければ、その分計算が複雑な暗号アルゴリズムを利用していることを目安として推測するなどが挙げられます。

史上最悪のクラッカーを捕まえた日本人

下村 努は世界でも有名なホワイトハッカーの一人です。全米史上最悪のクラッカーと言われたケビン・ミトニックを逮捕へと追い込んだ経歴を持ちます。

1995年頃、全米中のコンピューターから約2万件のクレジットカード情報を盗み出される事件が発生しました。捜査に協力された下村は監視を続けるうちに犯人がケビン・ミトニックだと確信します。最後は、アンテナを持ち町中を歩き回り、ミトニックの居所を突き止め、逮捕へとつなげることができました。

暗号解読

実装攻撃

●フォールト攻撃 (fault injection)

- 稼働中のデバイスに対し、故障のような事象（フォールト）を注入し、誤動作を発生させる攻撃です。
- フォールトを入れたシステムは、正常系の時以上にシステム内部の処理構造や、誤動作による機密領域の開示を発生させる可能性があります。
- また、誤動作処理時にサイドチャネル攻撃と組み合わせて実行されます。

ドラクエをホットプレートで温めて22分でクリア

RTA in Japan 2020において、ドラクエ3のクリア時間はおよそ40時間かかりますが、約22分でゲームクリアに到達し、世界記録を更新しました。ファミコン本体をホットプレートで暖めることで、電源ON/OFFによる意図的にゲームデータを崩壊させられるバグを発生しやすくし、このタイムになったとのこと。



暗号解読

実装攻撃

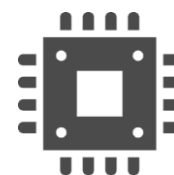
● タイミング攻撃 (timing attack)

- 暗号を処理する機器に対して、様々な入力情報を与えて、処理時間の違いから暗号鍵などを推測する攻撃です。
- 処理時間がかかっていれば、その分処理として正常に進んでいることを目安として推測できるなどが挙げられます。

計算しやすかった？



早い

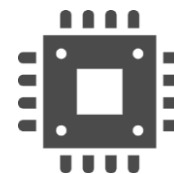


暗号化処理

計算しにくい？



遅い



暗号化処理

処理にかかった時間を精密に測定し、統計的に分析することで鍵を推測します。

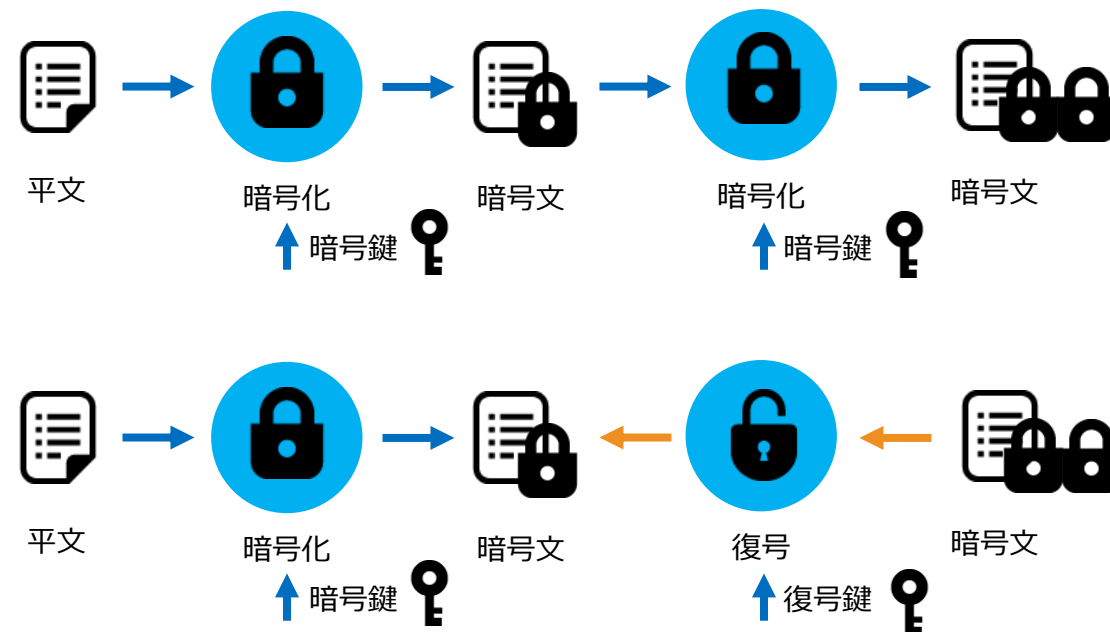
暗号解読

実装攻撃

●ロジカル攻撃

中間一致攻撃 (Meet-in-the-middle attack)

- 暗号化と復号を同時に行っていくことで、鍵を取得する攻撃です。
- 2段階の暗号化では平文の暗号化と暗号文の復号を同時に行えば、適切な鍵を選択している場合には、値が一致するものであり、2重の暗号強度にはなりません。
- 共通暗号化方式であるDESに脆弱性が公表されたとき、2回暗号化する(2DES)ことが考えられましたが、中間一致攻撃が理由で適切ではなくなりました。



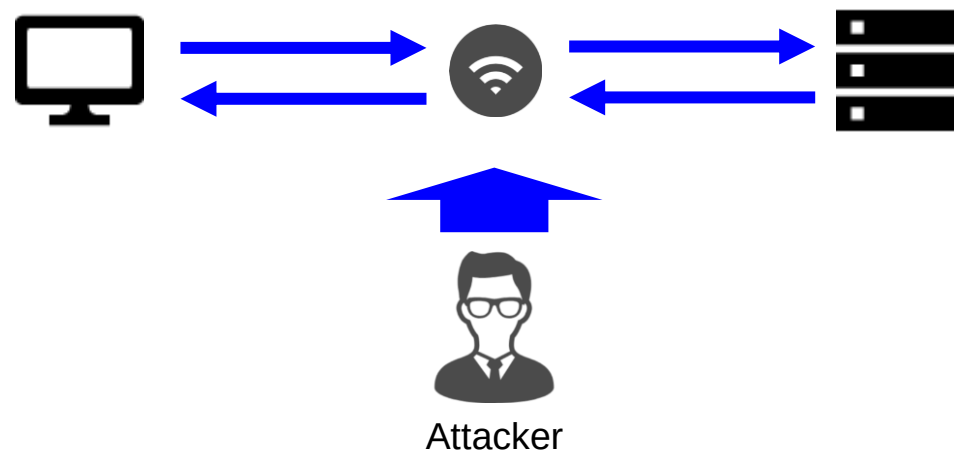
- 推測される鍵で暗号化と復号を並列して行い一致した場合、正常な鍵であると推測できる。
- 推測するための試行回数は、1回暗号した時とさほど変わらない。

暗号解読

中間者攻撃 (MITM)

● 中間者攻撃 (Man-in-the-Middle)

- 通信している2者間のデータのやり取りに第三者が密かに介入し、通信内容を盗聴したり、改ざんしたりする攻撃手法です。
- 攻撃者は、あたかも合法的な通信相手であるかのように振る舞い、両者の間に立ってデータを操作します。



暗号通信を解読するのではなく、通信間に入り込みます。

暗号解読

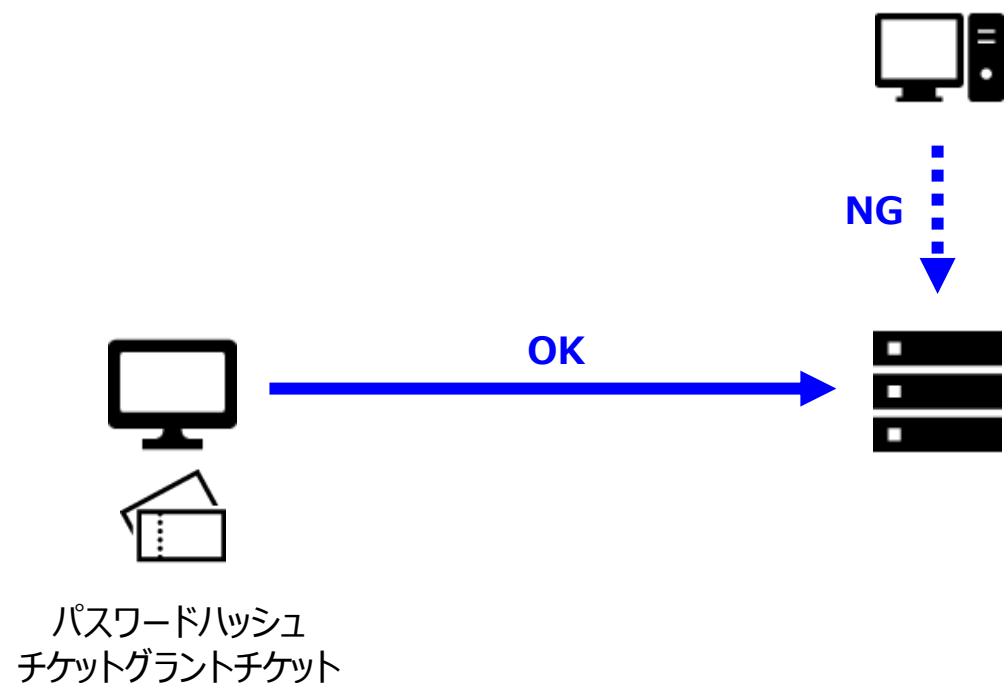
パス・ザ・ハッシュ

●パス・ザ・ハッシュ (pass the hash)

- 攻撃者がユーザーの平文パスワードを知らなくても、ハッシュ化されたパスワード（ハッシュ）を利用して認証を行う攻撃手法です。

●パス・ザ・チケット (pass the ticket)

- Kerberos認証プロトコルを使用する環境において、攻撃者が認証チケット（チケット・グラント・チケット、TGT）を盗み、それを使ってシステムに不正アクセスする攻撃手法です。
- この攻撃は、チケットを盗むことで、ユーザーのパスワードを知らずにそのユーザーの権限でシステムやネットワークリソースにアクセスできるようになる点で、パス・ザ・ハッシュ攻撃と似ています。



一度認証が成功した端末には、再認証不要の情報が残っている

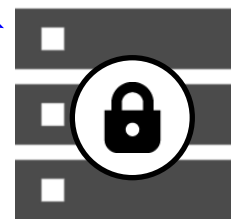
暗号解読

ランサムウェア

●ランサムウェア (ransomware)

- コンピュータやネットワーク内のファイルを暗号化してアクセス不能にし、復号のための身代金を要求する悪意のあるマルウェアの一種です。
- ランサムウェア攻撃は、個人や企業、さらには政府機関に対しても実行され、多くの場合、暗号通貨（ビットコインなど）での支払いが求められます。

業務に利用
できない



業務に利用しているコン
ピューターを暗号化

業務を再開するには復号鍵が必要になる。
身代金を支払っても復号される保証はない。

暗号解読

ランサムウェア

●ランサムウェア

CryptoLocker

- ランサムウェアの火付け役となったマルウェアです。
- Microsoft Windowsが動作しているコンピューターを標的とし、感染したコンピューターを暗号化して使えなくし、身代金を要求します。
- このマルウェアを作った犯人は、FBIに300万ドルの懸賞金をかけられ、捕まりました。

WannaCry

- 身代金にビットコインを要求するランサムウェアです。
- 凍結させた端末を開放する身代金を銀行口座やクレジットカードからの引き出させるのではなく、ビットコインを要求します。

生年月	被害を受けた医療機関	被害内容
2022/1	春日井リハビリテーション病院（愛知県）	電子カルテがランサムウェア攻撃を受け、患者約5万人分のデータが暗号化バックアップはも暗号化された
2022/1	日本歯科大学付属病院（東京都）	歯科・医科の電子カルテと会計システムが暗号化
2022/4	青山病院（大阪府）	電子カルテがランサムウェアに感染
2022/6	鳴門山上病院（徳島県）	ランサムウェアによる攻撃で、電子カルテや院内LANが使用不可にオフラインでバックアップしていたデータで復旧

<https://insights-jp.arcservice.com/ransomware-hospital>

2022年度に発覚した病院を狙ったランサムウェア被害（一部）



施設の設計

施設的设计

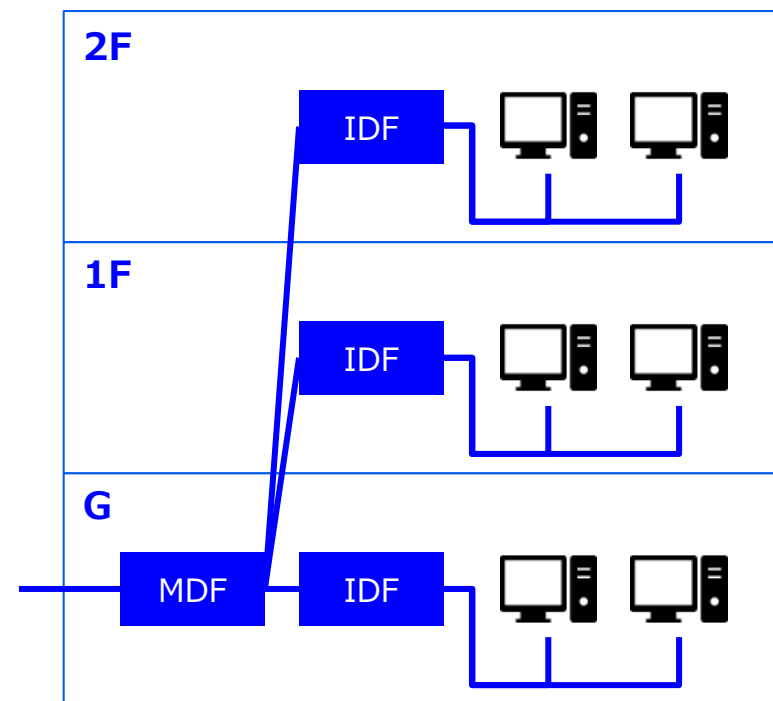
配線クローゼット／中間配線フレーム

● 配線クローゼット (Wiring Closet)

- 配線クローゼットは、ビルや施設内でネットワークケーブルをまとめるための部屋またはエンクロージャーです。

● 中間配線フレーム (IDF、Intermediate Distribution Frame)

- 中間配線フレーム (IDF) は、配線クローゼットの中にある、あるいはクローゼットとは別の場所に設置されたネットワーク接続ポイントです。
- IDFは、建物内での配線の中継地点として機能し、メインの配線クローゼット (Main Distribution Frame) との間で接続を行います。



各フロアにIDFを配置する

施設的设计

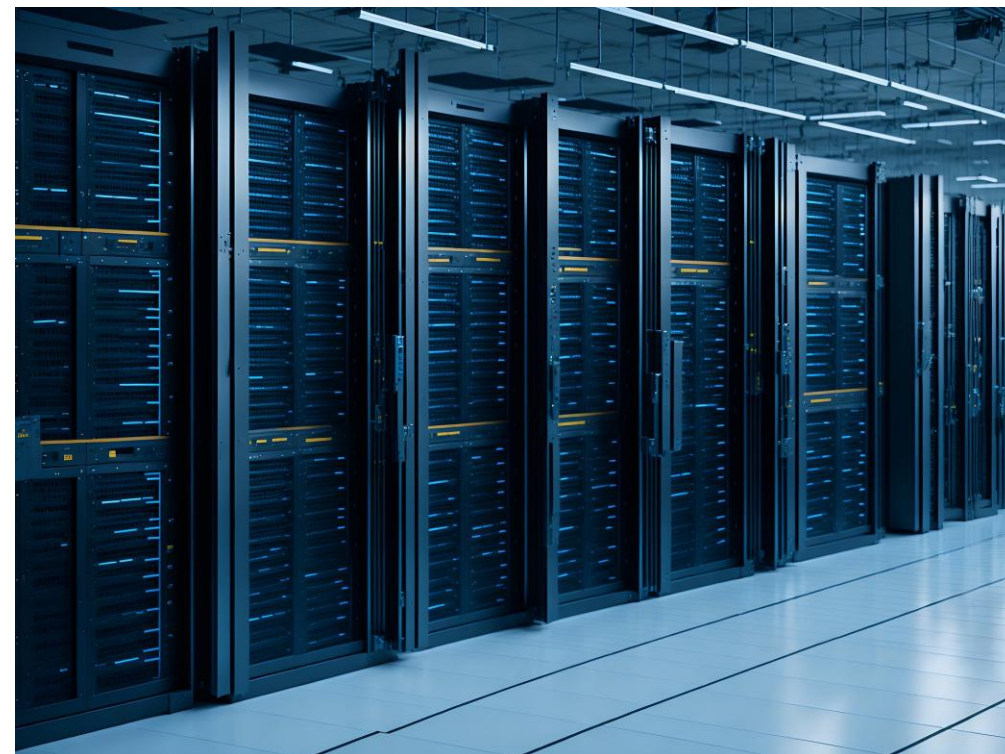
サーバールーム／データセンター

●サーバールーム（Server Room）

- ・サーバールームは、特定のビルや施設内に設置された比較的小規模な部屋で、ネットワーク機器やサーバーを管理・運用するための専用スペースです。

●データセンター（Data Center）

- ・データセンターは、サーバールームよりも大規模な施設で、数百から数千台のサーバーや関連機器が収容されている場所です。多くの場合、データセンターは大規模なインターネットサービスプロバイダ（ISP）やクラウドプロバイダが運営しています。



施設の設計

メディアストレージ施設

●メディアストレージ施設（Media Storage Facility）

- デジタルおよび物理的なメディアの安全な保管、管理、アーカイブを目的とした専用の施設です。
- データのバックアップ、リカバリ、およびアーカイブが可能で、長期にわたりデータの可用性と整合性を維持します。
- テープ、フィルム、CD、DVD、ブルーレイディスク、ハードドライブなど、物理的なメディアの保管場所も確保されていることが一般的です。



施設の設計

制限区域

● 施設設計

訪問者の扱い

- 多くの場合、訪問者には同伴者が付き、施設内の行動は厳密に監視されます。
- ゲストカードを付与し、低い権限内でのアクセスのみを許可します。

受付

- 受付エリアは、受付経由でセキュリティエリアに入らないように、セキュリティエリアから隔離され、すべての入館者を記録するようにセキュリティカメラで監視する必要があります。
- 訪問者が施設内に入る際の同伴者を受付で割り当てすることもできます。

パーティション

- 異なる作業エリアを区切るために使用します。
- ショルダーサーフィンや盗聴を抑止することもできます。



施設の設計

制限区域

● 施設設計

テールゲート/ピギーバック

- 入館証提示などが必要なゲートで、前の人が開けた際にゲートをくぐることです。
- 入館の記録はシステムで記録されているため、基本的に後に訴求もされる可能性があります。ドア付近には監視カメラの併設も有効です。

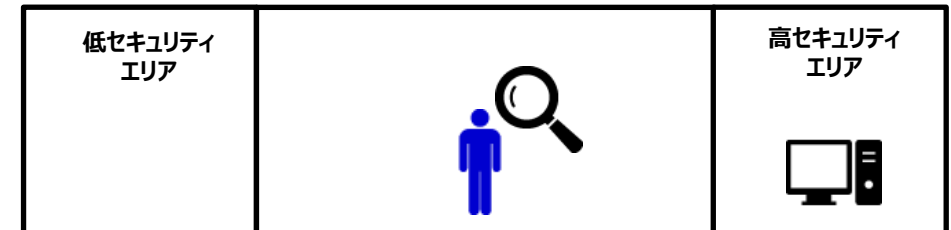
マントラップ

- ドアを連動させ、テールゲート/ピギーバックを防ぐための方法です。
- セキュリティレベルを満たしていない、もしくは不審な行動を行っている者の入室・入館を止めることができます。
- 一般にデータセンターのコンピュータ室などで見られます。

①入るときはもう片方のドアが閉まっている。



②室内で監査される。不正があれば閉じ込める。



③問題なければ、反対側のドアが開く。



施設の設計

制限区域

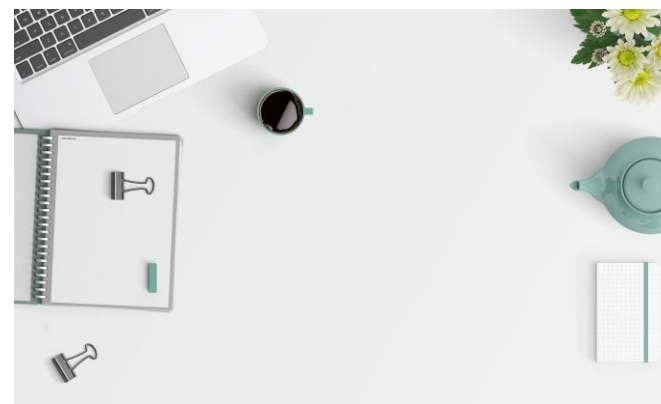
● 施設設計

機密区画情報施設（SCIF）

- 政府機関や軍事機関、部門、請負業者が、機密性の高いデータの保存や計算のための安全な環境を提供するためによく使用されます。
- 機密区画情報施設は保護された領域内で写真、ビデオ、記録装置の使用または所有することを禁止しています。

クリーンデスクポリシー

- 各作業時間の終わりに、どのように、なぜ机の上を掃除するのかを従業員に指示するために使用されます。
- 机の上の資料には、パスワード、財務記録、医療情報、機密性の高い計画やスケジュールなどの機密資料が含まれる可能性があります。



施設的设计

暖房、換気、空調 (HVAC)

● HVAC

- 機器が壊れないような環境づくりの要素、温度 (heating) と換気 (ventilation) と空調 (air-condition) の頭文字です。
- データセンターでは、湿度は40-55%、温度を20-25℃に保つ必要があります。

● 静電気放電 (ESD、Electro-Static Discharge)

- ESDを抑える湿気は40-60%といわれており、静電気モーションセンサーで検知します。
- サーバは精密機械であり、瞬間的な静電気で故障を引き起こす可能性がある。その上、人間は静電気を発生しやすくサーバ室に入室する際には、ゴム靴やサーバに触る際には放電シートに触れる必要がある。

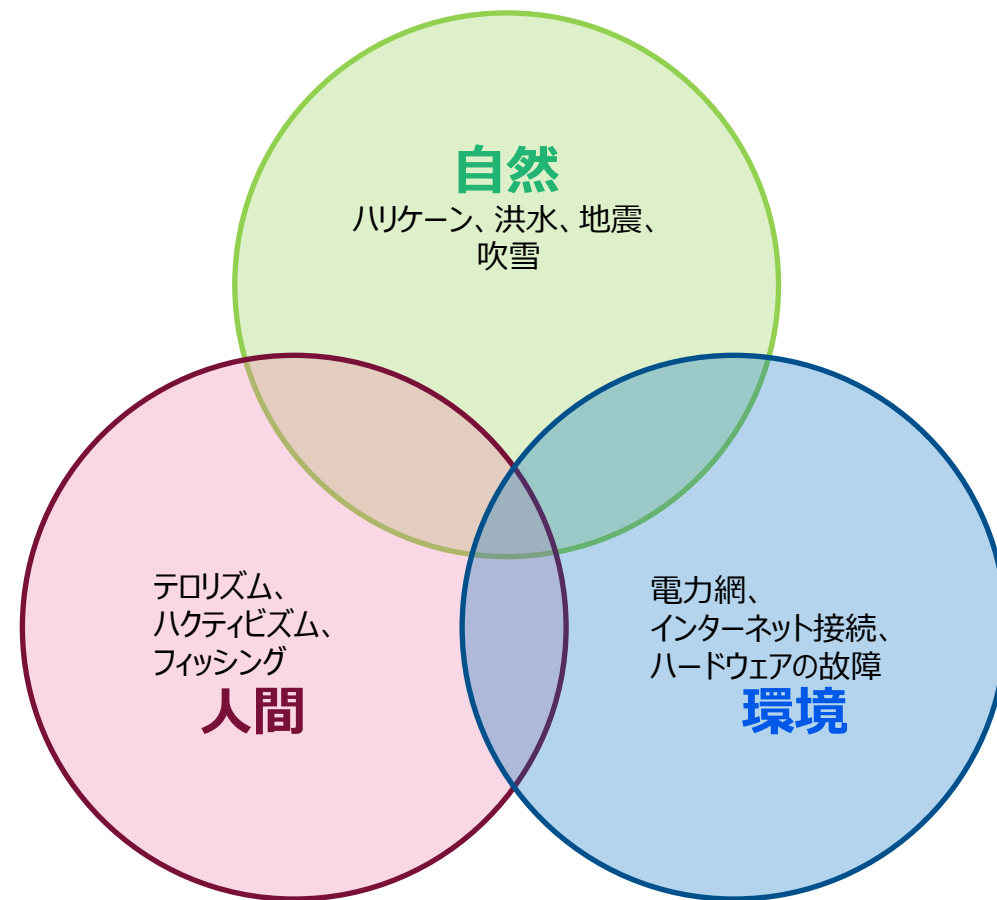
静電気力	損傷ケース
40	敏感な回路および他の電子部品の損傷
1000	モニターディスプレイの乱れ
1500	ハードドライブなどの損傷
2000	急なシャットダウン
4000	プリンタの詰まりやその他要素の損傷
17000	回路に永続的な損傷

施設の設計

災害のタイプ

● 災害の分類

- “自然”とは、ハリケーン、洪水、地震、吹雪など、自然に起因するものです。
- “人間”とは、テロリズム、ハクティビズム、フィッシングなど、人間が意図的または無意識に行うヒューマンエラーです。
- “環境”とは、電力網、インターネット接続、ハードウェアの故障、ソフトウェアの欠陥など私たちが働いている環境に起因するものです。



施設的设计

火災

● ABCDファイヤー

- 燃えるものをカテゴリー化して消火方法を分類する考え方です。
- 油が燃えている時に水をかけてはいけないというのは有名です。発火原因は様々ですので、迅速に安全に消火するための知識となる。

ハロン

- 消火などに使われていたが、クロロフルカーボンを含んでいる危険な物質です。
- 空気中にハロンが10%以上存在する場合は、人体に危険となる。
- モントリオール議定書により、オゾン層破壊する要因になるということで規制されており、その後も環境保護法・ハロン放出規制法・大気浄化法によって使用が禁止となっている。
- ハロンの代替品としてアルゴン、FE-13、FM-200、INERGENなどがある。

クラス	燃物	消火物
A	木、プラスチック	水、ソーダ酸
B	油、ガス	二酸化炭素、ソーダ酸
C	電気系統	二酸化炭素
D	鉄	ドライパウダー
K	キッチンオイル、化学混合物	湿式化学製品



施設の設計

火災

●スプリンクラー

- 火災の際に水を放出する装置です。通路などの施設内でも火が広がりやすい場所に設置されています。

閉鎖型湿式スプリンクラー

- 配管内に常時水を充満されており、火災時にスプリンクラーヘッドが熱を感知することにより自動的に消火します。

閉鎖型乾式スプリンクラー

- スプリンクラーヘッドから流水検知装置までの間に、圧縮空気を充填しています。
- 火災が発生したときスプリンクラーヘッドが解放されると、配管内の空気が抜け、流水検知装置が作動し、スプリンクラーヘッドまで水が登り、散水します。
- 寒冷地法では配管の水が凍結してしまう可能性があるため、閉鎖型乾式スプリンクラーが使われます。



閉鎖型湿式スプリンクラー



閉鎖型乾式スプリンクラー



施設的设计

火災

●スプリンクラー

閉鎖型予作動式スプリンクラー

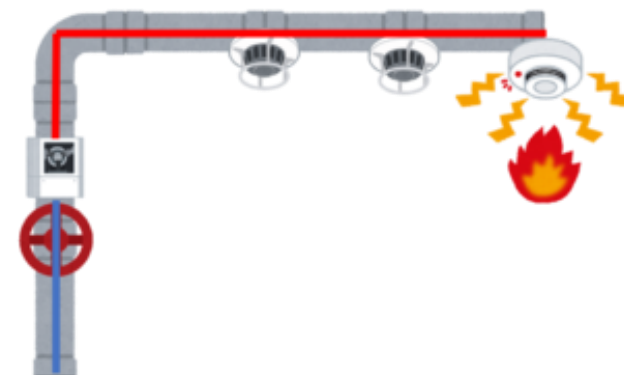
- スプリンクラーヘッドから流水検知装置までの間に、圧縮空気を充填しており、配管内の水の凍結を防ぎます。
- その上、火災発生探知機で火災を確実に検知した上で、スプリンクラーヘッドが解放され、散水します。
- コンピューター室などの精密機器が並ぶ部屋では誤作動による散水が大きな損失につながるため、閉鎖型予作動式スプリンクラーが使われます。

開閉型スプリンクラー

- スプリンクラーヘッドは常に解放された状態で設置します。
- その上、火災発生探知機で火災を確実に検知したら、水がスプリンクラーヘッドまで到達し、そのまま散水します。
- 多くの人々が混雑する劇場などの舞台部などに用いられます。



閉鎖型予作動式スプリンクラー



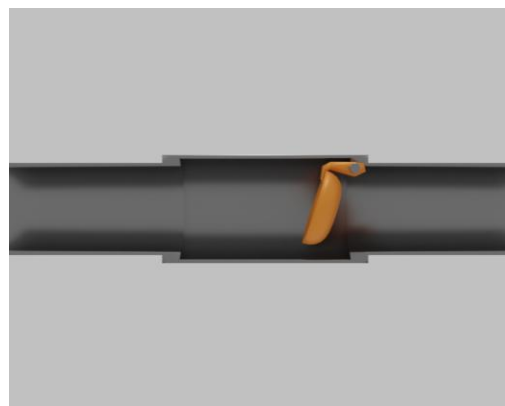
開閉型スプリンクラー

施設の設計

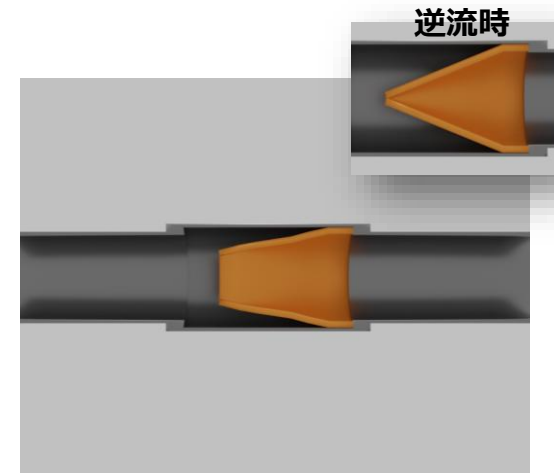
火災

●バルブ

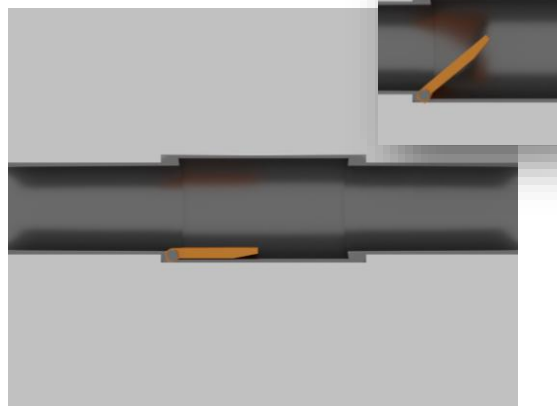
- クラッパー弁（clapper valve）とは、片方にしか開かない小窓がついているタイプです。
- ダックビル弁（duckbill valve）とは、出口が細くなっていて逆流しないタイプです。
- バックウォーター弁（backwater valve）とは、排水溝で水が逆流しないようにするタイプです。
- ダイアグラム弁（diagram valve）とは、蛇口をひねって止めるタイプです。



クラッパー弁



ダックビル弁



バックウォーター弁



ダイアグラム弁

施設の設計

電力

●電気系障害

電力停止への対応

- ・サージプロテクターとは、サージが発生したときに、機器にダメージにならぬように電力を抑える。
- ・無停電電源装置（UPS、Uninterruptible Power Supply）とは、電気が供給されなくなった時、蓄積していた電力を供給する。
- ・発電機とは、自ら電気を生成することで、UPSよりも長い期間電気を供給する。

名称	停止時間	供給量
ブラックアウト	長期	無し
ブラウンアウト	長期	低い
フォールト	単発	無し
サージ	長期	高い
スパイク	単発	高い
サグ	単発	低い

電気系障害

施設的设计

物理的検知

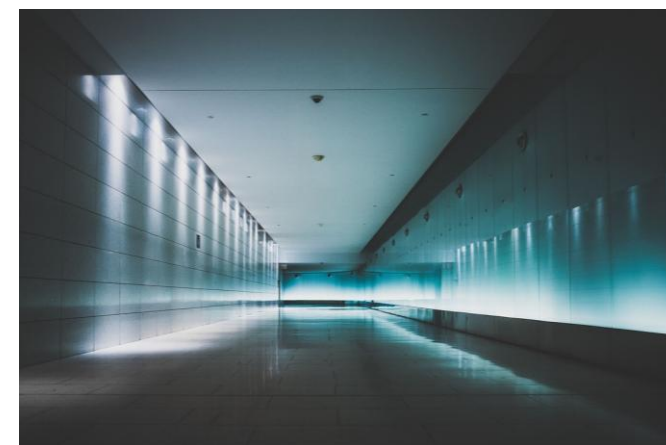
● 監視カメラ

- ・ 監視カメラを目立せることで、不正行動を抑止することもできます。
- ・ CCTVを利用し、記録データはリアルタイムに遠隔地に保管することで監視カメラを破壊され、データを失うことを防止できる。また、遠隔地（安全なところ）から監視活動を行うことが可能になる。
- ・ 監視カメラのデータは、否認防止の役割も果たすことができる。



● 照明

- ・ 連続照明とは、常に照明し見渡します。照明されることで、侵入者へのけん制や、作業の効率化を図ります。
- ・ トリップ照明とは、センサーに引っかかった時に点灯します。侵入者の発見、もしくは電力の節約を目的に、センサーに反応した場合に光を照らします。
- ・ 予備照明とは、他の照明がつかなくなったとき用の予備の照明になります。
- ・ 非常用照明とは、施設の電気が落とされると点灯します。蓄光など電源設備に頼らない照明製品が利用されます。



施設的设计

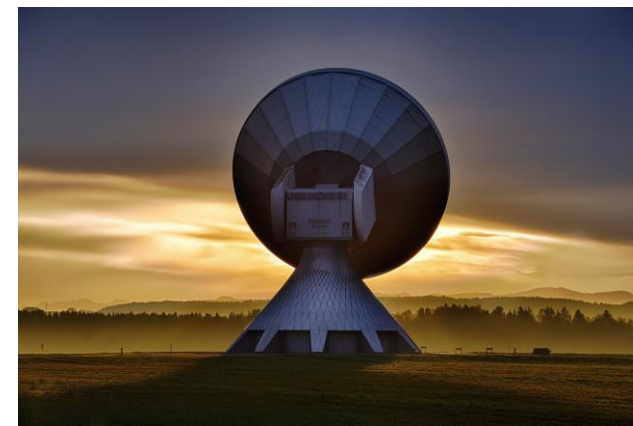
物理的検知

● アンテナ

- ホイップ（ロッド）アンテナとは、携帯電話などに付属している小型アンテナです。
- ダイポールアンテナとは、アマチュア無線などで使うアンテナです。
- 八木・宇田アンテナとは、テレビアンテナなどに使うアンテナです。
- パラボラアンテナとは、衛星放送の受信に使うアンテナです。
- ループアンテナとは、導体部分を環状のコイルにしたアンテナです。
- 誘電体アンテナとは、高周波用誘電体セラミックスで小型かつ高機能のアンテナです。



八木・宇田アンテナ



パラボラアンテナ

施設の設計

物理的防衛

● ドア

- 空洞タイプとは、ドアの中が空洞になっているタイプのドアです。
- ソリッドタイプとは、中身も埋められている堅守なタイプのドアです。

● ゲート

- 建物や敷地への出入口や、空港の搭乗口です。

クラス	施設
クラスⅠ	住宅
クラスⅡ	ビル
クラスⅢ	工場
クラスⅣ	公共機関

● ガラス

- 透過性のある壁は、区域を分断しながら目視での確認を行うことができる。
- 同様の防衛性能を持つ資材の強度を比べると、以下のようになる。

平面ガラス < 強化ガラス < ワイヤーガラス < アクリル < ポリカーボン



施設の設計

物理的防衛

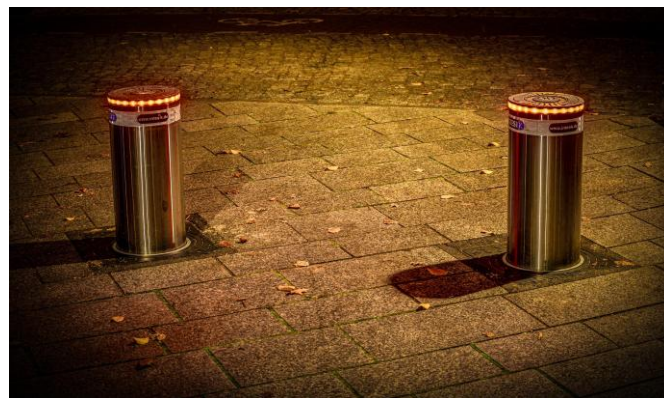
●フェンス

- フェンスは侵入を完全に防ぐという目的ではなく、侵入もしくは脱走の際の時間稼ぎとして有効です。
- 一般に壁を設置するよりも安価であり、移動も容易であるが代替品ではありません。

高さ	想定される攻撃者
4フィート（1.2m）	一般的な不法侵入を抑止する。区分けするということ合いに近い。
6フィート（1.8m）	一般的な不法侵入を抑止する。侵入させない意図が明確になるレベル。
8フィート（2.4m）	強い侵入者の抑止する。破壊すること考えられる。

●ボラード

- 施設の入り口など設置する杭です。
- 頑強に作られており、破壊することは難しい。車の衝突などの強引な侵入を防ぐ効果があります。



施設的设计

物理的防御

● 鍵

鍵の種類

- シリンダーキーとは、挿入部の凹凸が内部のばねを適切な位置まで教えることで解錠する鍵です。安全性は低くピッキングしやすい。
- デンプルキーとは、家の鍵などで利用する鍵の平面に凹凸を持たせた鍵です。
- ウォードキーとは、鍵柱が長く、先端に特有の形状がある。
- ダイヤルキーとは、金庫で使われている鍵です。適切にダイヤルを回せば、物理的に鍵と同じ役割で解錠する。

鍵の使い道

- マスターキーとは、特定のエリアまたはセキュリティゾーンのロックを解錠する事が可能です。誰が持っているのか、どこに保管しているのか、適切に管理する必要があります。
- コアキーとは、「交換可能なコアロック」を取り外すために使用されます。交換可能なコアロックとは、8の字型の鍵の施業部分のことです。



シリンダーキー



デンプルキー



ウォードキー



ダイヤルキー

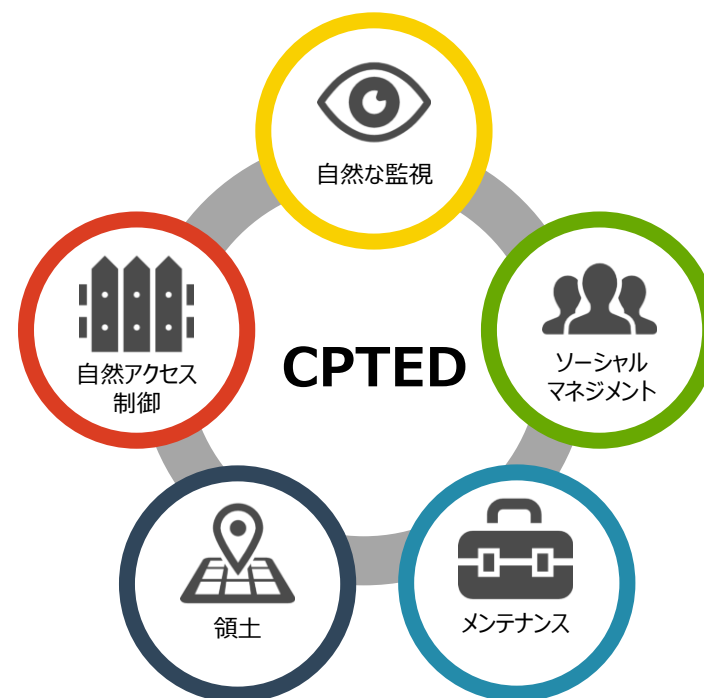


施設の設計

防犯環境設計

●CPTED (Crime Prevention Through Environmental Design)

- CPTEDは、施設のデザイン、景観、エントランスのコンセプト、キャンパスのレイアウト、照明、道路の位置、車両と徒歩の交通管理などを扱います。
- CPTEDの基本理念は、「物理的環境のデザインは、意図的に管理、操作、細工することができ、その場所にいる人々の行動効果や変化を生み出し、犯罪の減少や犯罪に対する恐怖の減少につながる」とです。





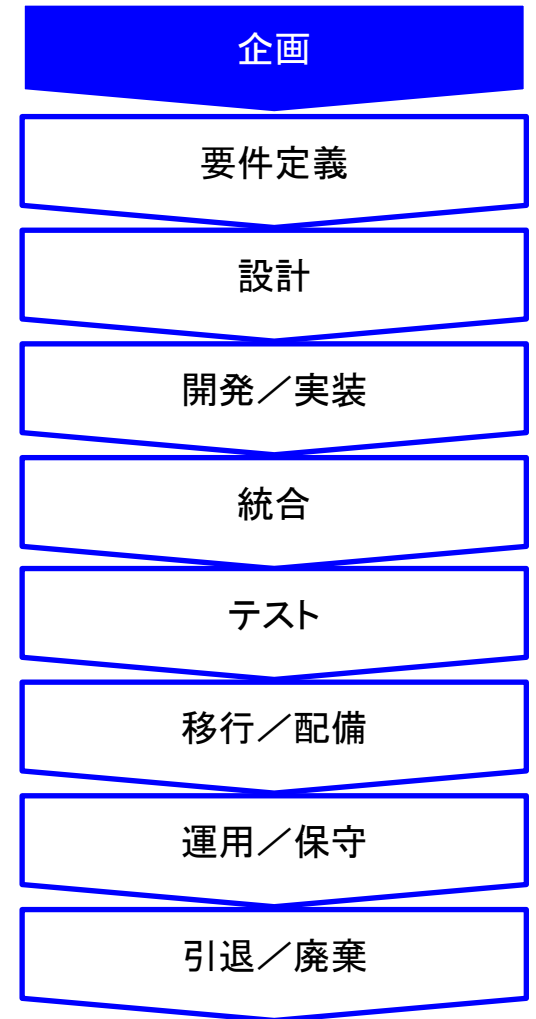
システムライフサイクル

システムライフサイクル

企画

●企画フェーズ

- 新しいシステムの開発または既存システムの改修を検討する最初の段階です。このフェーズは、システムの必要性を明確にし、プロジェクトの方向性を定めるために重要です。
- 主な目的は、ビジネスニーズを理解し、システムの目的、範囲、目標を設定することです。また、初期のコストと利益の分析を行い、プロジェクトの実現可能性を評価します。
- 成果物には、プロジェクト憲章や初期ビジネスケースが含まれます。

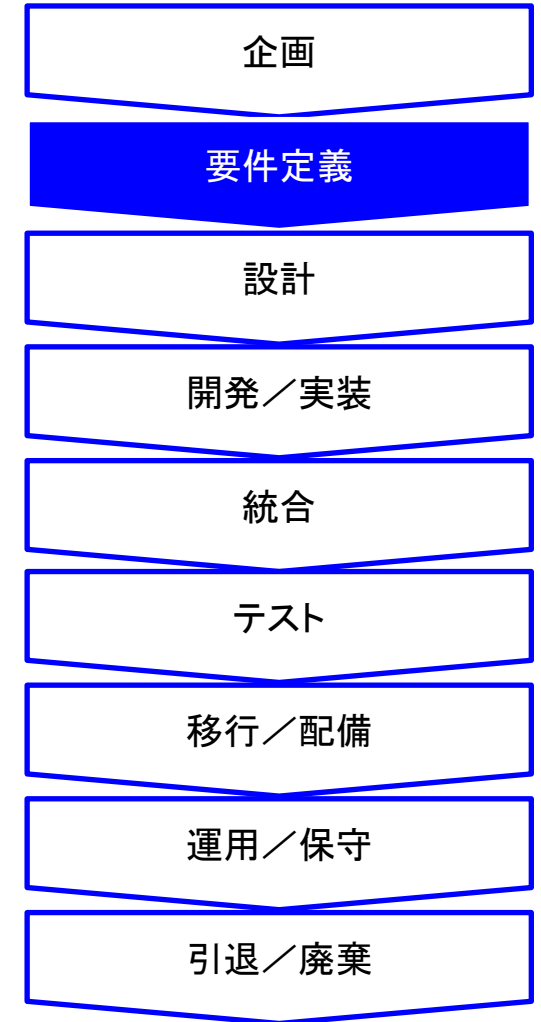


システムライフサイクル

要件定義

● 要件定義フェーズ

- システムが満たすべき要件を明確に定義し、文書化するフェーズです。
- 機能要件とは、システムが提供するべき具体的な機能やサービスを定義する要件です。
- 非機能要件とは、性能、セキュリティ、ユーザビリティなどサービスとして提供するものではないが、そのサービスを安全に継続運用するために求められる要件を指します。
- 成果物には、要件定義書が含まれます。

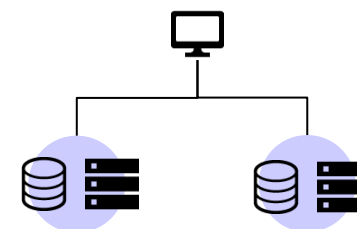


システムライフサイクル

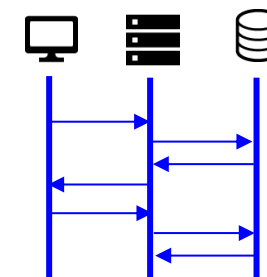
設計

● 設計フェーズ

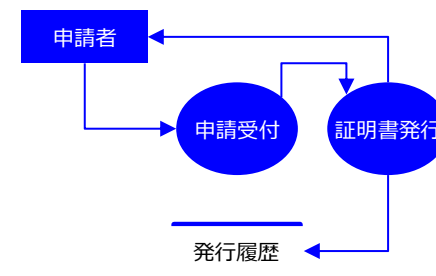
- システムの具体的な仕様や構造を定めるフェーズです。
- ハードウェア、ソフトウェア、ネットワークなどのシステムアーキテクチャの設計を行います。
- データベース設計やユーザインターフェースのデザインなどを含む各コンポーネントの具体的な動作やインターフェースを定義します。
- 成果物としては、システム設計書、詳細設計書、データフローダイアグラム（DFD）、エンティティ・リレーションシップ・ダイアグラム（ERD）などが作成されます。



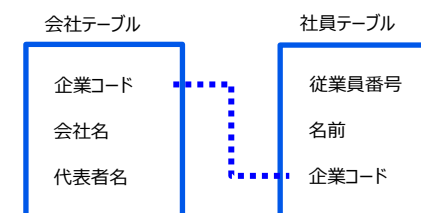
システム設計書



詳細設計書



データフローダイアグラム



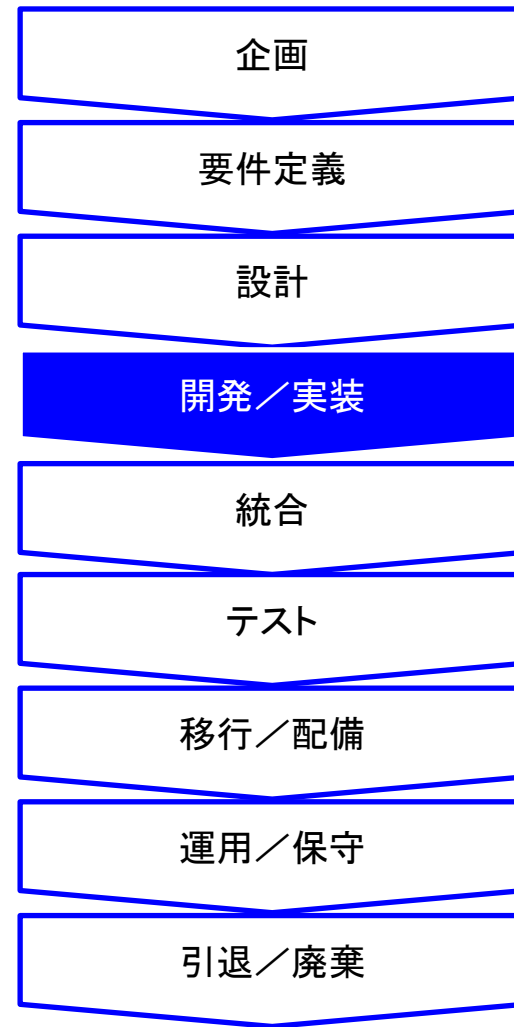
エンティティ・リレーションシップ・ダイアグラム

システムライフサイクル

開発／実装

● 開発／実装フェーズ

- 計画および設計段階で確立された要件を実際のシステムとして具体化する重要なフェーズです。
- ソフトウェアのコーディング、ハードウェアのセットアップが行われます。
- 開発では、設計仕様書に基づき、プログラマーがソースコードを作成します。コードはレビューとテストを経て品質を確保していきます。
- ユニットテストと呼ばれ、開発した個々のコンポーネントが正しく機能するかを確認します。バグの早期発見と修正が目的です。

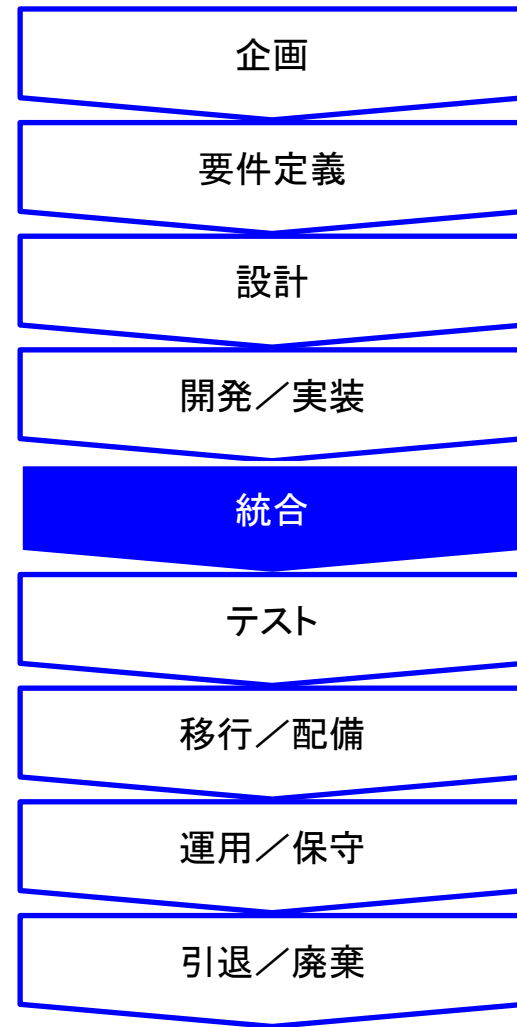


システムライフサイクル

統合

● 統合フェーズ

- 新しいシステムまたはシステムコンポーネントが一体となって機能するように組み合わせるフェーズです。
- 設計フェーズで認識を合わせた開発物を持ち寄って、相互に連携できるのかを確認します。
- 開発チーム、テストチーム、運用チームの緊密な協力が求められます。



システムライフサイクル

テスト

●テストフェーズ

単体テスト (Unit Testing)

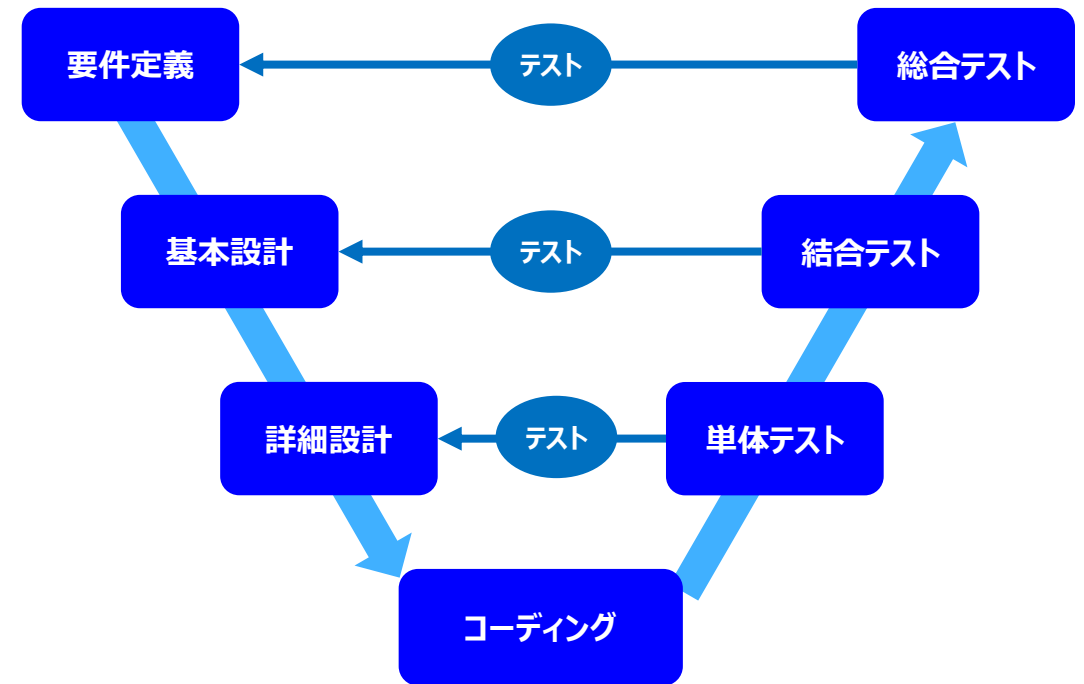
- 単体テストとは、開発したモジュールが単体で動くことを確認するテストです。モジュールとは、ソフトウェアの最小単位です。
- 詳細設計書で定義された内容に対するテストです。

結合テスト・機能テスト (Integration Testing)

- 結合テストとは、開発したモジュール同士を連結して全体動作を確認するテストです。
- 基本設計書で定義された内容に対するテストです。

総合テスト・システムテスト (System Testing)

- 総合テストとは、提示された要件通りに開発されているかを確認するテストです。



各設計フェーズに対するテスト種類

システムライフサイクル

テスト

● 受け入れテスト

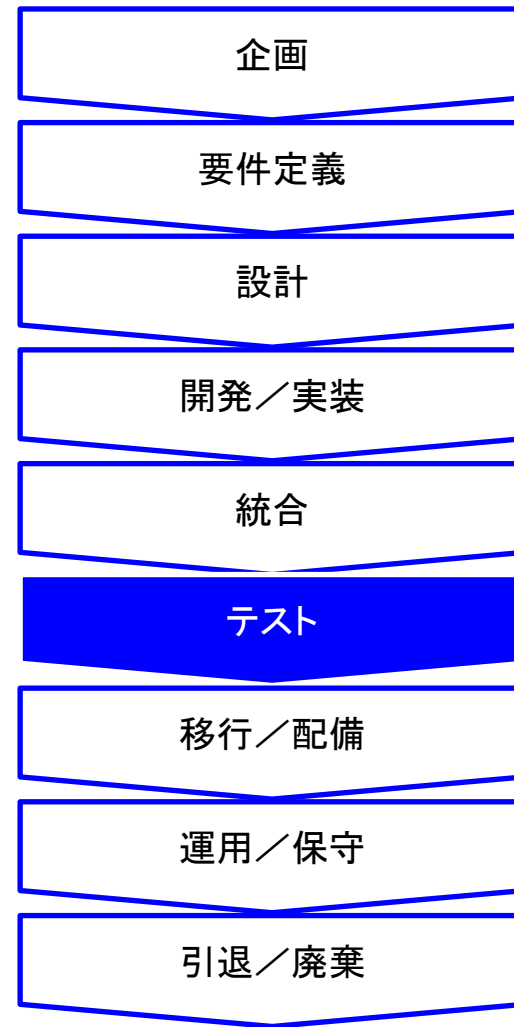
- 開発の最終段階では、受け入れテストを行い、ソフトウェアが想定通りに動作し、堅牢で安全であることを確認する必要があります。

ユーザー面での受け入れテスト

- ソフトウェアは、それを使用するユーザーにとって機能的であるかをユーザーとアプリケーションマネージャによってテストされます。

契約面の受け入れテスト

- ソフトウェアが契約の仕様を満たしているか。何を、どこで、どのように受け入れるかは、契約書で定義されています。

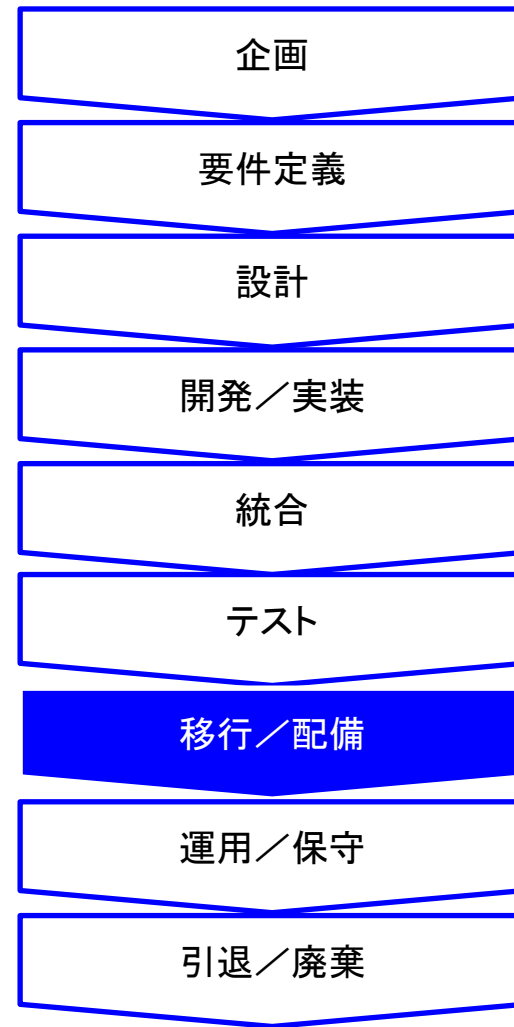


システムライフサイクル

移行／配備

● 移行／配備フェーズ

- 新しいシステムを検証環境から本番環境へ移行し、正式に運用を開始するフェーズです。
- 詳細な移行計画を策定し、移行の手順を明確にします。
- 回帰テストとは、アプリケーションの導入やアプリケーション機能の一部を修正したことで、他の箇所に不具合が出ていないかを確認するテストです。

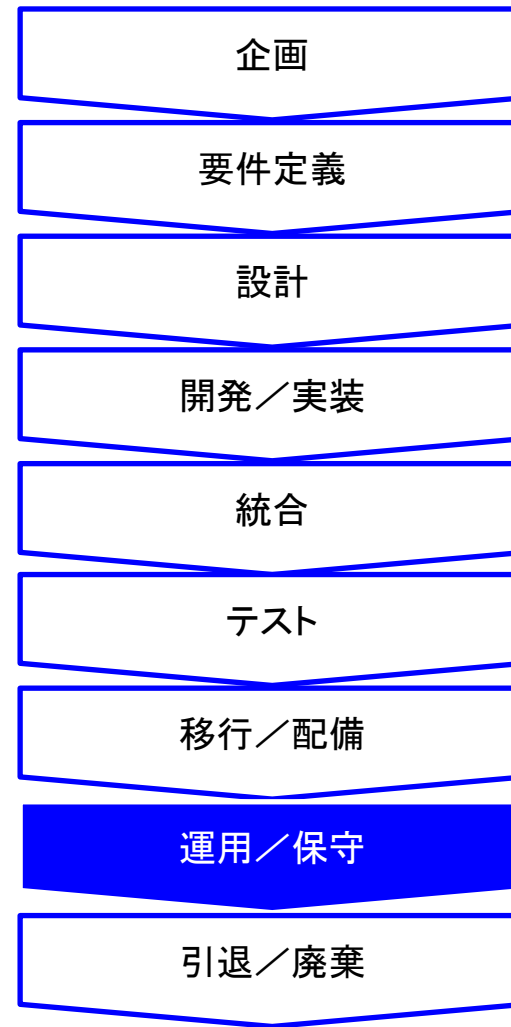


システムライフサイクル

運用／保守

●運用／保守フェーズ

- システムが導入され、実際に使用されるフェーズです。
- システムの安定した運用とパフォーマンスの維持が最重要課題となります。
- 公開されたシステムを安定的に稼働させるため、システムのパフォーマンスを最適化し、セキュリティパッチの適用やソフトウェアの更新を行います。

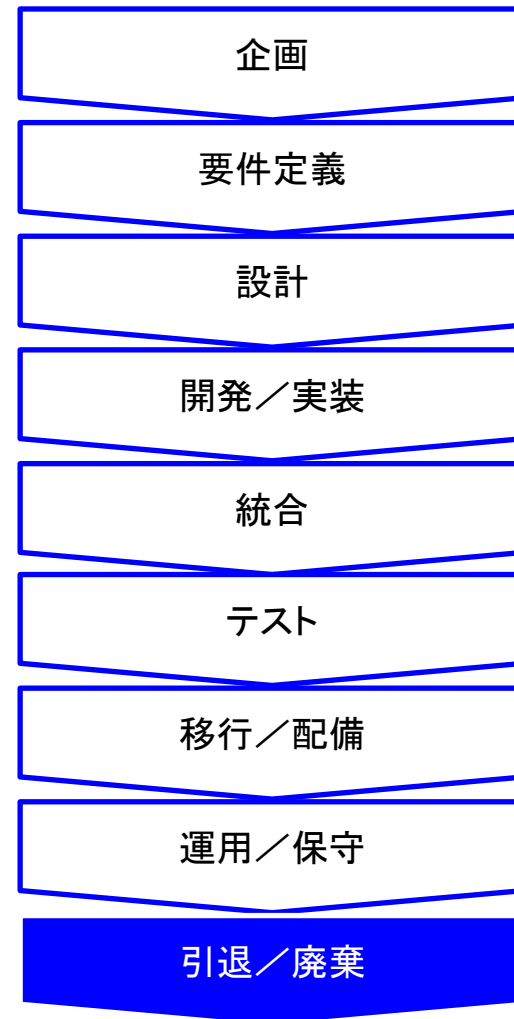


システムライフサイクル

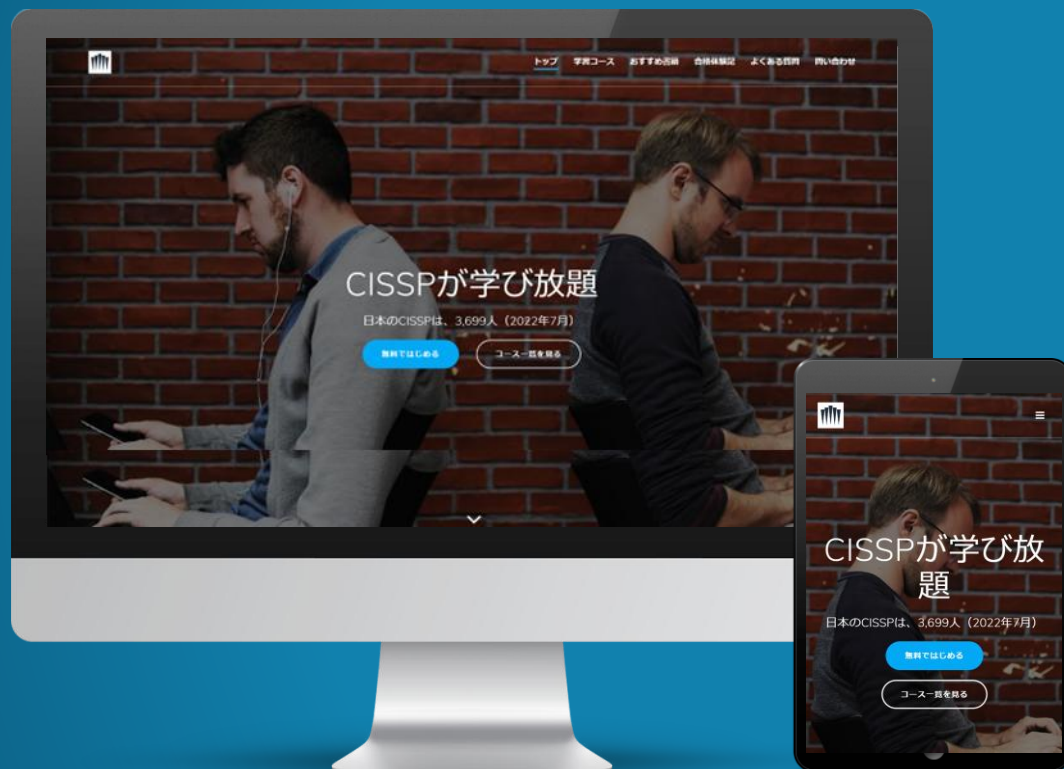
引退／廃棄

●引退／廃棄フェーズ

- システムが有用性を失い、運用を終了するフェーズです。
- データ保護法や業界の規制に従い、適切な引退／廃棄手順を確立します。
- 重要なデータを新しいシステムに移行し、必要に応じてバックアップを取ります。次に、個人情報のようなセンシティブなデータを適切な方法で完全に消去し、情報漏洩を防ぎます。また、使用済みのハードウェアは安全な方法で廃棄し、不要なソフトウェアはライセンス契約に基づき無効化やアンインストールを行います。



PIEDPINの紹介



なぜセキュリティを学ぶのか

セキュリティの最も大きな敵は、人はあらゆるものに慣れ、そのリスクを考えなくなる習性にあります。セキュリティの学びを通して得られる貴重なものは、あらゆる事象に対して好奇心を持つことであり、それは目まぐるしい現代を生きるのに大いに役立つでしょう。

- EASY DESIGN
- SUPER EFFICIENT
- EVERYWHERE

PIEDPINを選ぶ理由



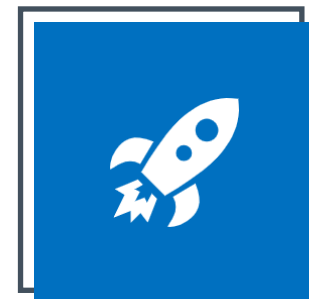
**EASY
DESIGN**

日本語で学べる



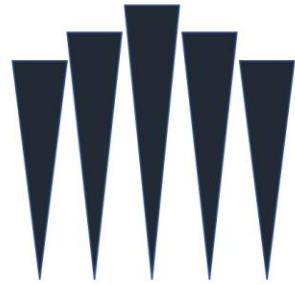
**SUPER
EFFICIENT**

わかりやすい説明



EVERYWHERE

どこからでも学べる環境



THANK YOU

Let's try CISSP!